

Digital Electronics & Systems

Lecture 1 Introduction

23ECE205

Digital Electronics & Systems

- **TEXT BOOK:** Stephen Brown, Zvonko Vranesic, “Fundamentals of Digital logic with Verilog Design”, Tata McGraw Hill Publishing Company Limited, Special Indian Edition, 2007.

Course Objectives

- To understand the fundamentals of Boolean Logic and the building blocks of digital circuits
- To introduce the abstraction of simple practical problems into Boolean Logic and their efficient implementation and to introduce the fundamentals of design with combinational and sequential subsystems

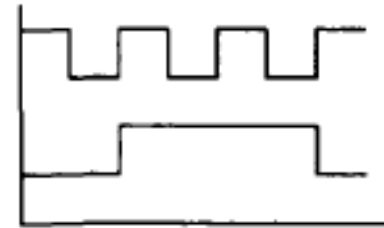
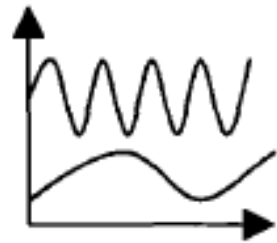
Course Outcomes

- **CO1:** Able to frame Boolean equations for solving a simple real-life problems and realize them using gate-level building blocks
- **CO2:** Able to apply minimization techniques for efficient Boolean logic implementation
- **CO3:** Able to realize digital blocks using combinational and sequential subsystems
- **CO4:** Able to design using state machine descriptions for practical real-life engineering problems

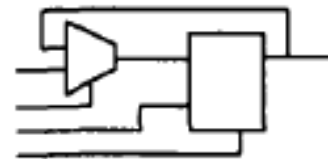
Evaluation Scheme

ANALOG VS DIGITAL

- Analog devices and systems: Process analog signals (time-varying signals that can take any value across a continuous range known as dynamic range)
- Digital devices and systems: Process digital signals (analog signals that are modeled as having at any time one of two discrete values)



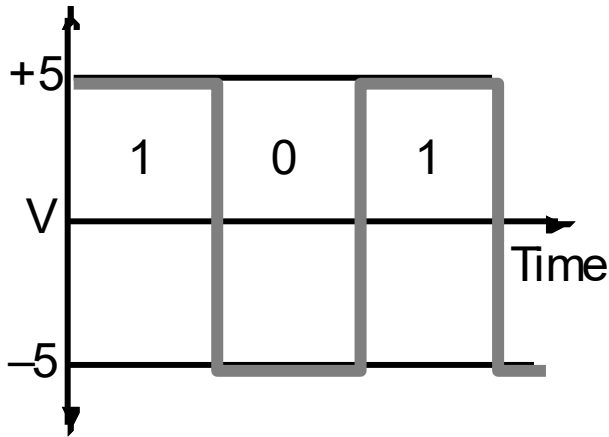
a. Analog Circuit.



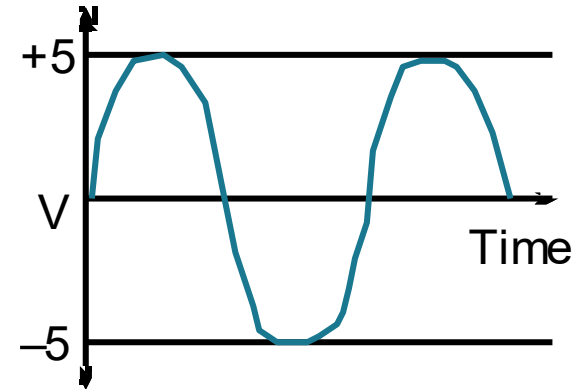
b. Digital Circuit.

Digital Systems

Digital vs. Analog Waveforms



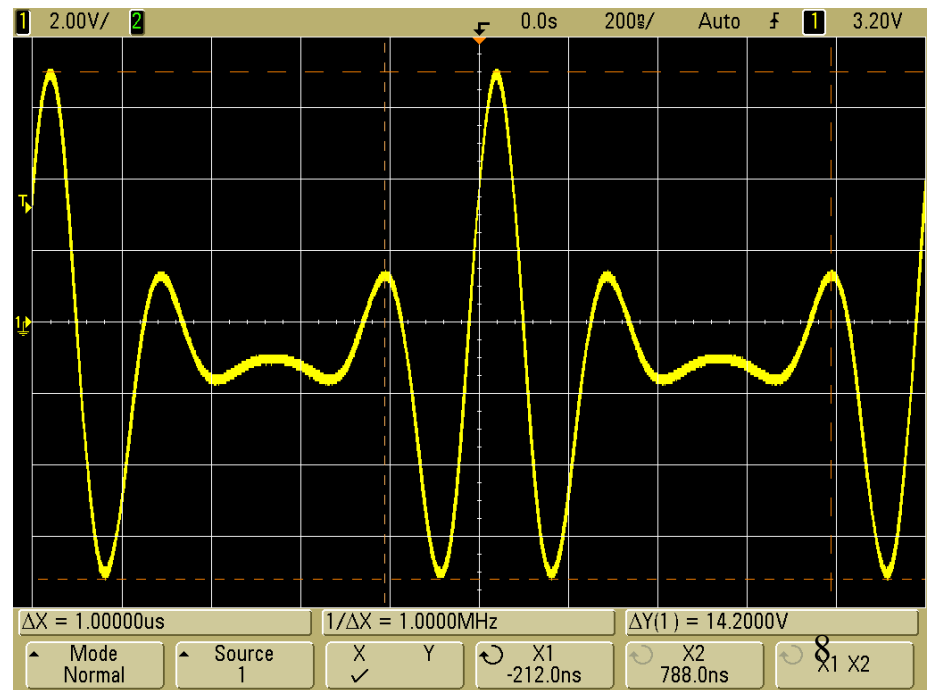
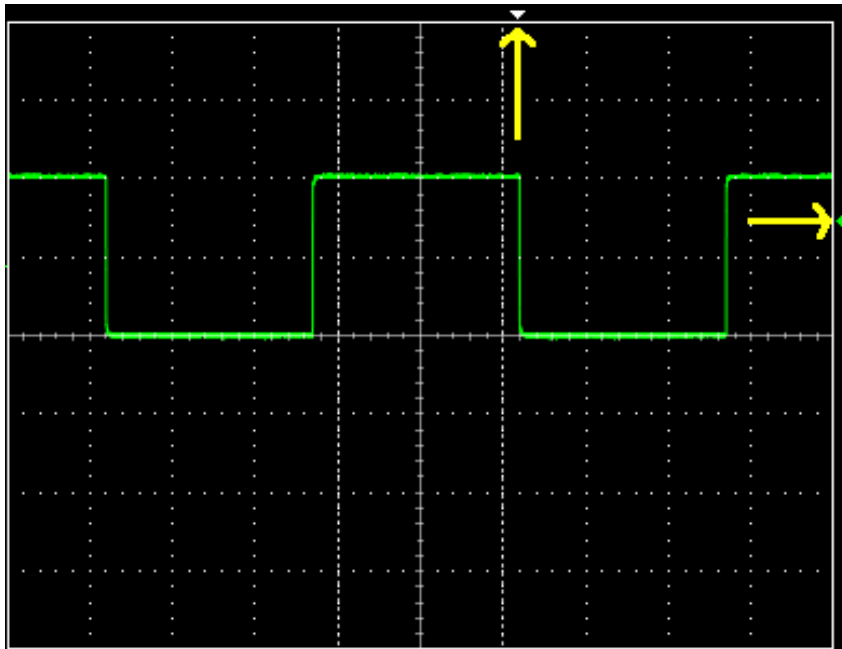
Digital:
only assumes discrete values



Analog:
values vary over a broad range
continuously

Digital Systems cntd..

- A digital **signal** is a signal that can only have discrete values in time
 - Most common are **binary** digital signals, where only two values are allowed often designated as 0 and 1
- The opposite is analog signals that can take infinite values



Digital Hardware Systems

Boolean Algebra and Logical Operators

Algebra: variables, values, operations

In Boolean algebra, the values are the symbols 0 and 1

If a logic statement is false, it has value 0

If a logic statement is true, it has value 1

Operations: AND, OR, NOT

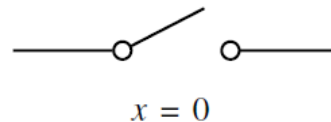
X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1

X	Y	X OR Y
0	0	0
0	1	1
1	0	1
1	1	1

X	NOT X
0	1
1	0

Variables & Functions

- Logic circuits form the foundation of digital systems
- Binary logic circuits perform operations of binary signals
- Let's consider the simplest element logic circuit: switch.
 - A switch can be in two states: “open” and “close”.
 - Graphical symbol for a switch



State of the switch can be given as a binary variable x

Representations of Digital Design: Switches

Normally Open

A switch connects two points under control signal.

- when the control signal is 0 (false), the switch is open
- when it is 1 (true), the switch is closed

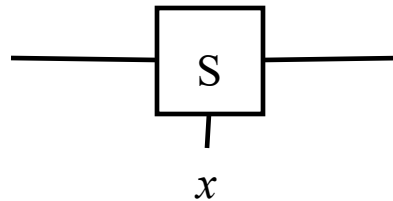
We will use Normally open switch in our discussion

A switch connects two points under control signal.

- when the control signal x is 0 (false), the switch is open
- when it is 1 (true), the switch is closed



(a) Two states of a switch



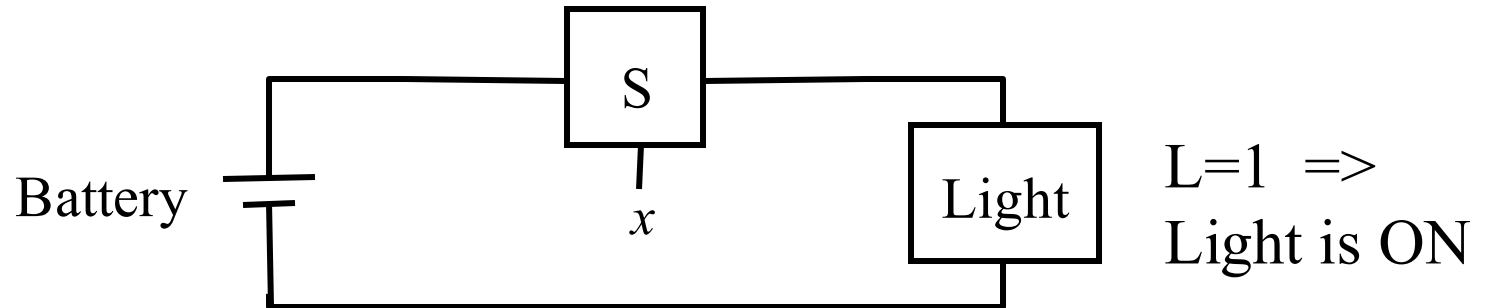
(b) Symbol for a switch

A simple application using a switch : “lightbulb”

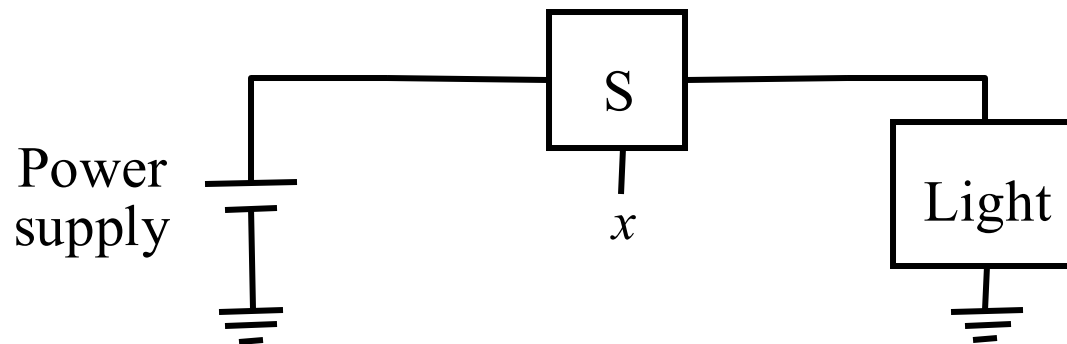
State of the switch can be given as a **binary variable** x s.t.:

- $x = 0$ when switch is open, and
- $x = 1$ when switch is closed.

Figure: A binary switch.



(a) Simple connection to a battery



(b) Using a ground connection as the return path

Figure : A light controlled by a switch.

Logic expression/Logic Function

State of Light is function of input variable x .

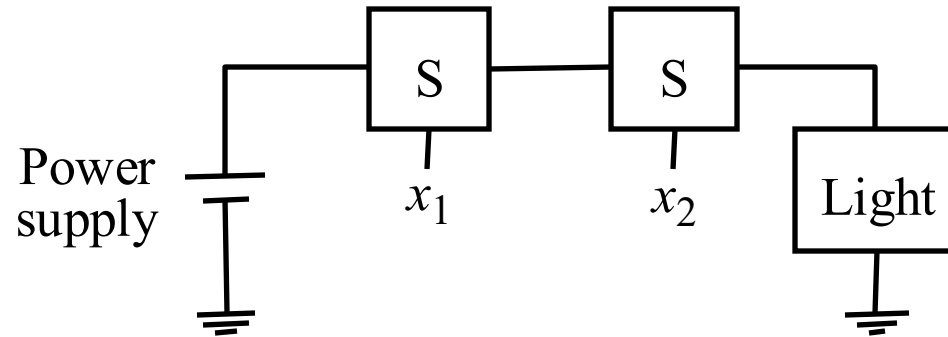
- $L=1$ if $x=1$
- $L=0$ if $x=0$

$$L(x)=x$$

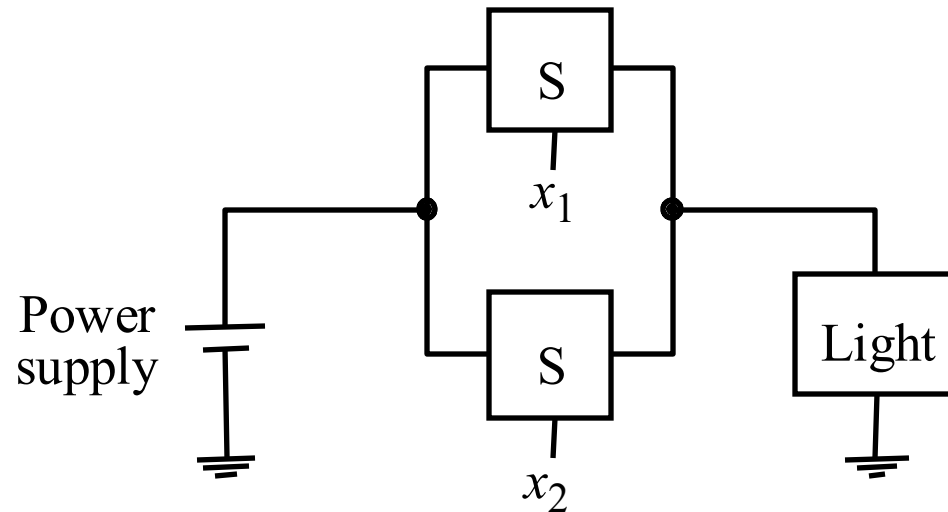
$L(x)=x$ is a logic expression/function

Here only one switch was controlling Light.

If there are two switches then?



(a) The logical AND function (series connection)



(b) The logical OR function (parallel connection)

Figure : Two basic functions.

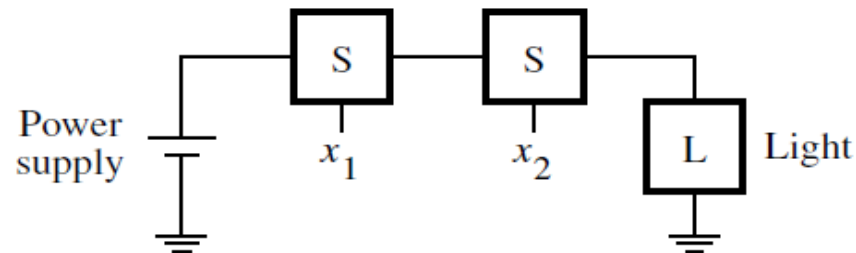
Series Connection $\Rightarrow$ logical AND operation

$$L(x_1, x_2) = x_1 \cdot x_2$$

where $L = 1$ if $x_1 = 1$ and $x_2 = 1$,
 $L = 0$ otherwise.

The symbol

- Is called **AND** operator



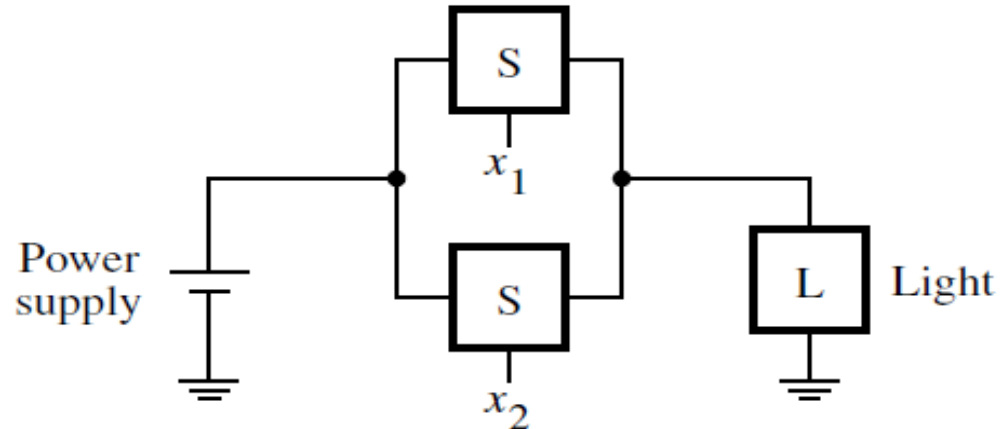
(a) The logical AND function (series connection)

Parallel Connection $\Rightarrow$ logical OR operation

where
$$L(x_1, x_2) = x_1 + x_2$$

 $L = 1$ if $x_1 = 1$ or $x_2 = 1$ or if $x_1 = x_2 = 1$,
 $L = 0$ if $x_1 = x_2 = 0$.

Symbol $+$ is
called **OR**
operator



(b) The logical OR function (parallel connection)

Here three switches are controlling the light in more complex way.
 Can you identify the relation?

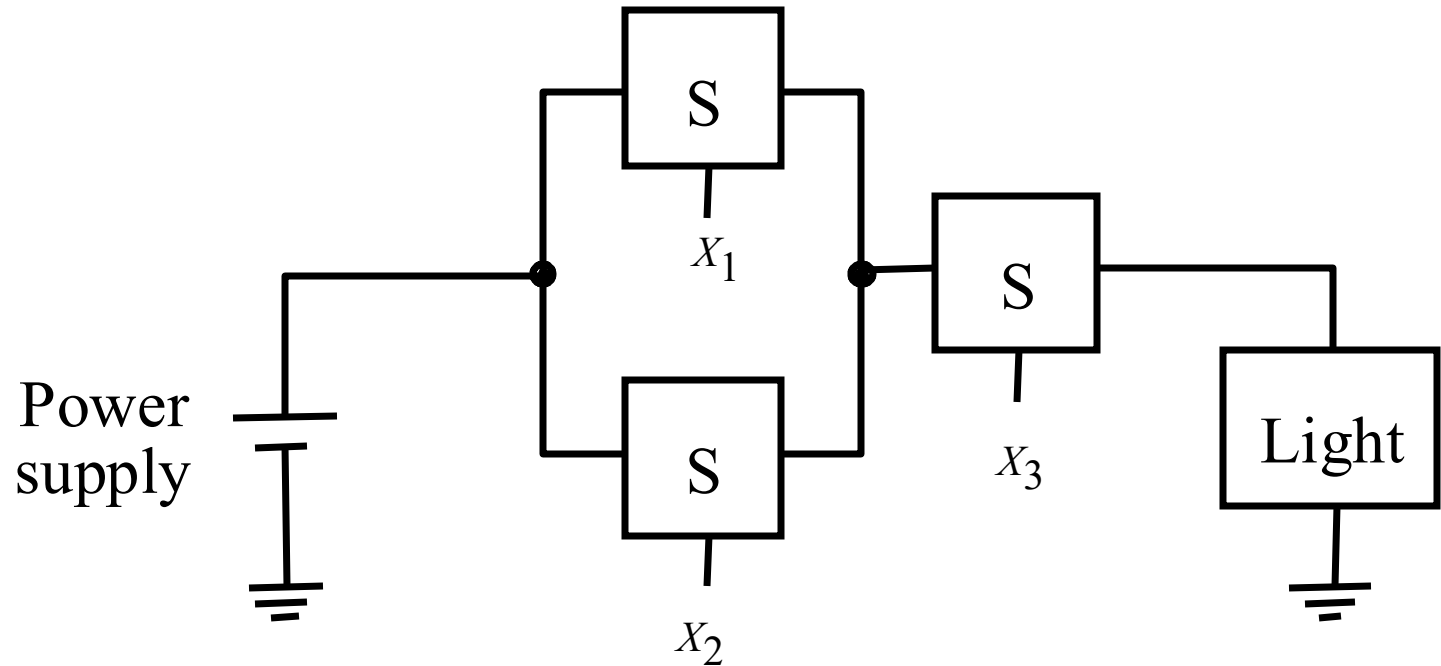
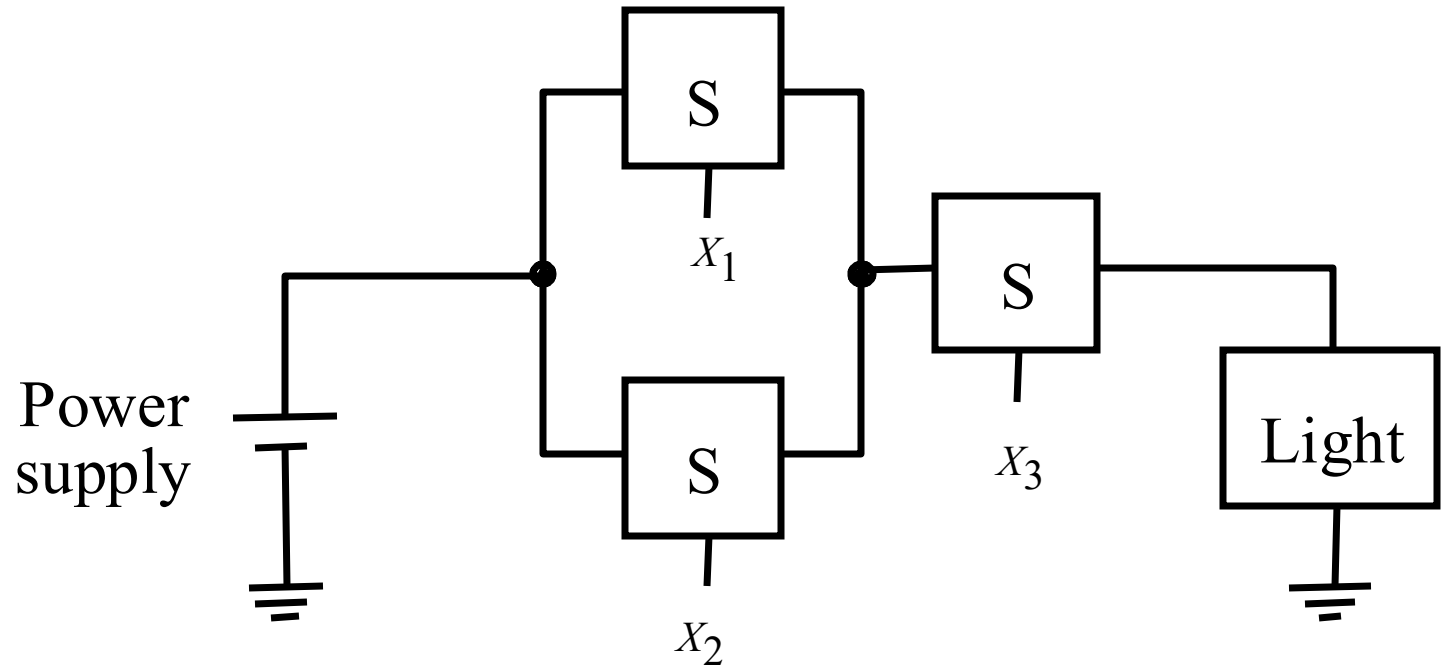


Figure: A series-parallel connection.

Here the series parallel connection of switches realizes the logic function

$$L(x_1, x_2, x_3) = (x_1 + x_2) \cdot x_3$$



The light is on if $x_3 = 1$ and, at the same time, at least one of the x_1 or x_2 inputs is equal to 1.

Figure : A series-parallel connection.

Try implementation using switches

- $(A+B).(C+D)$
- $A.(B+C).D$

LECTURE 2

- Till now we saw some positive action takes place when a switch is closed, such as turning the light on.
- It is equally interesting and useful to consider the possibility that a positive action takes place when a switch is opened

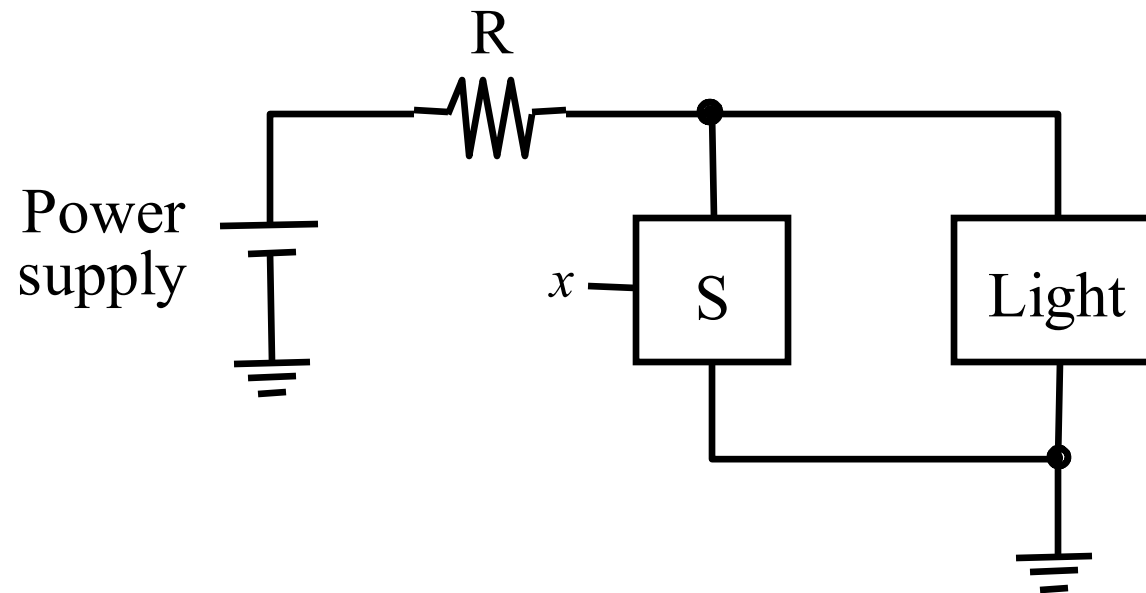


Figure : An inverting circuit.

Complement Operation

$L(x) = \bar{x}$
 where $L = 1$ if $x = 0$,
 $L = 0$ if $x = 1$

Symbol $\bar{}$ over a variable called **NOT** operator.

$$\bar{x} = x' = !x = \sim x$$

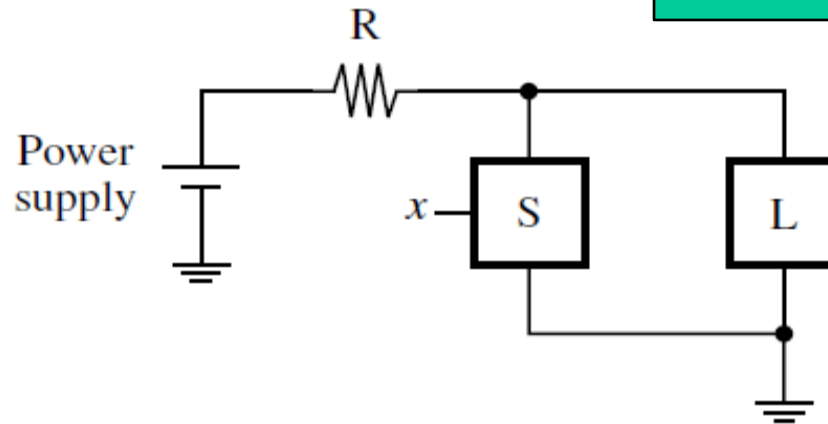


Figure 2.5 An inverting circuit.

Try implementation using switches

- $((A.B)+C).(D)$
- $(A+B)'$

Truth Tables

Useful aid for giving information for a logic function.

x_1	x_2	$x_1 \cdot x_2$	$x_1 + x_2$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

AND OR

Figure: A truth table for the AND and OR operations.

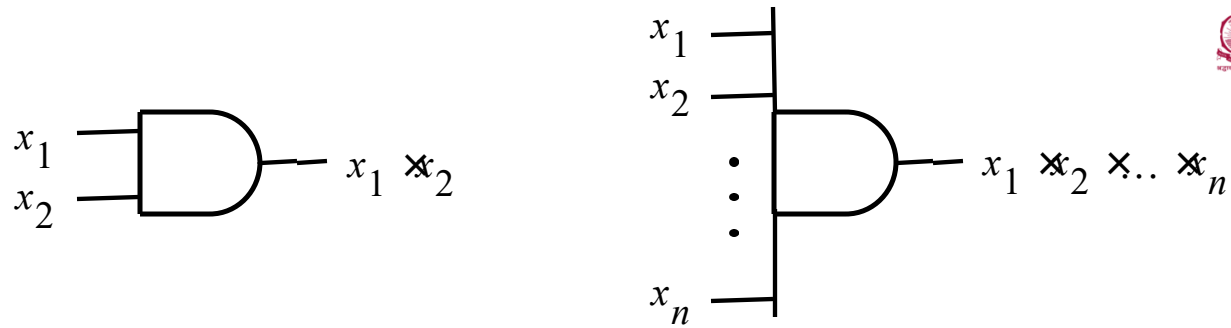
Truth Table for 3 input variables

x_1	x_2	x_3	$x_1 \cdot x_2 \cdot x_3$	$x_1 + x_2 + x_3$
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

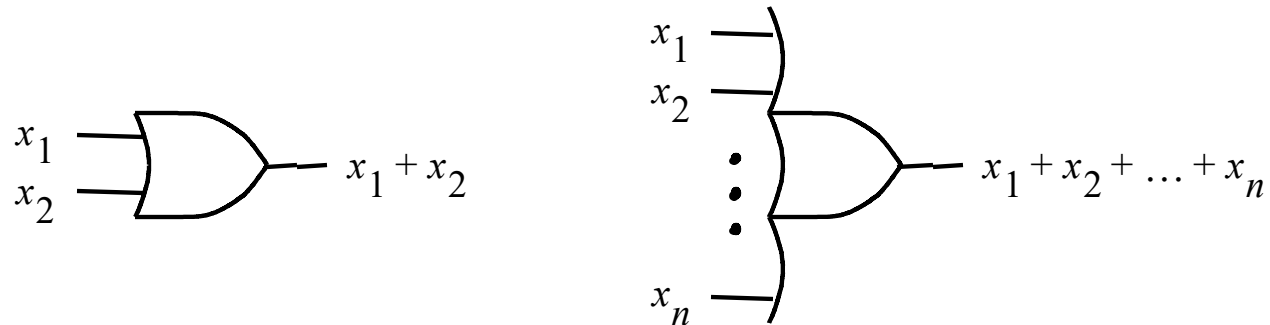
Figure : Three-input AND and OR operations.

Logic Gates

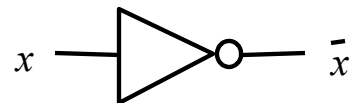
- ❖ Three basic logic operations can be used to implement logic functions of any complexity.
- ❖ Each logic operation can be implemented using Logic Gate.
- ❖ A logic gate has one or more inputs and one output that is a function of its inputs.
- ❖ It is often convenient to describe a logic circuit by drawing a circuit diagram, or *schematic*, consisting of graphical symbols representing the logic gates.



(a) AND gates



(b) OR gates



(c) NOT gate

Figure: The basic gates.

A Larger circuit is implemented by Network of Gates

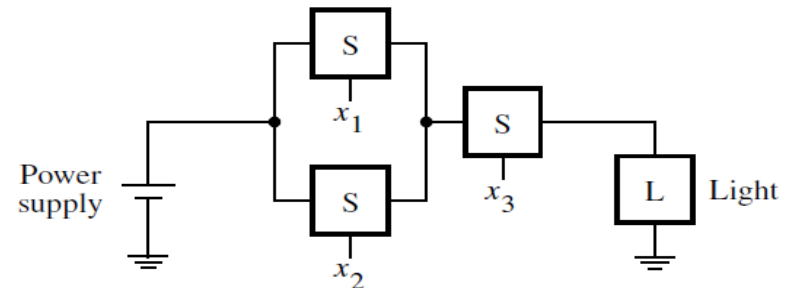
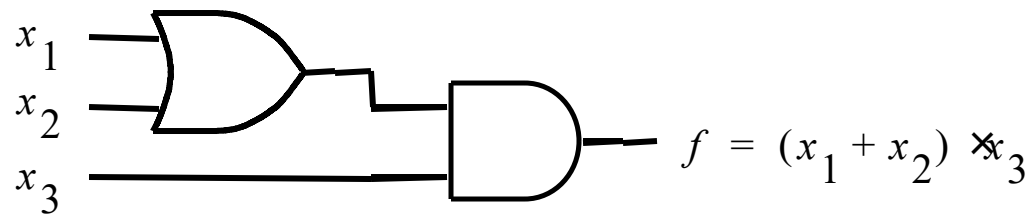
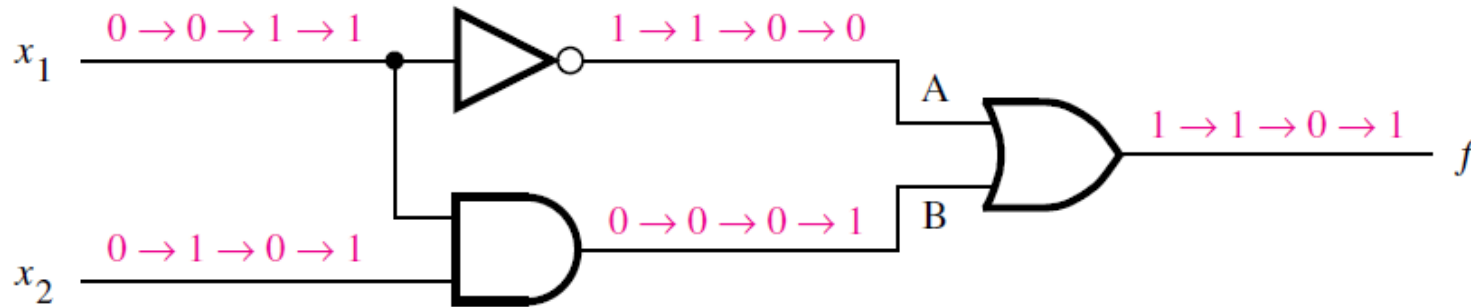


Figure: The function from Figure of switches .

Analysis vs Synthesis of Logic Networks

- Analysis Process: For an existing logic network, it must be possible to determine the function performed by the network. This task is referred to as the *analysis* process.
- Synthesis Process: The reverse task of designing a new network that implements a desired functional behavior is referred to as the *synthesis* process.
- The analysis process is straightforward and much simpler than the synthesis process

Analysis example



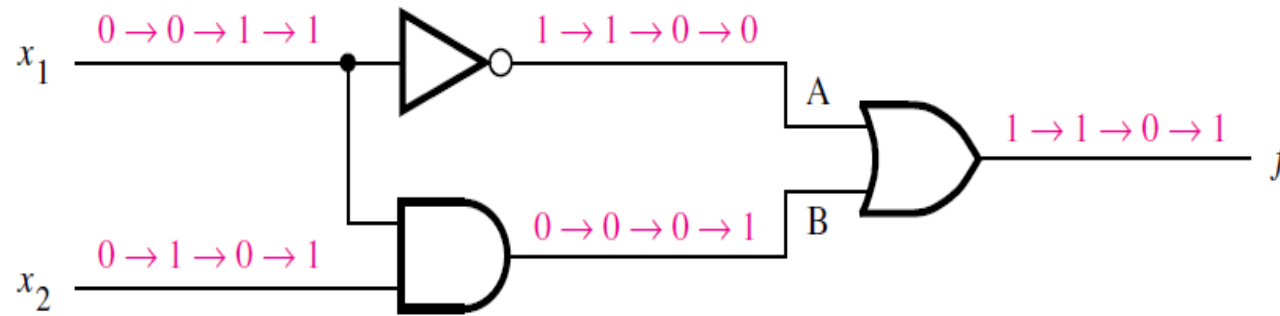
(a) Network that implements $f = \bar{x}_1 + x_1 \cdot x_2$

x_1	x_2	$f(x_1, x_2)$
0	0	1
0	1	1
1	0	0
1	1	1

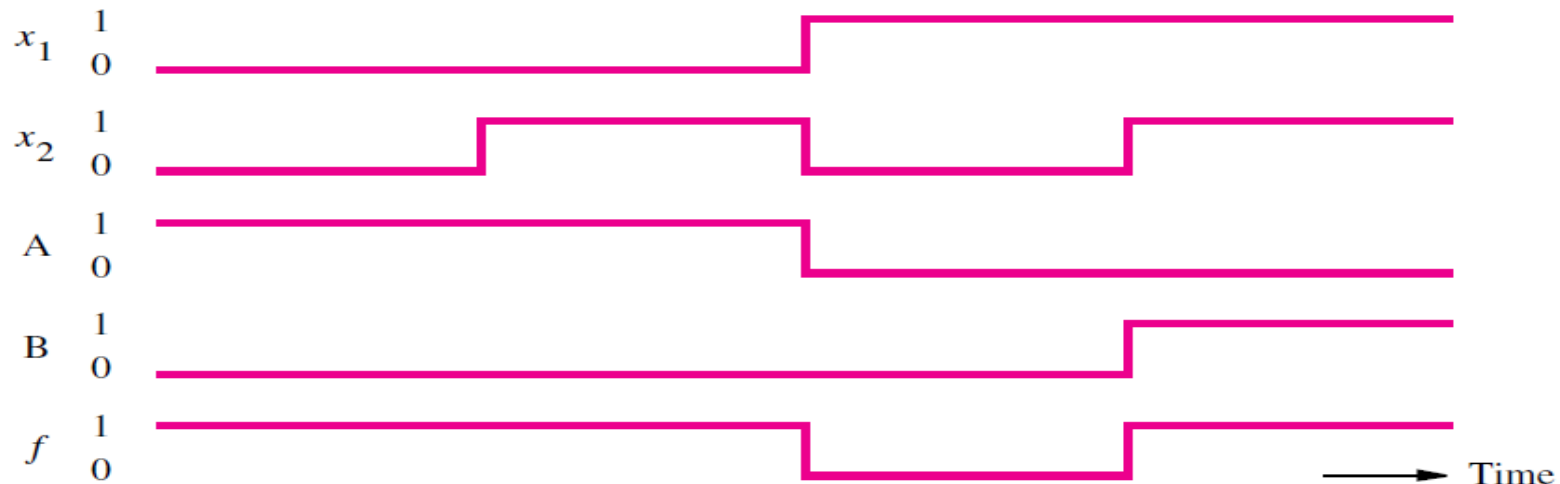
(b) Truth table for f

Check how signal A & B
Are changing with input
 x_1 & x_2

Timing Diagram: Functional Behavior



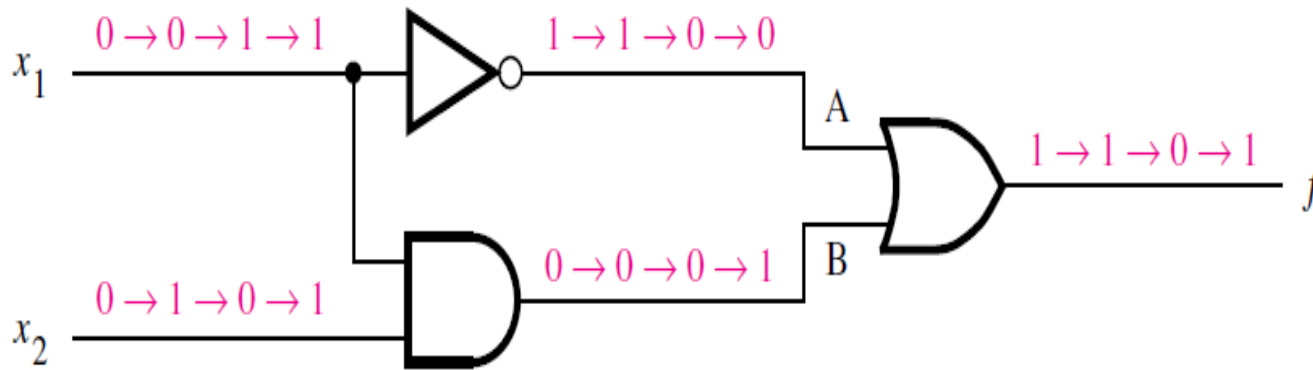
(a) Network that implements $f = \bar{x}_1 + x_1 \cdot x_2$



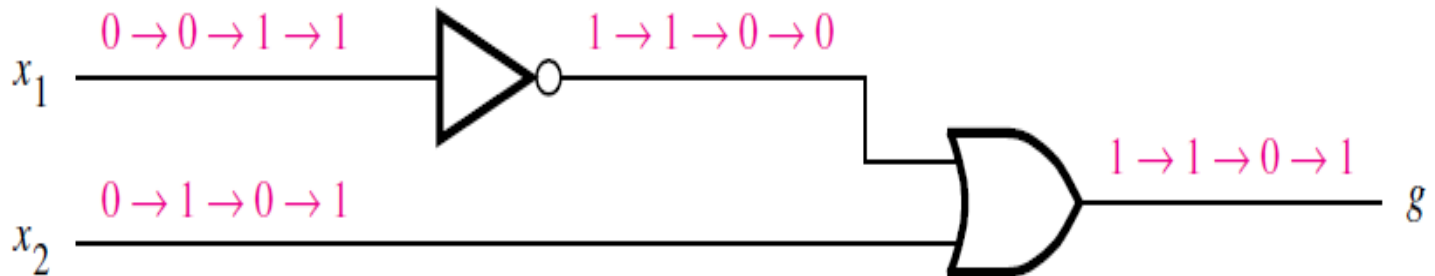
(c) Timing diagram

Check how signal A & B
Are changing with input
 x_1 & x_2

Functionally Equivalent Networks



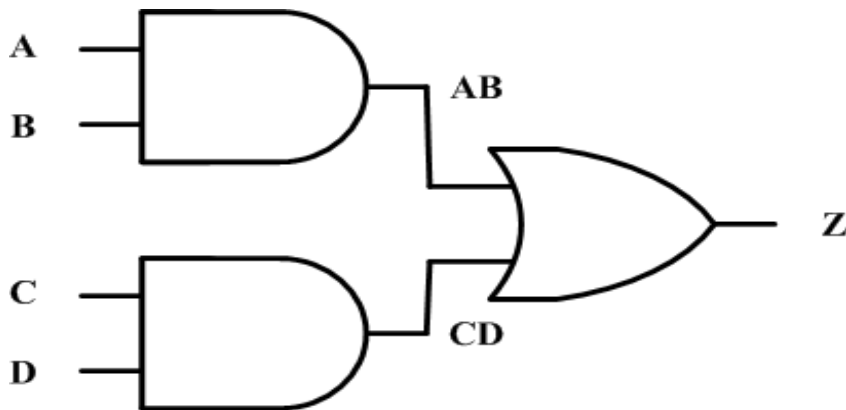
(a) Network that implements $f = \bar{x}_1 + x_1 \cdot x_2$



(d) Network that implements $g = \bar{x}_1 + x_2$

Truth table with 4 input variables

- we can also construct truth tables for any function.



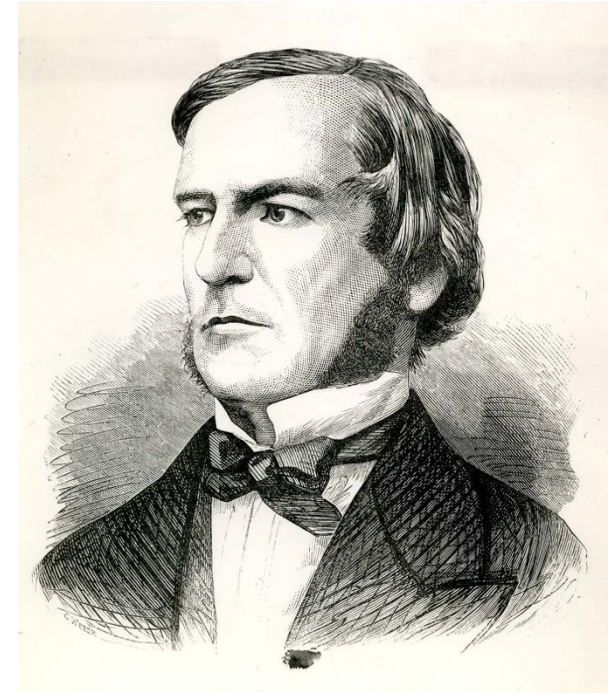
A	B	C	D	Z	AB	CD
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	0	1	1	1	0	1
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	1	1	1	0	1
1	0	0	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	0	0	0
1	0	1	1	1	0	1
1	1	0	0	1	1	0
1	1	0	1	1	1	0
1	1	1	0	1	1	0
1	1	1	1	1	1	1

Boolean Algebra

- Boolean Algebra
 - Axioms of Boolean Algebra
 - Single Variable Theorem
 - Two and Three variable Properties
 - Complements and Duals

Boolean Algebra

- In 1849 George Boole published a scheme for the algebraic description of processes involved in logical thought and reasoning . Subsequently, this scheme and its further refinements became known as *Boolean algebra*.
- It was almost 100 years later that this algebra found application in the engineering sense. In the late 1930s Claude Shannon showed that Boolean algebra provides an effective means of describing circuits built with switches .The algebra can be used to describe logic circuits.



Boolean Algebra: Axioms

$$1a \quad 0 \cdot 0 = 0 \quad (1)$$

$$1b \quad 1 + 1 = 1 \quad (2)$$

$$2a \quad 1 \cdot 1 = 1 \quad (3)$$

$$2b \quad 0 + 0 = 0 \quad (4)$$

$$3a \quad 0 \cdot 1 = 1 \cdot 0 = 0 \quad (5)$$

$$3b \quad 0 + 1 = 1 + 0 = 1 \quad (6)$$

$$4a \quad \text{If } x = 0 \quad \text{then} \quad \bar{x} = 1 \quad (7)$$

$$4b \quad \text{If } x = 1 \quad \text{then} \quad \bar{x} = 0 \quad (8)$$

Boolean Algebra: Single Variable Theorems

$$5a \quad x \cdot 0 = 0 \quad (9)$$

$$5b \quad x + 1 = 1 \quad (10)$$

$$6a \quad x \cdot 1 = x \quad (11)$$

$$6b \quad x + 0 = x \quad (12)$$

$$7a \quad x \cdot x = x \quad (13)$$

$$7b \quad x + x = x \quad (14)$$

$$8a \quad x \cdot \bar{x} = 0 \quad (15)$$

$$8b \quad x + \bar{x} = 1 \quad (16)$$

$$9 \quad \bar{\bar{x}} = x \quad (17)$$

Principle of Duality

- Given a logic expression, its *dual* is obtained by replacing all $+$ operators with $\cdot$ operators, and vice versa, and by replacing all 0s with 1s, and vice versa.
- The dual of any true statement (axiom or theorem) in Boolean algebra is also a true statement.

$$X + 0 = X$$

OR $+$

A	0	RESULT
0	0	0
1	0	1

$$X \cdot 1 = X$$

AND $\cdot$

A	1	RESULT
0	1	0
1	1	1

LECTURE 3

Boolean Algebra: Two & Three variable Properties

Commutativity:

$$10a \quad x \cdot y = y \cdot x \quad (18)$$

$$10b \quad x + y = y + x \quad (19)$$

Associativity:

$$11a \quad x \cdot (y \cdot z) = (x \cdot y) \cdot z \quad (20)$$

$$11b \quad x + (y + z) = (x + y) + z \quad (21)$$

Distributivity:

$$12a \quad x \cdot (y + z) = x \cdot y + x \cdot z \quad (22)$$

$$12b \quad x + y \cdot z = (x + y) \cdot (x + z) \quad (23)$$

Absorption:

$$13a \quad x + x \cdot y = x \quad (24)$$

$$13b \quad x \cdot (x + y) = x \quad (25)$$

Boolean Algebra: Two & Three variable Properties

Combining:

14a

$$x \cdot y + x \cdot \bar{y} = x \quad (26)$$

14b

$$(x + y) \cdot (x + \bar{y}) = x \quad (27)$$

Consensus:

17a

$$x \cdot y + y \cdot z + \bar{x} \cdot z = x \cdot y + \bar{x} \cdot z \quad (28)$$

17b

$$(x + y) \cdot (y + z) \cdot (\bar{x} + z) = (x + y) \cdot (\bar{x} + z) \quad (29)$$

Boolean Algebra: DeMorgans Theorem



Augustus De Morgan (27 June 1806 — 18 March 1871)

$$15a \quad \overline{x \cdot y} = \bar{x} + \bar{y} \quad (30)$$

$$15b \quad \overline{x + y} = \bar{x} \cdot \bar{y} \quad (31)$$

16a $x + \bar{x} \cdot y = x + y$

16b $x \cdot (\bar{x} + y) = x \cdot y$

- Huntington's basic postulates-5,8,10,12

$$x \cdot 0 = 0$$

$$x + 1 = 1$$

$$x \cdot \bar{x} = 0$$

$$x + \bar{x} = 1$$

$$x \cdot y = y \cdot x$$

$$x + y = y + x$$

$$x \cdot (y + z) = x \cdot y + x \cdot z$$

$$x + y \cdot z = (x + y) \cdot (x + z)$$

Proof of DeMorgan's Theorem using Truth Table

$$\overline{x \cdot y} = \bar{x} + \bar{y} \quad (30)$$

$$\overline{x + y} = \bar{x} \cdot \bar{y} \quad (31)$$

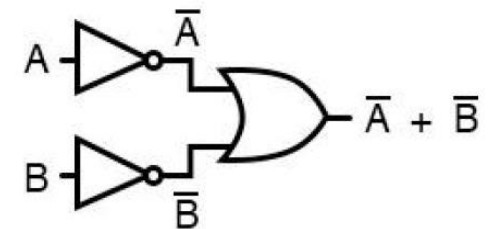
x	y	$x \cdot y$	$\overline{x \cdot y}$	$\bar{x}$	$\bar{y}$	$\bar{x} + \bar{y}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

$\underbrace{\quad\quad\quad}_{\text{LHS}}$

$\underbrace{\quad\quad\quad}_{\text{RHS}}$



... is equivalent to ...



$$\overline{AB} = \bar{A} + \bar{B}$$

Proof of DeMorgan's Theorem using Truth Table

$$\overline{x \cdot y} = \bar{x} + \bar{y} \quad (30)$$

$$\overline{x + y} = \bar{x} \cdot \bar{y} \quad (31)$$

X	Y	X+Y	$\overline{X+Y}$	X	Y	$\bar{X}$	$\bar{Y}$	$\bar{X} \cdot \bar{Y}$
0	0	0	1	0	0	1	1	1
0	1	1	0	0	1	1	0	0
1	0	1	0	1	0	0	1	0
1	1	1	0	1	1	0	0	0

Other examples

1. Proof of 13a(Absorption law)

$$X + XY = X \cdot 1 + XY = X(1+Y) = X \cdot 1 = X$$

• Use 6a 12a 5b

2. Proof of 16a

$$X+X'Y = (X+X')(X+Y) = 1 \cdot (X+Y) = X+Y$$

• Use 12b(distributive) 8b 6a

3. Proof

$$XY+XY' = X(Y + Y') = X \cdot 1 = X$$

• Use 12a(distributive) 8b 6a

Further Examples

$$4. X \cdot (X+Y) = X \cdot X + X \cdot Y = X + XY = X(1+Y) = X \cdot 1 = X$$

• Use 12b(distributive) 7a 12a 5b

$$5. (X+Y) \cdot (X+Y') = XX + XY' + XY + YY' =$$

$$X + XY' + XY + 0 = X(1+Y'+Y) = X \cdot 1 = X$$

$$6. X(X'+Y) = XX' + XY = 0 + XY = XY$$

$$\overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$

Factoring **BC** out of 1st and 4th terms

$$BC(\overline{A} + A) + A\overline{B}C + AB\overline{C}$$

Applying identity **A + $\overline{A}$ = 1**

$$BC(1) + A\overline{B}C + AB\overline{C}$$

Applying identity **1A = A**

$$BC + A\overline{B}C + AB\overline{C}$$

Factoring **B** out of 1st and 3rd terms

$$B(C + A\overline{C}) + A\overline{B}C$$

Applying rule **A + $\overline{A}B$ = A + B** to the C + $A\overline{C}$ term

$$B(C + A) + A\overline{B}C$$

Distributing terms

$$BC + AB + A\overline{B}C$$

Factoring **A** out of 2nd and 3rd terms

$$BC + A(B + \overline{B}C)$$

Applying rule **A + $\overline{A}B$ = A + B** to the B + $\overline{B}C$ term

$$BC + A(B + C)$$

Distributing terms

$$BC + AB + AC$$

or

Simplified result

$$AB + BC + AC$$

Proof of Consensus Theorem

- Consensus Theorem gives us the relationship

$$XY + X'Z + YZ = XY + X'Z$$

- Note that in doing the reduction the first step is to add a 1 to the YZ term. That 1 is in the form, $(X+X')$.
- Try it.

The consensus theorem can be stated as follows:

$$XY + X'Z + YZ = XY + X'Z$$

Proof

$$\begin{aligned} XY + X'Z + YZ &\rightarrow XY + X'Z + (X + X')YZ \\ &\rightarrow XY + X'Z + XYZ + X'YZ \\ &\rightarrow (XY + XYZ) + (X'Z + X'YZ) \\ &\rightarrow XY(1 + Z) + X'Z(1 + Y) \\ &\rightarrow XY + X'Z \end{aligned}$$

Consensus theorem Proof–POS form

$$(A + B)(B' + C)(A + C) = (A + B)(B' + C)$$

$$\begin{aligned}\text{LHS} &= (A + B)(B' + C)(A + C) \\ &= (A + B)(B' + C)(A + C + \mathbf{0}) \\ &= (A + B)(B' + C)(A + \mathbf{B.B'} + C) \\ &= (A + B)(B' + C)(\mathbf{A + B + C})(\mathbf{A + B' + C}) \\ &= [(A + B)(A + B + C)] [(B' + C)(A + B' + C)] \\ &= (A + B)(B' + C) = \mathbf{RHS}\end{aligned}$$

Examples

$$\begin{aligned} VX'Y + WXZ + VWYZ &= \\ \textcolor{blue}{V}X'\textcolor{blue}{Y} + \textcolor{red}{W}X\textcolor{red}{Z} + \textcolor{blue}{V}\textcolor{blue}{W}\textcolor{blue}{Y}\textcolor{blue}{Z} &= \\ &= \textcolor{blue}{V}X'\textcolor{blue}{Y} + \textcolor{red}{W}X\textcolor{red}{Z} \end{aligned}$$

$$\begin{aligned} (A + B + E') (E + F + G') (A + B + F + G') &= \\ (\textcolor{blue}{A} + \textcolor{blue}{B} + E') (E + \textcolor{red}{F} + \textcolor{red}{G}') (\textcolor{blue}{A} + \textcolor{blue}{B} + \textcolor{red}{F} + \textcolor{red}{G}') &= \\ &= (A + B + E') (E + F + G) \end{aligned}$$

Simplify

- Simplify $F = X'YZ + X'Y\bar{Z} + XZ$

$$F = \bar{X}YZ + \bar{X}Y\bar{Z} + XZ$$

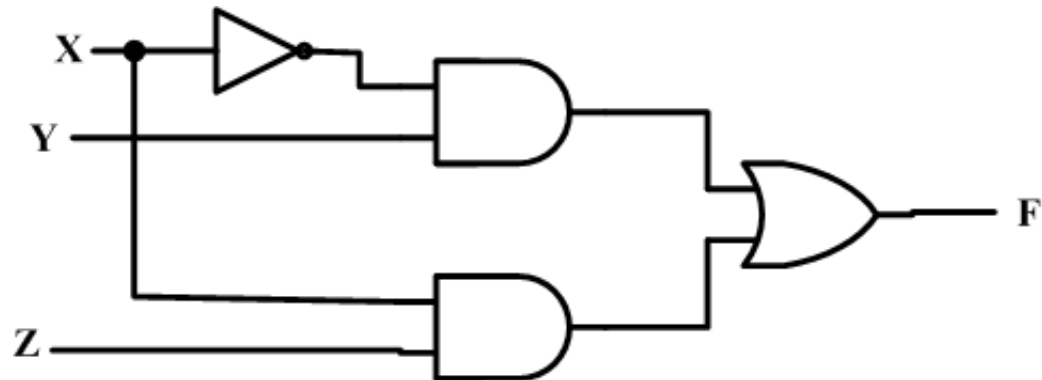
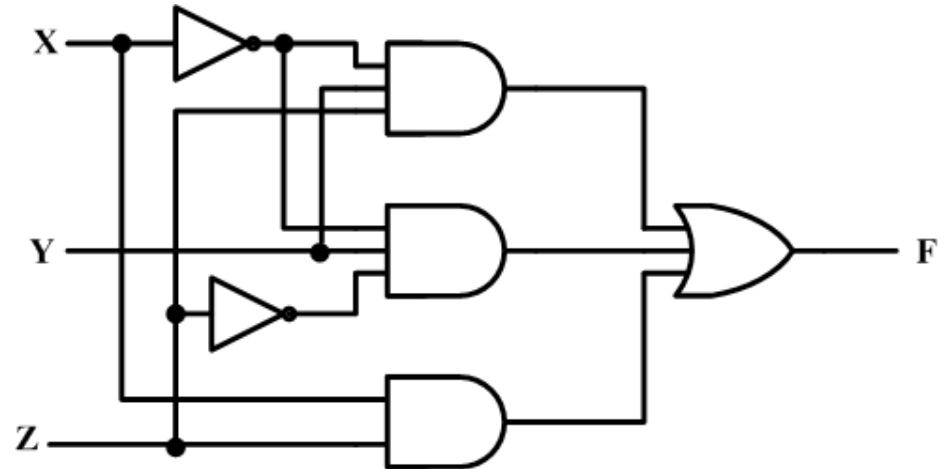
$$= \bar{X}Y(Z + \bar{Z}) + XZ \quad \text{by identity 14}$$

$$= \bar{X}Y \cdot 1 + XZ \quad \text{by identity 7}$$

$$= \bar{X}Y + XZ \quad \text{by identity 2}$$

Affect on implementation

- $F = X'YZ + X'YZ' + XZ$
- Reduces to $F = X'Y + XZ$



Let us prove the validity of the logic equation

$$(x_1 + x_3) \cdot (\bar{x}_1 + \bar{x}_3) = x_1 \cdot \bar{x}_3 + \bar{x}_1 \cdot x_3$$

The left-hand side can be manipulated as follows. Using the distributive property, 12a, gives

$$\text{LHS} = (x_1 + x_3) \cdot \bar{x}_1 + (x_1 + x_3) \cdot \bar{x}_3$$

Applying the distributive property again yields

$$\text{LHS} = x_1 \cdot \bar{x}_1 + x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3 + x_3 \cdot \bar{x}_3$$

Note that the distributive property allows ANDing the terms in parenthesis in a way analogous to multiplication in ordinary algebra. Next, according to theorem 8a, the terms $x_1 \cdot \bar{x}_1$ and $x_3 \cdot \bar{x}_3$ are both equal to 0. Therefore,

$$\text{LHS} = 0 + x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3 + 0$$

From 6b it follows that

$$\text{LHS} = x_3 \cdot \bar{x}_1 + x_1 \cdot \bar{x}_3$$

Finally, using the commutative property, 10a and 10b, this becomes

$$\text{LHS} = x_1 \cdot \bar{x}_3 + \bar{x}_1 \cdot x_3$$

which is the same as the right-hand side of the initial equation.

- Try

$$x_1 \cdot \bar{x}_3 + \bar{x}_2 \cdot \bar{x}_3 + x_1 \cdot x_3 + \bar{x}_2 \cdot x_3 = \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot x_2 + x_1 \cdot \bar{x}_2$$

- Solution:

$$x_1 \cdot \bar{x}_3 + \bar{x}_2 \cdot \bar{x}_3 + x_1 \cdot x_3 + \bar{x}_2 \cdot x_3 = \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot x_2 + x_1 \cdot \bar{x}_2$$

The left-hand side can be manipulated as follows

$$\begin{aligned} \text{LHS} &= x_1 \cdot \bar{x}_3 + x_1 \cdot x_3 + \bar{x}_2 \cdot \bar{x}_3 + \bar{x}_2 \cdot x_3 && \text{using } 10b \\ &= x_1 \cdot (\bar{x}_3 + x_3) + \bar{x}_2 \cdot (\bar{x}_3 + x_3) && \text{using } 12a \\ &= x_1 \cdot 1 + \bar{x}_2 \cdot 1 && \text{using } 8b \\ &= x_1 + \bar{x}_2 && \text{using } 6a \end{aligned}$$

The right-hand side can be manipulated as

$$\begin{aligned} \text{RHS} &= \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot (x_2 + \bar{x}_2) && \text{using } 12a \\ &= \bar{x}_1 \cdot \bar{x}_2 + x_1 \cdot 1 && \text{using } 8b \\ &= \bar{x}_1 \cdot \bar{x}_2 + x_1 && \text{using } 6a \\ &= x_1 + \bar{x}_1 \cdot \bar{x}_2 && \text{using } 10b \\ &= x_1 + \bar{x}_2 && \text{using } 16a \end{aligned}$$

Complement of a function

- In real implementation sometimes the complement of a function is needed.
 - Have $F = X'YZ' + X'Y'Z$

$$F = \overline{X}Y\overline{Z} + \overline{X}\overline{Y}Z$$

$$\overline{F} = \overline{\overline{X}Y\overline{Z} + \overline{X}\overline{Y}Z}$$

$$= (\overline{\overline{X}Y\overline{Z}}) \cdot (\overline{\overline{X}\overline{Y}Z})$$

$$= (X + \overline{Y} + Z) \cdot (X + Y + \overline{Z})$$

How Duals and Complement are related

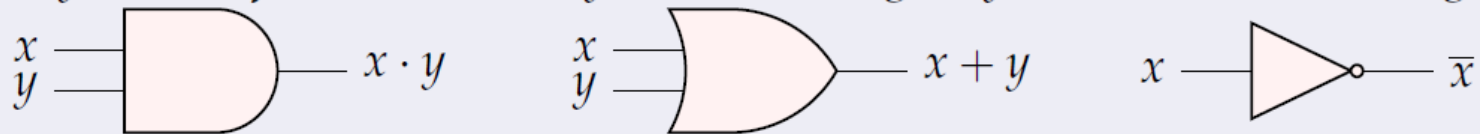
- What is meant by the dual of a function?
 - The *dual* of a function is obtained by interchanging OR and AND operations and replacing 1s and 0s with 0s and 1s.
- Shortcut to getting function complement
 - Starting with the equation on the previous slide
 - Generate the dual $F=(X'+Y+Z')(X'+Y'+Z)$
 - Complement each literal to get:
 - $F'=(X+Y'+Z)(X+Y+Z')$

LECTURE 4

Synthesis of simple circuits

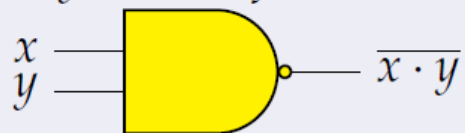
Theorem

Any Boolean function can be synthesized using only AND, OR, and NOT gates.



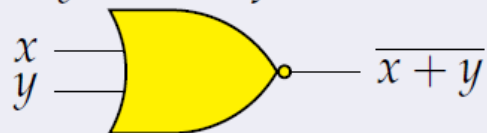
Theorem

Any Boolean function can be synthesized using only NAND gates.



Theorem

Any Boolean function can be synthesized using only NOR gates.



Synthesis using AND, OR, NOT Gates

Given a Boolean function f given in the form of a truth table, the expression that realizes f can be obtained:

- (SUM-OF-PRODUCTS) by considering rows for which $f = 1$, or
- (PRODUCTS-OF-SUMS) by considering rows for which $f = 0$.

Sum of Products Canonical form

- For a function of n variables a **minterm** is a product term in which each of the variable occur exactly once. For example: $x_0 \cdot x_1 \cdot \overline{x_2}$ and $\overline{x_0} \cdot \overline{x_1} \cdot x_2$.
- A function f can be represented by an expression that is sum of **minterms**.
- A logical expression that consists of product terms that are summed (ORed) together is called a **sum-of-product** form.
- If each of the product term is a minterm, then the expression is called **canonical sum-of-product** expression.
- Notice that two logically equivalent functions will have the same canonical representation.

Two Variable Minterms

Minterms			
x	y	term	notation
0	0	$x'.y'$	m0
0	1	$x'.y$	m1
1	0	$x.y'$	m2
1	1	$x.y$	m3

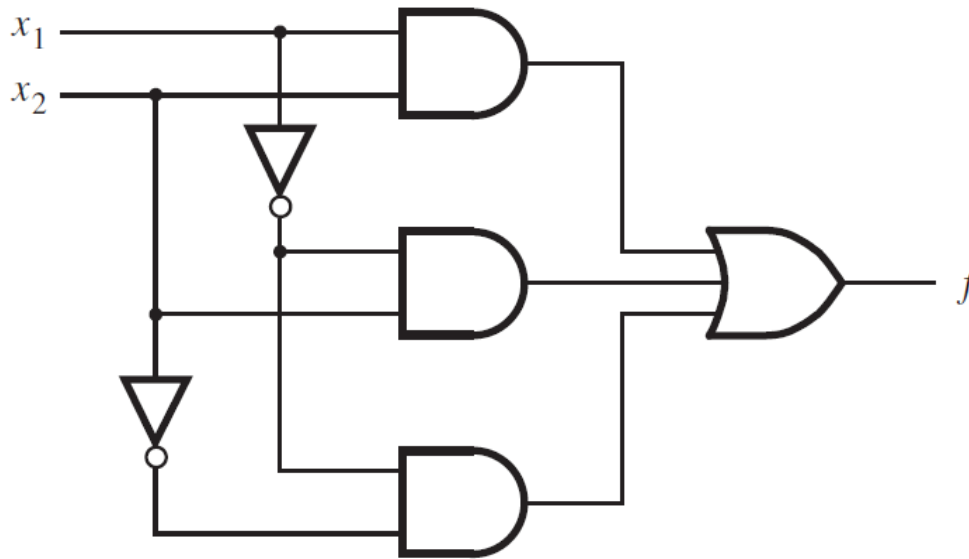
Simple problem of synthesis

- Assume that x_1 and x_2 represent the states of two switches, either of which may produce a 0 or 1.
- The function of the circuit is to continuously monitor the state of the switches and to produce an output logic value 1 whenever the switches (x_1, x_2) are in states $(0, 0)$, $(0, 1)$, or $(1, 1)$. If the state of the switches is $(1, 0)$, the output should be 0.
- We can express the required behavior using a truth table,

x_1	x_2	$f(x_1, x_2)$
0	0	1 ← $\bar{x}_1\bar{x}_2$
0	1	1 ← $\bar{x}_1x_2$
1	0	0
1	1	1 ← x_1x_2

$$f(x_1, x_2) = x_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2$$

Canonical SOP –implementation using Logic Gates



(a) Canonical sum-of-products

$$f(x_1, x_2) = x_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2$$

The process whereby we begin with a description of the desired functional behavior and then generate a circuit that realizes this behavior is called *synthesis*.

Canonical SOP expression can be reduced using Boolean Algebra

$$f(x_1, x_2) = x_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2$$

1. Using 7b, third Product term is replicated

$$f(x_1, x_2) = x_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2 + \bar{x}_1x_2$$

2. Using the commutative property 10b to interchange the second and third product terms gives

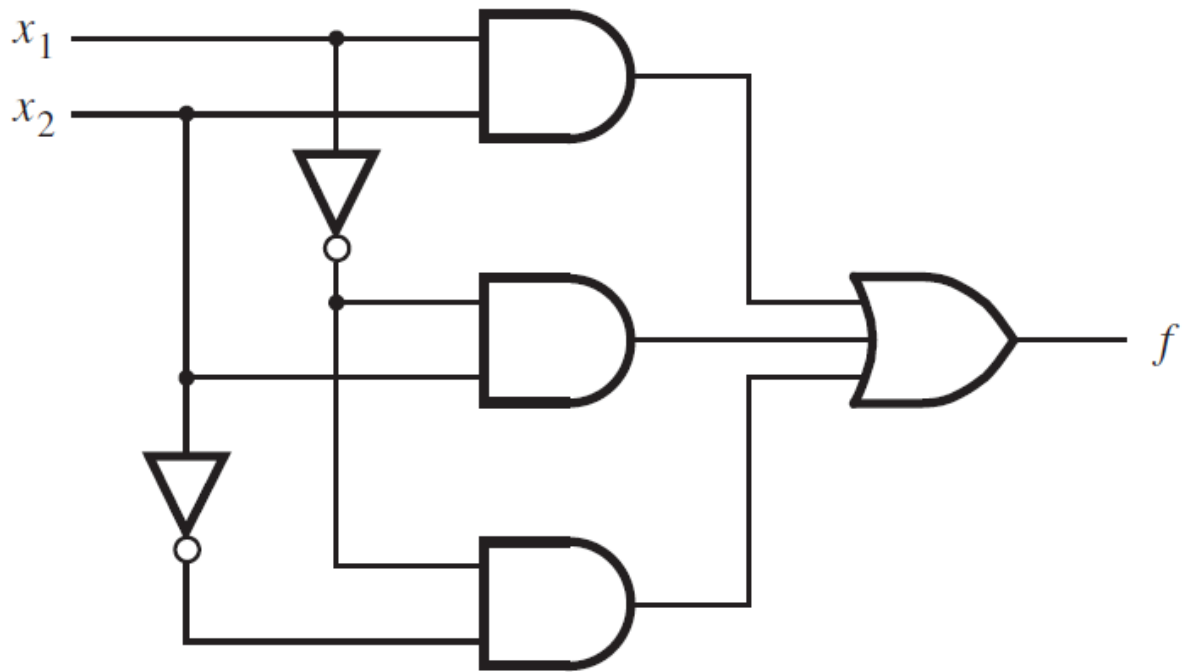
$$f(x_1, x_2) = x_1x_2 + \bar{x}_1x_2 + \bar{x}_1\bar{x}_2 + \bar{x}_1x_2$$

3. The distributive property 12a

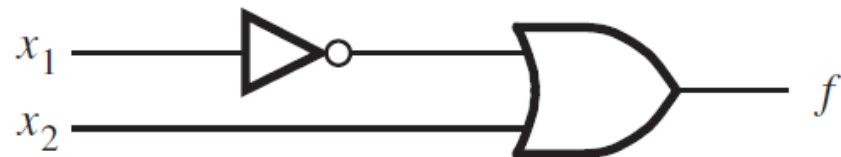
$$f(x_1, x_2) = (x_1 + \bar{x}_1)x_2 + \bar{x}_1(\bar{x}_2 + x_2)$$

4. Using 8b and 6a

$$f(x_1, x_2) = 1 \cdot x_2 + \bar{x}_1 \cdot 1 \quad \longrightarrow \quad f(x_1, x_2) = x_2 + \bar{x}_1$$



(a) Canonical sum-of-products



(b) Minimal-cost realization

Three Variable Minterms

Row number	x_1	x_2	x_3	Minterm
0	0	0	0	$m_0 = \bar{x}_1\bar{x}_2\bar{x}_3$
1	0	0	1	$m_1 = \bar{x}_1\bar{x}_2x_3$
2	0	1	0	$m_2 = \bar{x}_1x_2\bar{x}_3$
3	0	1	1	$m_3 = \bar{x}_1x_2x_3$
4	1	0	0	$m_4 = x_1\bar{x}_2\bar{x}_3$
5	1	0	1	$m_5 = x_1\bar{x}_2x_3$
6	1	1	0	$m_6 = x_1x_2\bar{x}_3$
7	1	1	1	$m_7 = x_1x_2x_3$

- Consider three variable function and synthesize the circuit for it.

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

- Consider three variable function and synthesize the circuit for it.

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

$$f(x_1, x_2, x_3) = \sum (m_1, m_4, m_5, m_6)$$

$$f(x_1, x_2, x_3) = \sum m(1, 4, 5, 6)$$

- Consider three variable function and synthesize the circuit for it.

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0
1	0	0	1	1 ← $\bar{x}_1\bar{x}_2x_3$
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1 ← $x_1\bar{x}_2\bar{x}_3$
5	1	0	1	1 ← $x_1\bar{x}_2x_3$
6	1	1	0	1 ← $x_1x_2\bar{x}_3$
7	1	1	1	0

Canonical SOP expression

$$f(x_1, x_2, x_3) = \bar{x}_1\bar{x}_2x_3 + x_1\bar{x}_2\bar{x}_3 + x_1\bar{x}_2x_3 + x_1x_2\bar{x}_3$$

- Draw circuit diagram
- Reduce it and again draw the reduced circuit diagram.
- Compare the cost-A good indication of the *cost* of a logic circuit is the total number of gates plus the total number of inputs to all gates in the circuit.

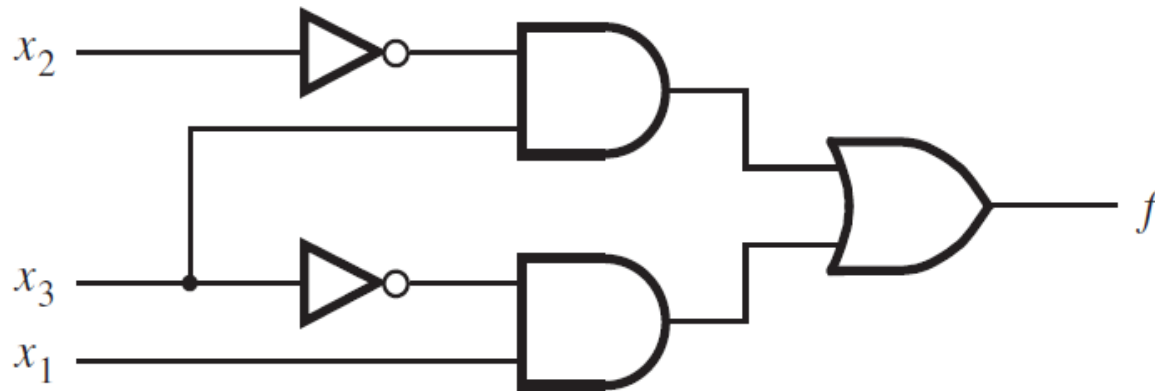
$$f(x_1, x_2, x_3) = \bar{x}_1\bar{x}_2x_3 + x_1\bar{x}_2\bar{x}_3 + x_1\bar{x}_2x_3 + x_1x_2\bar{x}_3$$



$$\begin{aligned} f &= (\bar{x}_1 + x_1)\bar{x}_2x_3 + x_1(\bar{x}_2 + x_2)\bar{x}_3 \\ &= 1 \cdot \bar{x}_2x_3 + x_1 \cdot 1 \cdot \bar{x}_3 \\ &= \bar{x}_2x_3 + x_1\bar{x}_3 \end{aligned}$$

$$= \bar{x}_2x_3 + x_1\bar{x}_3$$

Cost: 13(5 Gates & 8 inputs) vs
27 for canonical exp ckt realization



(a) A minimal sum-of-products realization

Example

Another example:

A	B	Cin	Sum	Cout	Sum = $\bar{A} \bar{B} C_{in} + \bar{A} B \bar{C}_{in} + A \bar{B} \bar{C}_{in} + A B C_{in}$
0	0	0	0	0	$C_{out} = \bar{A} B C_{in} + A \bar{B} C_{in} + A B \bar{C}_{in} + A B C_{in}$
0	0	1	1	0	
0	1	0	1	0	
0	1	1	0	1	
1	0	0	1	0	
1	0	1	0	1	
1	1	0	0	1	
1	1	1	1	1	

Product of Sum Canonical Form

- For a function of n variables a **maxterm** is a sum term in which each of the variable occur exactly once. For example

$$(x_0 + x_1 + \overline{x_2}) \text{ and } (\overline{x_0} + \overline{x_1} + x_2).$$

- A function f can be represented by an expression that is product of **maxterms**.
- A logical expression that consists of sum terms that are factors of logical product (AND) is called a **product-of-sum** form.
- If each of the sum term is a maxterm, then the expression is called **canonical product-of-sum** expression.
- Notice that two logically equivalent functions will have the same canonical product-of-sum representation.

Maxterms

Row number	x_1	x_2	x_3	Minterm	Maxterm
0	0	0	0	$m_0 = \bar{x}_1 \bar{x}_2 \bar{x}_3$	$M_0 = x_1 + x_2 + x_3$
1	0	0	1	$m_1 = \bar{x}_1 \bar{x}_2 x_3$	$M_1 = x_1 + x_2 + \bar{x}_3$
2	0	1	0	$m_2 = \bar{x}_1 x_2 \bar{x}_3$	$M_2 = x_1 + \bar{x}_2 + x_3$
3	0	1	1	$m_3 = \bar{x}_1 x_2 x_3$	$M_3 = x_1 + \bar{x}_2 + \bar{x}_3$
4	1	0	0	$m_4 = x_1 \bar{x}_2 \bar{x}_3$	$M_4 = \bar{x}_1 + x_2 + x_3$
5	1	0	1	$m_5 = x_1 \bar{x}_2 x_3$	$M_5 = \bar{x}_1 + x_2 + \bar{x}_3$
6	1	1	0	$m_6 = x_1 x_2 \bar{x}_3$	$M_6 = \bar{x}_1 + \bar{x}_2 + x_3$
7	1	1	1	$m_7 = x_1 x_2 x_3$	$M_7 = \bar{x}_1 + \bar{x}_2 + \bar{x}_3$

Maxterms

- The principle of duality suggests that if it is possible to synthesize a function f by considering the rows in the truth table for which $f = 1$, then it should also be possible to synthesize f by considering the rows for which $f = 0$.
- This alternative approach uses the complements of minterms, which are called *maxterms*.

- Consider three variable function and synthesize the circuit for it using Maxterms

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0 ← $(x_1 + x_2 + x_3)$
1	0	0	1	1
2	0	1	0	0 ← $(x_1 + \bar{x}_2 + x_3)$
3	0	1	1	0 ← $(x_1 + \bar{x}_2 + \bar{x}_3)$
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0 ← $(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)$

$$f(x_1, x_2, x_3) = \Pi(M_0, M_2, M_3, M_7)$$

$$f(x_1, x_2, x_3) = \Pi M(0, 2, 3, 7)$$

- Consider three variable function and synthesize the circuit for it using Maxterms

Row number	x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0	0 ← $(x_1 + x_2 + x_3)$
1	0	0	1	1
2	0	1	0	0 ← $(x_1 + \bar{x}_2 + x_3)$
3	0	1	1	0 ← $(x_1 + \bar{x}_2 + \bar{x}_3)$
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0 ← $(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)$

Canonical POS expression

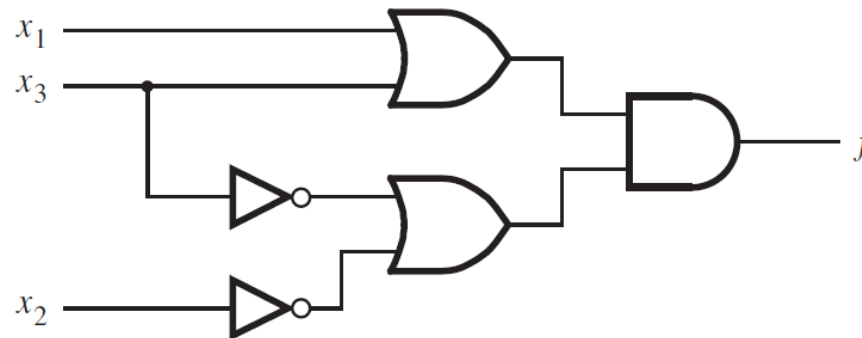
$$\begin{aligned}
 f(x_1, x_2, x_3) &= M_0 \cdot M_2 \cdot M_3 \cdot M_7 \\
 &= (x_1 + x_2 + x_3)(x_1 + \bar{x}_2 + x_3)(x_1 + \bar{x}_2 + \bar{x}_3)(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)
 \end{aligned}$$

Reduce it and draw circuit diagram

$$f(x_1, x_2, x_3) = (x_1 + x_2 + x_3)(x_1 + \bar{x}_2 + x_3)(x_1 + \bar{x}_2 + \bar{x}_3)(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)$$

$$f = ((x_1 + x_3) + x_2)((x_1 + x_3) + \bar{x}_2)(x_1 + (\bar{x}_2 + \bar{x}_3))(\bar{x}_1 + (\bar{x}_2 + \bar{x}_3))$$

$$f = (x_1 + x_3)(\bar{x}_2 + \bar{x}_3) \quad \text{Using 14b-combining property}$$



(b) A minimal product-of-sums realization

Consider the function

$$f(x_1, x_2, x_3) = \sum m(2, 3, 4, 6, 7)$$

- Q1>Find the cost of Canonical SOP expression.
- Q2>Reduce this expression and find cost again.
- Q3>Write corresponding Maxterm list-canonical POS form
- Q4>Reduce canonical POS expression and find the cost.
- Q5>Compare Q2 and Q4 costs

$$\begin{aligned}
 f &= m_2 + m_3 + m_4 + m_6 + m_7 \\
 &= \bar{x}_1 x_2 \bar{x}_3 + \bar{x}_1 x_2 x_3 + x_1 \bar{x}_2 \bar{x}_3 + x_1 x_2 \bar{x}_3 + x_1 x_2 x_3
 \end{aligned}$$

$$\begin{aligned}
 f &= \bar{x}_1 x_2 (\bar{x}_3 + x_3) + x_1 (\bar{x}_2 + x_2) \bar{x}_3 + x_1 x_2 (\bar{x}_3 + x_3) \\
 &= \bar{x}_1 x_2 + x_1 \bar{x}_3 + x_1 x_2 \\
 &= (\bar{x}_1 + x_1) x_2 + x_1 \bar{x}_3 \\
 &= x_2 + x_1 \bar{x}_3
 \end{aligned}$$

$$f(x_1, x_2, x_3) = \Pi M(0, 1, 5)$$

Then, the canonical POS expression is derived as

$$\begin{aligned}
 f &= M_0 \cdot M_1 \cdot M_5 \\
 &= (x_1 + x_2 + x_3)(x_1 + x_2 + \bar{x}_3)(\bar{x}_1 + x_2 + \bar{x}_3)
 \end{aligned}$$

A simplified POS expression can be derived as

$$\begin{aligned}
 f &= (x_1 + x_2 + x_3)(x_1 + x_2 + \bar{x}_3)(x_1 + x_2 + \bar{x}_3)(\bar{x}_1 + x_2 + \bar{x}_3) \\
 &= ((x_1 + x_2) + x_3)((x_1 + x_2) + \bar{x}_3)(x_1 + (x_2 + \bar{x}_3))(\bar{x}_1 + (x_2 + \bar{x}_3)) \\
 &= ((x_1 + x_2) + x_3 \bar{x}_3)(x_1 \bar{x}_1 + (x_2 + \bar{x}_3)) \\
 &= (x_1 + x_2)(x_2 + \bar{x}_3)
 \end{aligned}$$

Find minimum cost SOP expression for this function

Suppose that a four-variable function is defined by

$$f(x_1, x_2, x_3, x_4) = \sum m(3, 7, 9, 12, 13, 14, 15)$$

The canonical SOP expression for this function is

$$f = \bar{x}_1\bar{x}_2x_3x_4 + \bar{x}_1x_2x_3x_4 + x_1\bar{x}_2\bar{x}_3x_4 + x_1x_2\bar{x}_3\bar{x}_4 + x_1x_2\bar{x}_3x_4 + x_1x_2x_3\bar{x}_4 + x_1x_2x_3x_4$$

A simpler SOP expression can be obtained as follows

$$\begin{aligned} f &= \bar{x}_1(\bar{x}_2 + x_2)x_3x_4 + x_1(\bar{x}_2 + x_2)\bar{x}_3x_4 + x_1x_2\bar{x}_3(\bar{x}_4 + x_4) + x_1x_2x_3(\bar{x}_4 + x_4) \\ &= \bar{x}_1x_3x_4 + x_1\bar{x}_3x_4 + x_1x_2\bar{x}_3 + x_1x_2x_3 \\ &= \bar{x}_1x_3x_4 + x_1\bar{x}_3x_4 + x_1x_2(\bar{x}_3 + x_3) \\ &= \bar{x}_1x_3x_4 + x_1\bar{x}_3x_4 + x_1x_2 \end{aligned}$$

Problem: Design the minimum-cost product-of-sums expression for the function $f(x_1, x_2, x_3, x_4) = \sum m(0, 2, 4, 5, 6, 7, 8, 10, 12, 14, 15)$.

Solution: The function is defined in terms of its minterms. To find a POS expression we should start with the definition in terms of maxterms, which is $f = \Pi M(1, 3, 9, 11, 13)$. Thus,

$$\begin{aligned} f &= M_1 \cdot M_3 \cdot M_9 \cdot M_{11} \cdot M_{13} \\ &= (x_1 + x_2 + x_3 + \bar{x}_4)(x_1 + x_2 + \bar{x}_3 + \bar{x}_4)(\bar{x}_1 + x_2 + x_3 + \bar{x}_4)(\bar{x}_1 + x_2 + \bar{x}_3 + \bar{x}_4)(\bar{x}_1 + \bar{x}_2 + x_3 + \bar{x}_4) \end{aligned}$$

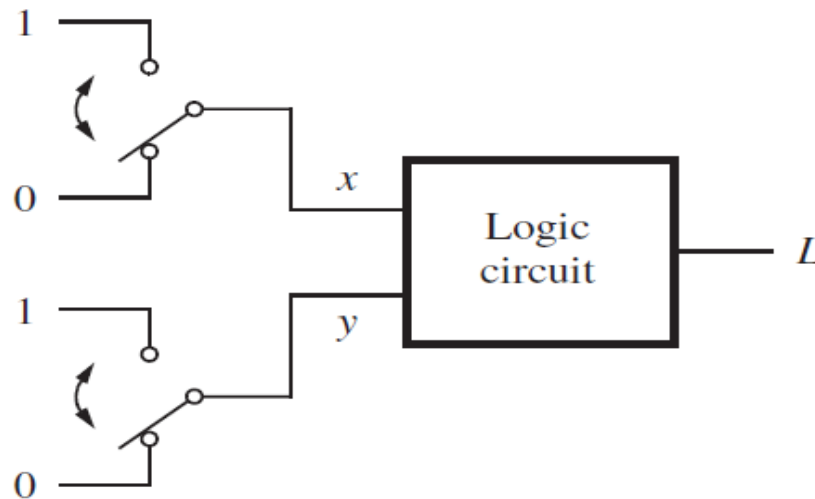
We can rewrite the product of the first two maxterms as

$$\begin{aligned} M_1 \cdot M_3 &= (x_1 + x_2 + \bar{x}_4 + x_3)(x_1 + x_2 + \bar{x}_4 + \bar{x}_3) && \text{using commutative property } 10b \\ &= x_1 + x_2 + \bar{x}_4 + x_3\bar{x}_3 && \text{using distributive property } 12b \\ &= x_1 + x_2 + \bar{x}_4 + 0 && \text{using theorem } 8a \\ &= x_1 + x_2 + \bar{x}_4 && \text{using theorem } 6b \end{aligned}$$

Similarly, $M_9 \cdot M_{11} = \bar{x}_1 + x_2 + \bar{x}_4$. Now, we can use M_9 again, according to property 7a, to derive $M_9 \cdot M_{13} = \bar{x}_1 + x_3 + \bar{x}_4$. Hence

$$f = (x_1 + x_2 + \bar{x}_4)(\bar{x}_1 + x_2 + \bar{x}_4)(\bar{x}_1 + x_3 + \bar{x}_4)$$

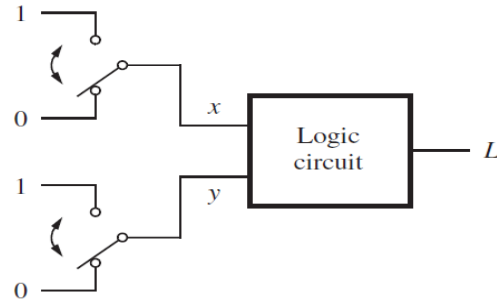
Try this problem: Write Truth Table and draw circuit diagram



(a) Two switches that control a light

The required behavior is that the light should be on only if one, but not both, of the toggle switches is in the top position.

Truth Table and Circuit diagram

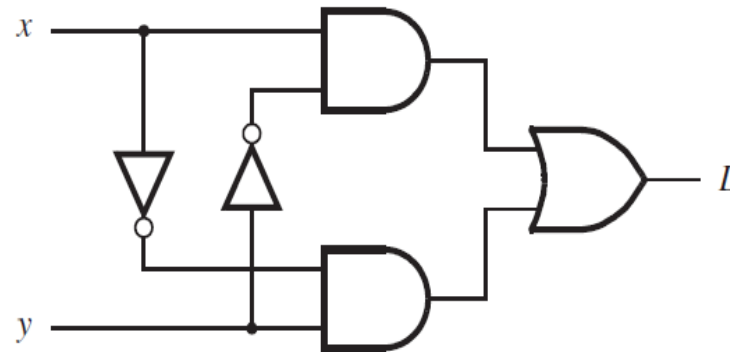


$$L = \bar{x} \cdot y + x \cdot \bar{y}, \text{ we can write } L = x \oplus y.$$

(a) Two switches that control a light

x	y	L
0	0	0
0	1	1
1	0	1
1	1	0

(b) Truth table



(c) Logic network



(d) XOR gate symbol

Synthesis Example

A circuit that controls a given digital system has three inputs: x_1 , x_2 , and x_3 . It has to recognize three different conditions:

- Condition A is true if x_3 is true and either x_1 is true or x_2 is false
- Condition B is true if x_1 is true and either x_2 or x_3 is false
- Condition C is true if x_2 is true and either x_1 is true or x_3 is false

The control circuit must produce an output of 1 if at least two of the conditions A , B , and C are true. Design the simplest circuit that can be used for this purpose.

Solution: Using 1 for true and 0 for false, we can express the three conditions as follows:

$$A = x_3(x_1 + \bar{x}_2) = x_3x_1 + x_3\bar{x}_2$$

$$B = x_1(\bar{x}_2 + \bar{x}_3) = x_1\bar{x}_2 + x_1\bar{x}_3$$

$$C = x_2(x_1 + \bar{x}_3) = x_2x_1 + x_2\bar{x}_3$$

Then, the desired output of the circuit can be expressed as $f = AB + AC + BC$. These product terms can be determined as:

$$\begin{aligned} AB &= (x_3x_1 + x_3\bar{x}_2)(x_1\bar{x}_2 + x_1\bar{x}_3) \\ &= x_3x_1x_1\bar{x}_2 + x_3x_1x_1\bar{x}_3 + x_3\bar{x}_2x_1\bar{x}_2 + x_3\bar{x}_2x_1\bar{x}_3 \\ &= x_3x_1\bar{x}_2 + 0 + x_3\bar{x}_2x_1 + 0 \\ &= x_1\bar{x}_2x_3 \end{aligned}$$

$$\begin{aligned} AC &= (x_3x_1 + x_3\bar{x}_2)(x_2x_1 + x_2\bar{x}_3) \\ &= x_3x_1x_2x_1 + x_3x_1x_2\bar{x}_3 + x_3\bar{x}_2x_2x_1 + x_3\bar{x}_2x_2\bar{x}_3 \\ &= x_3x_1x_2 + 0 + 0 + 0 \\ &= x_1x_2x_3 \end{aligned}$$

$$\begin{aligned} BC &= (x_1\bar{x}_2 + x_1\bar{x}_3)(x_2x_1 + x_2\bar{x}_3) \\ &= x_1\bar{x}_2x_2x_1 + x_1\bar{x}_2x_2\bar{x}_3 + x_1\bar{x}_3x_2x_1 + x_1\bar{x}_3x_2\bar{x}_3 \\ &= 0 + 0 + x_1\bar{x}_3x_2 + x_1\bar{x}_3x_2 \\ &= x_1x_2\bar{x}_3 \end{aligned}$$

Therefore, f can be written as

$$\begin{aligned} f &= x_1\bar{x}_2x_3 + x_1x_2x_3 + x_1x_2\bar{x}_3 \\ &= x_1(\bar{x}_2 + x_2)x_3 + x_1x_2(x_3 + \bar{x}_3) \\ &= x_1x_3 + x_1x_2 \\ &= x_1(x_3 + x_2) \end{aligned}$$

Digital Electronics & Systems

19ECE204/19CSE203

LECTURE 5

XOR and XNOR Logic Gates

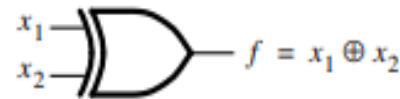
Exclusive-OR (XOR) Gate

- So far we have encountered AND, OR, and NOT gates as the basic elements from which logic circuits can be constructed.
- There is another basic element that is very useful in practice, particularly for building circuits that perform arithmetic operations.
- This element realizes the Exclusive-OR function.

For n-input XOR gate
output is high, when odd
number of inputs are high

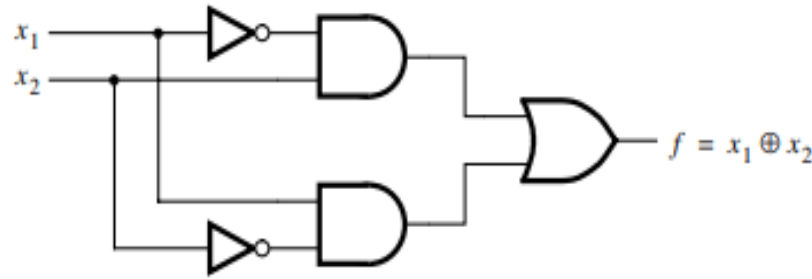
x_1	x_2	$f = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

(a) Truth table



(b) Graphical symbol

XOR and XNOR Logic Gates



(c) Sum-of-products implementation

- The truth table for this function is similar to the OR function except that $f = 0$ when both inputs are 1.
- Because of this similarity, the function is called Exclusive-OR, which is commonly abbreviated as XOR.
- The XOR operation is usually denoted with the $\oplus$ symbol. It can be realized in the sum-of-products form as

$$x_1 \oplus x_2 = \overline{x_1}x_2 + x_1\overline{x_2}$$

XOR and XNOR Logic Gates

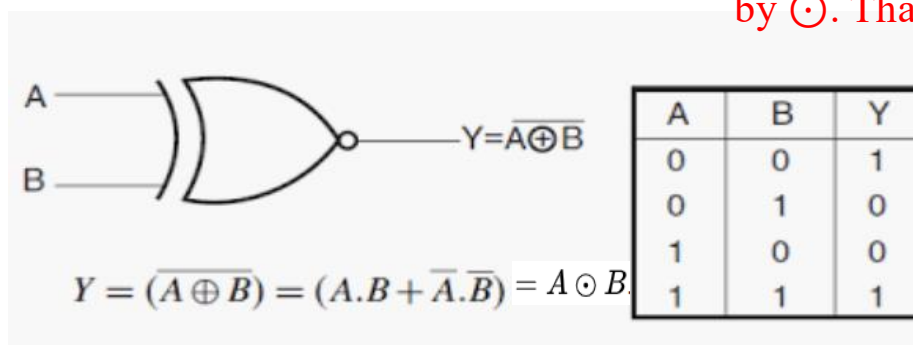
Exclusive-NOR (XNOR) Gate

- The Exclusive-NOR Gate function is a digital logic gate that is the reverse or complementary form of the Exclusive-OR function.
- Basically the “Exclusive-NOR” gate is a combination of the Exclusive-OR gate and the NOT gate.



2-input "Ex-OR" gate plus a "NOT" gate

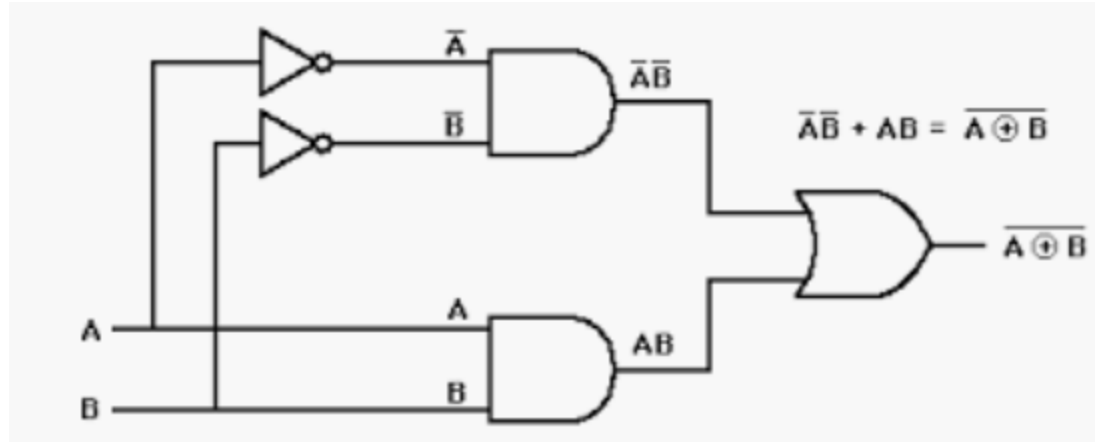
The logical XNOR operation is represented by $\odot$. That is a dot surrounded by circle.



XOR and XNOR Logic Gates

Exclusive-NOR (XNOR) Gate

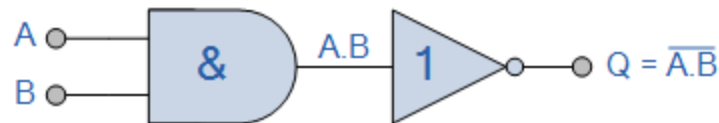
- The output of a 2-input XNOR gate is high when all of its inputs are same, either high or low. If its inputs are different then the output of the XNOR gate is low.
- For n-input XNOR gate, output is high when even number of inputs are high.



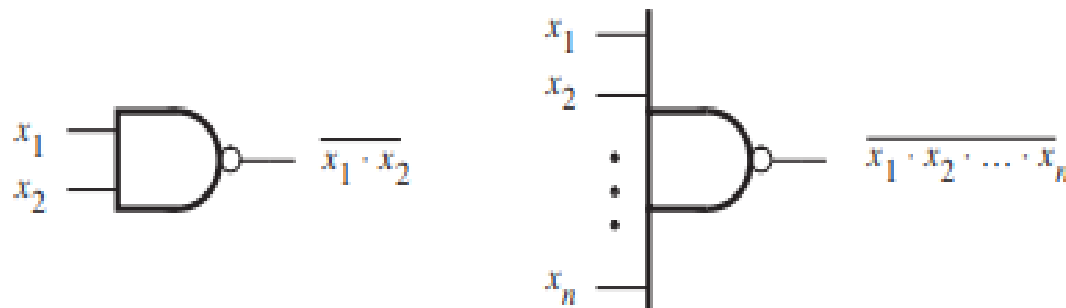
SOP Implementation

NAND and NOR Logic Networks

- The NAND and NOR gates are called Universal gates since with either all other logic gates can be implemented.
- NAND Gate-**
 - NAND gate is equal to AND gate followed by NOT gate.



Logic NAND Gate Equivalence



Symbol for NAND Gate

NAND and NOR Logic Networks

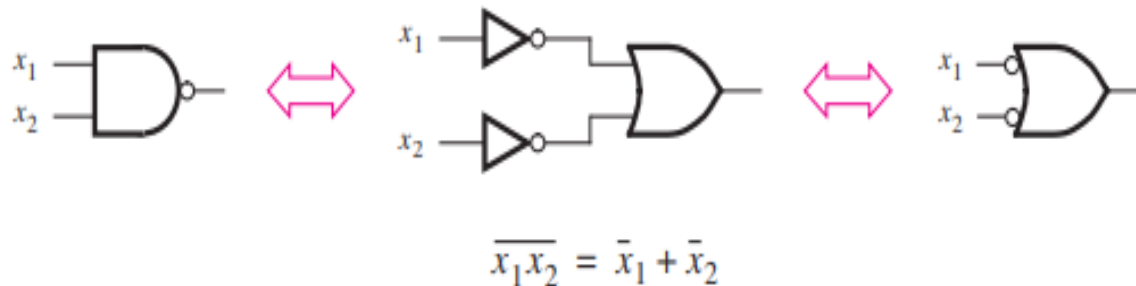
NAND Gate-

Truth Table

2 Input NAND gate		
A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

In NAND gate, if any input is LOW (0), a HIGH (1) output results.

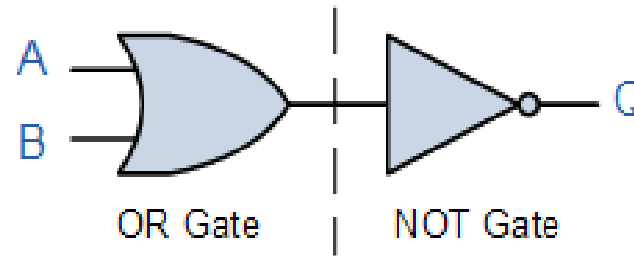
- A 2-input NAND gate is equivalent to 2-input OR gate with complemented inputs.



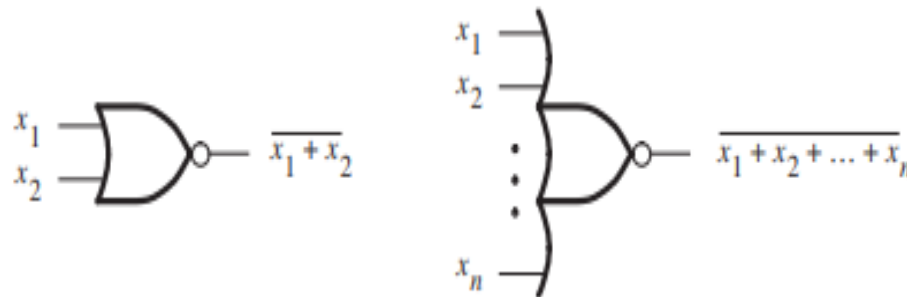
NAND and NOR Logic Network

NOR Gate-

- NOR gate is equal to OR gate followed by NOT gate.



Logic NOR Gate Equivalence



Symbol for NOR Gate

NAND and NOR Logic Networks

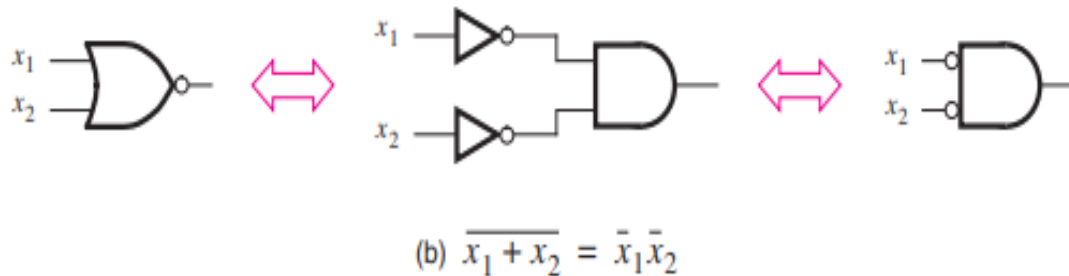
NOR Gate-

Truth Table

2 Input NOR gate		
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

The output of **NOR gate** is high only if all the inputs are low.

- A 2-input NOR gate is equivalent to 2-input AND gate with complemented inputs.



NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

- Sum-of-Product form (SOP Form)-

➤ First Method:

- 1) Draw AND-OR Network for the given function.
- 2) Each connection between the AND gate and an OR gate is replaced by a connection that includes two inversions of the signal: one inversion at the output of the AND gate and the other at the input of the OR gate.
- 3) Such double inversion has no effect on the behavior of the network.

Consider a function

$$f = x_1 \cdot x_2 + x_3 \cdot x_4 \cdot x_5$$

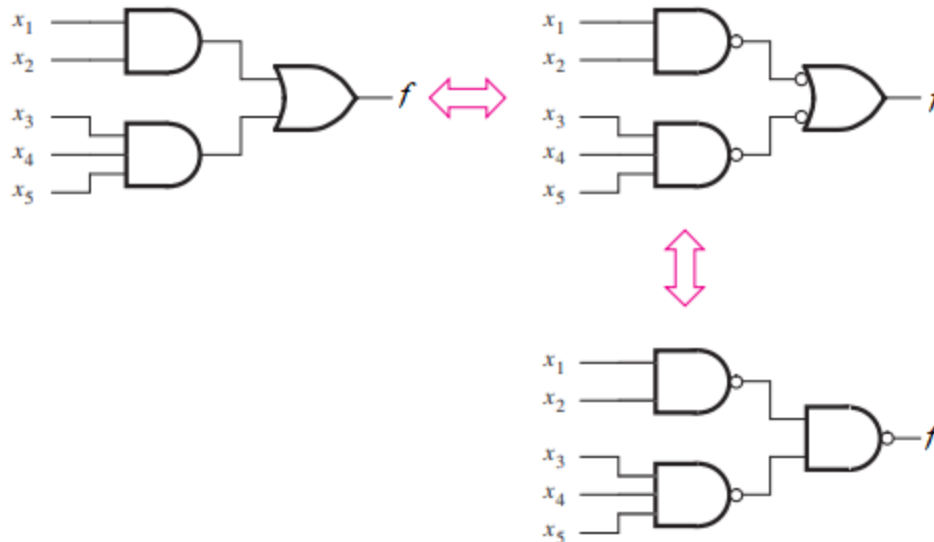
NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

- Sum-of-Product form (SOP Form)-

➤ First Method:

$$f = x_1 \cdot x_2 + x_3 \cdot x_4 \cdot x_5$$



any AND-OR network
can be implemented as
a NAND-NAND network
having the same topology

Using NAND gates to implement a sum-of-products

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

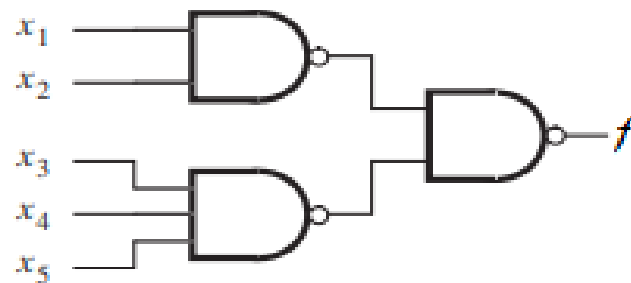
- Sum-of-Product form (SOP Form)-

➤ Second Method:

$$f = x_1 \cdot x_2 + x_3 \cdot x_4 \cdot x_5$$

$$f = \overline{\overline{x_1 \cdot x_2 + x_3 \cdot x_4 \cdot x_5}}$$

$$f = \overline{(\overline{x_1 \cdot x_2}) \cdot (\overline{x_3 \cdot x_4 \cdot x_5})} \quad \text{(Using Demorgan's Theorem)}$$



Using NAND gates to implement a sum-of-products

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

- Product-of-Sum form (POS Form)-

➤ First Method:

- 1) Draw OR-AND Network for the given function.
- 2) Each connection between the OR gate and AND gate is replaced by a connection that includes two inversions of the signal: one inversion at the output of the OR gate and the other at the input of the AND gate.
- 3) Such double inversion has no effect on the behavior of the network.

Consider a function

$$f = (x_1 + x_2) \cdot (x_3 + x_4 + x_5)$$

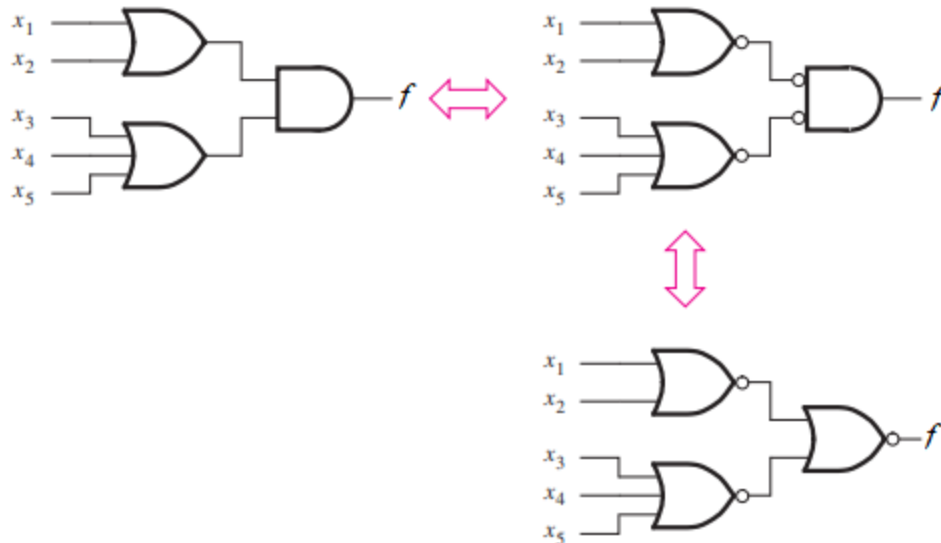
NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

- Product-of-Sum form (POS Form)-

➤ First Method:

$$f = (x_1 + x_2) \cdot (x_3 + x_4 + x_5)$$



any OR-AND network
can be implemented as
a NOR-NOR network
having the same topology

Using NOR gates to implement a product-of-sums

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

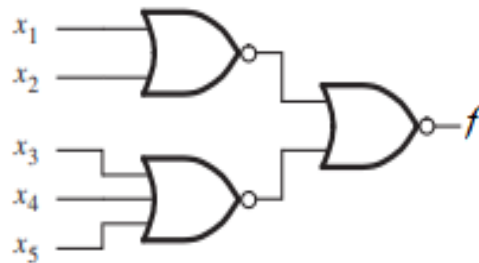
- Product-of-Sum form (POS Form)-

➤ Second Method:

$$f = (x_1 + x_2) \cdot (x_3 + x_4 + x_5)$$

$$f = \overline{\overline{(x_1 + x_2)} \cdot \overline{(x_3 + x_4 + x_5)}}$$

$$f = \overline{\overline{(x_1 + x_2)} + \overline{(x_3 + x_4 + x_5)}} \quad \text{(Using Demorgan's Theorem)}$$

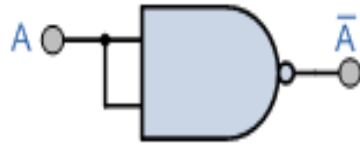


Using NOR gates to implement a product-of-sums

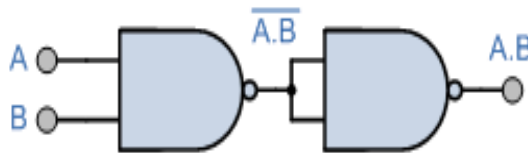
NAND and NOR Logic Networks

Various Logic Gates Using Only NAND Gates-

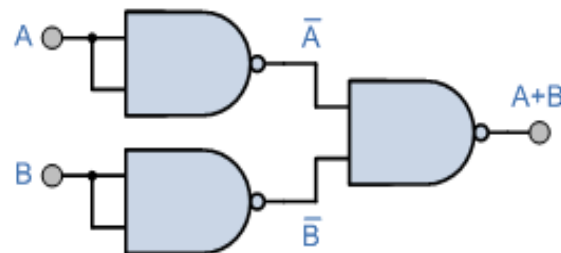
1) NOT Gate



2) AND Gate



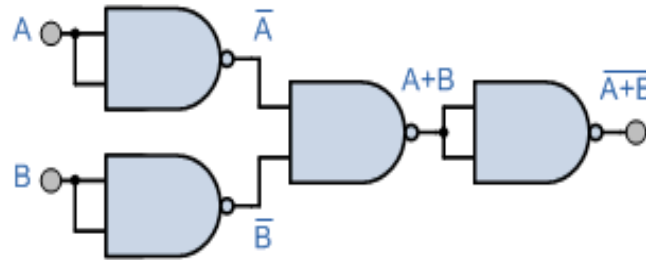
3) OR Gate



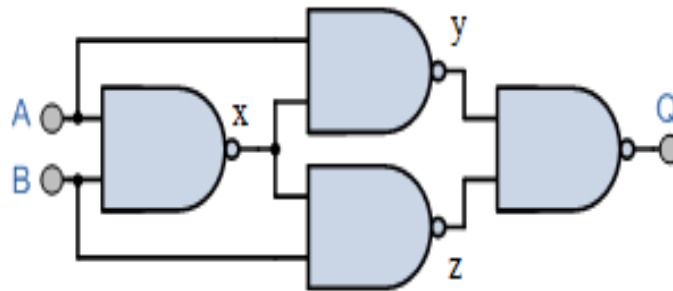
NAND and NOR Logic Networks

Various Logic Gates Using Only NAND Gates-

4) NOR Gate



5) XOR Gate



$$x = \overline{A \cdot B} = \overline{A} + \overline{B}$$

$$y = \overline{A \cdot x} = A \cdot (\overline{A} + \overline{B})$$

$$y = \overline{A \cdot B}$$

$$z = \overline{B \cdot x} = B \cdot (\overline{A} + \overline{B})$$

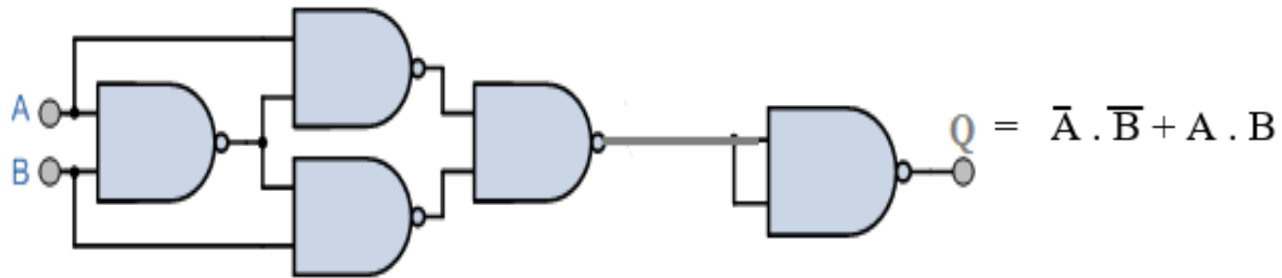
$$z = \overline{A \cdot B}$$

$$Q = \overline{y \cdot z} = A \cdot \overline{B} + \overline{A} \cdot B$$

NAND and NOR Logic Networks

Various Logic Gates Using Only NAND Gates-

6) XNOR Gate

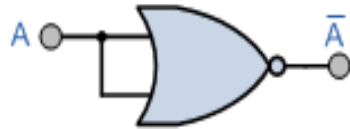


LECTURE 6

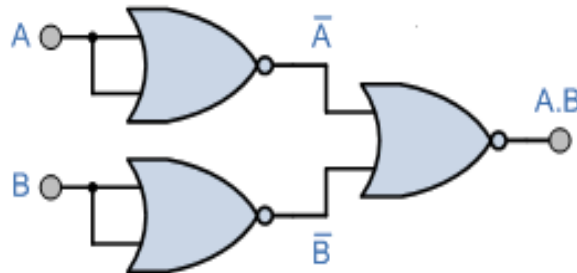
NAND and NOR Logic Networks

Various Logic Gates Using Only NOR Gates-

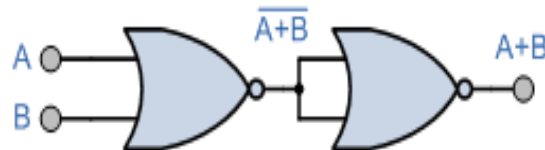
1) NOT Gate



2) AND Gate



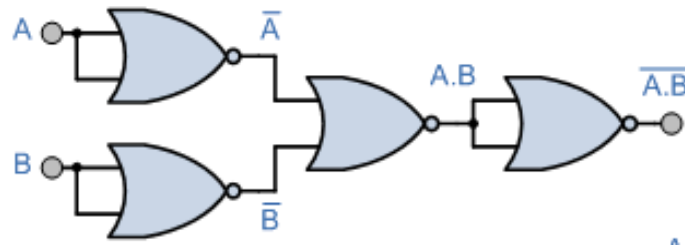
3) OR Gate



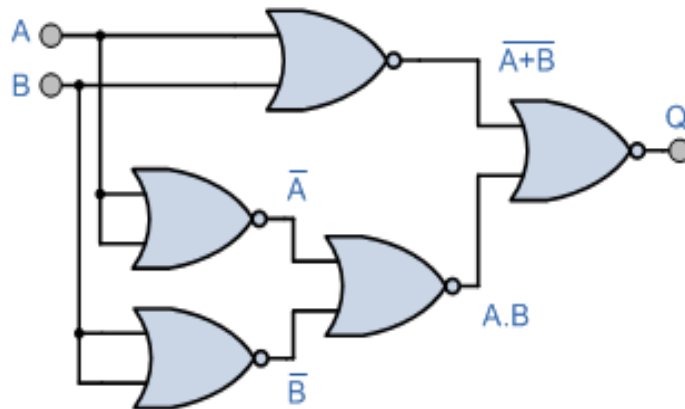
NAND and NOR Logic Networks

Various Logic Gates Using Only NOR Gates-

4) NAND Gate



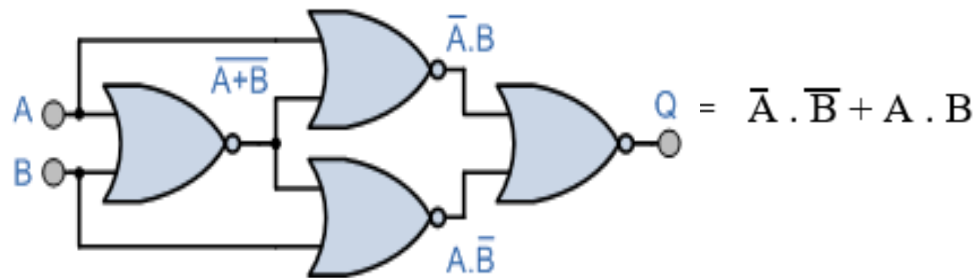
5) XOR Gate



NAND and NOR Logic Networks

Various Logic Gates Using Only NOR Gates-

6) XNOR Gate



NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

Example 1-

Implement the function $f(a, b, c) = \sum m(2, 3, 4, 6, 7)$ using NAND gates only.

(a) In SOP form

(b) In POS form

Solution- The canonical SOP expression for the function is derived using minterms-

$$f = m_2 + m_3 + m_4 + m_6 + m_7$$

$$f = a'bc' + a'bc + ab'c' + abc' + abc$$

$$f = a'b(c' + c) + ac'(b' + b) + ab(c' + c) \quad \{x + x' = 1\}$$

$$f = a'b + ac' + ab = (a' + a)b + ac'$$

$$f = b + ac'$$

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

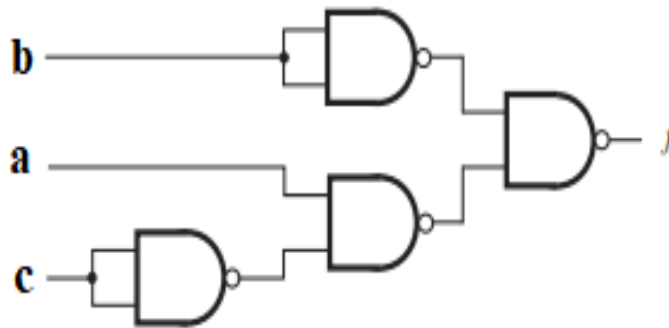
Example 1 Solution-

(a) SOP form

$$f = b + ac' = \overline{\overline{b + ac'}}$$

$x' = \overline{x}$ = complement of x

$$f = \overline{b' \cdot (ac')'}$$



Function implementation using NAND Gates only

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

Example 1 Solution-

(b) POS form

$$f = b + \underline{ac'} = \underline{(a + b) \cdot (b + c')}$$

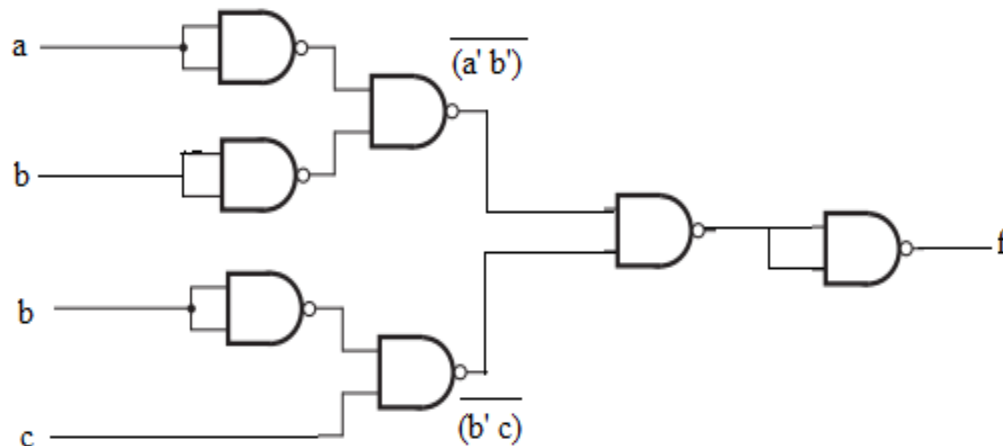
{Distributive Law}

$$f = \underline{(a + b) \cdot (b + c')}$$

{ $x' = \overline{x}$ = complement of x }

$$f = \underline{(a' \cdot b')} + \underline{(b' \cdot c)} = \overline{(a' \cdot b')} \cdot \overline{(b' \cdot c)}$$

{Demorgan's Theorem}



Function implementation using NAND Gates only

NAND and NOR Logic Networks

Function Implementation Using NAND Gates Only

Example 1 Solution-

$$f(a, b, c) = \sum m(2, 3, 4, 6, 7) = b + ac' = (a + b) \cdot (b + c')$$

- Given function implementation in SOP form- No. of NAND gates required – 4
- Given function implementation in POS form- No. of NAND gates required - 7

Conclusion- Any logic function can be implemented using NAND gates. To achieve this, first the logic function has to be written in Sum of Product (SOP) form. Once logic function is converted to SOP, then is very easy to implement using NAND gate and it requires less number of gates.

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

Example 2-

Implement the function $f(a, b, c) = \sum m(2, 3, 4, 6, 7)$ using NOR gates only.

(a) In SOP form

(b) In POS form

Solution- From Example 1 we got-

$$f = b + a c'$$

(a) In SOP form

$$f = \overline{\overline{b + a c'}} = \overline{b' \cdot a c'}$$

{Demorgan's Theorem}

$$f = b' \cdot \overline{(a' + c)}$$

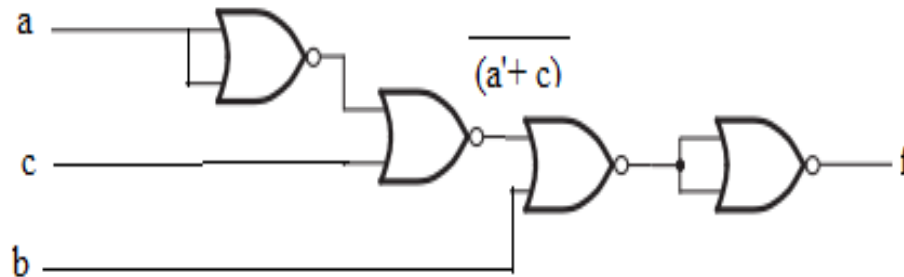
$$f = b + (a' + c)$$

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

Example 2 Solution-

(a) In SOP form



Function implementation using NOR Gates only

NAND and NOR Logic Networks

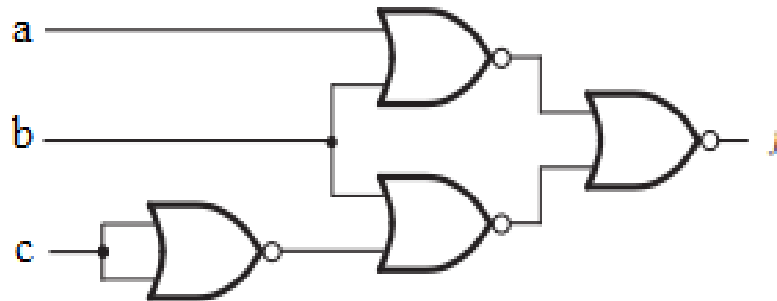
Function Implementation Using NOR Gates Only

Example 2 Solution-

(a) In POS form

$$f = \overline{b + a c'} = \overline{(a + b) \cdot (b + c')}$$

$$f = \overline{(a + b)} + \overline{(b + c')}$$



Function implementation using NOR Gates only

NAND and NOR Logic Networks

Function Implementation Using NOR Gates Only

Example 2 Solution-

Conclusion- Any logic function can be implemented using NOR gates. To achieve this, first the logic function has to be written in Product of Sum (POS) form. Once logic function is converted to POS, then is very easy to implement using NOR gate and it requires less number of gates.

(in example 2 number of NOR gates, in both SOP and POS form implementations, are same but it is not applicable on all the functions)

NAND and NOR Logic Networks

Example 3-

Design the simplest circuit that implements the given function using NAND gates only.

$$F(x, y, z) = \sum m(1, 2, 3, 4, 5, 7)$$

Solution- The canonical SOP expression for the function is derived using minterms-

$$F = m_1 + m_2 + m_3 + m_4 + m_5 + m_7$$

$$F = x'y'z + x'yz' + x'yz + xy'z' + xy'z + xyz$$

$$F = x'y'z + x'yz + x'yz' + x'yz + xy'z' + xy'z + xy'z + xyz$$

$$F = x'z(y' + y) + x'y(z' + z) + xy'(z' + z) + xz(y' + y) \quad \{x + x' = 1\}$$

$$F = x'z + x'y + xy' + xz = (x' + x)z + x'y + xy'$$

$$F = xy' + x'y + z$$

NAND and NOR Logic Networks

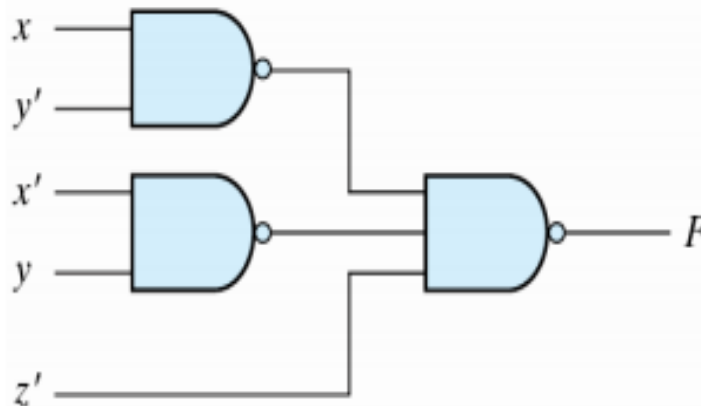
Example 3 Solution-

$$F = xy' + x'y + z$$

$$F(x, y, z) = xy' + x'y + z$$

$$= \overline{\overline{xy' + x'y + z}}$$

$$= \overline{(\overline{xy'}) (\overline{x'y}) z'}$$



(Assuming both normal and complemented inputs are available)

NAND and NOR Logic Networks

Example 4-

Design the simplest circuit that implements the given function using NOR gates only.

$$F(x, y, z) = \sum m(1, 2, 3, 4, 5, 7)$$

Solution-

$$F(x, y, z) = \sum m(1, 2, 3, 4, 5, 7) = \prod M(0, 6)$$

The canonical POS expression for the function is derived using maxterms-

$$F = M_0 \cdot M_6$$

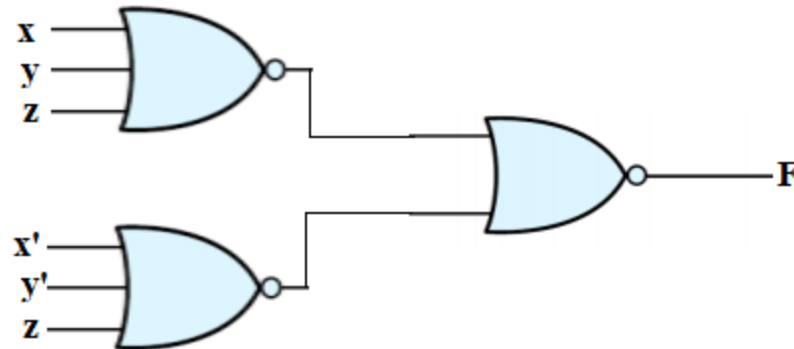
$$F = (x + y + z) \cdot (x' + y' + z) = \overline{\overline{(x + y + z)} \cdot \overline{(x' + y' + z)}}$$

$$F = \overline{\overline{(x + y + z)} + \overline{(x' + y' + z)}}$$

NAND and NOR Logic Networks

Example 4 Solution-

$$F = \overline{(x + y + z)} + \overline{(x' + y' + z)}$$



LECTURE 7

Optimized Implementation of Logic Functions

- The algebraic manipulation can be used to find the lowest-cost implementations of logic functions.
- It is easy to derive a straightforward realization of a logic function in a canonical form, but it is not at all obvious how to choose and apply the theorems and properties of Boolean algebra to find a minimum-cost circuit.
- Indeed, the algebraic manipulation is rather tedious and quite impractical for functions of many variables.
- Another minimization approach, which provides a neat way to manually derive minimum-cost implementations of logic functions, is known as Karnaugh Map.

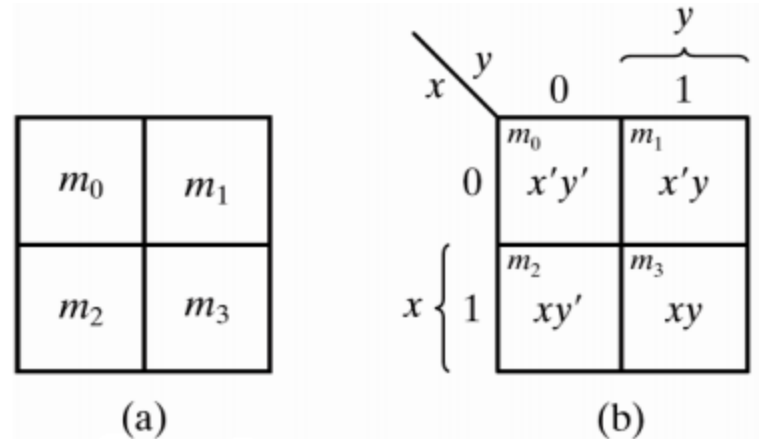
Optimized Implementation of Logic Functions – Karnaugh Map

- The Karnaugh map approach provides a systematic way of performing this optimization.
- The Karnaugh map (K-Map) is an alternative to the truth-table form for representing a function. It is A pictorial form of a truth table.
- The K-Map is a diagram made up of squares. Each square is a cell of K-map. The map consists of cells that correspond to the rows of the truth table.

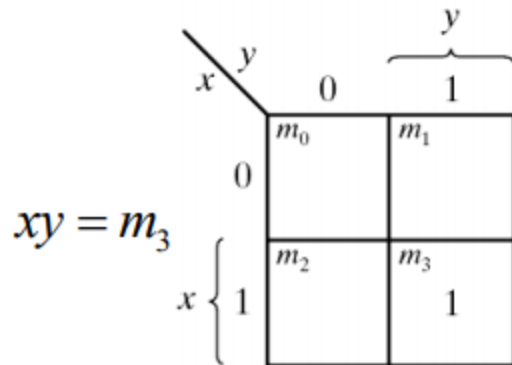
Optimized Implementation of Logic Functions – Karnaugh Map

Two-Variable K-Map

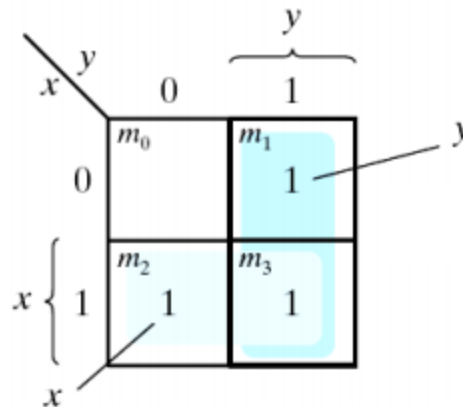
- Four minterms
- $x' = \text{row } 0$; $x = \text{row } 1$
- $y' = \text{column } 0$; $y = \text{column } 1$
- A truth table in square diagram



Two-variable Map



(a) xy



(b) $x + y$

Representation of functions in the map

$$F(x, y) = \sum m(1, 2, 3)$$

$$= x'y + xy' + xy$$

After minimization

$$F(x, y) = x + y$$

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

- A three-variable Karnaugh map is constructed by placing 2 two-variable maps side by side.
- Eight minterms
- The Gray code sequence
- Any two adjacent squares in the map differ by only one variable.
- In a three-variable map it is possible to combine cells to produce product terms that correspond to
 - a single cell – gives 3 variable term
 - two adjacent cells (pair) – gives 2 variable term
 - a group of four adjacent cells (quad)- gives single variable term

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

x_1	x_2	x_3	
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

(a) Truth table

		$x_1 x_2$			
		00	01	11	10
x_3	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

(b) Karnaugh map

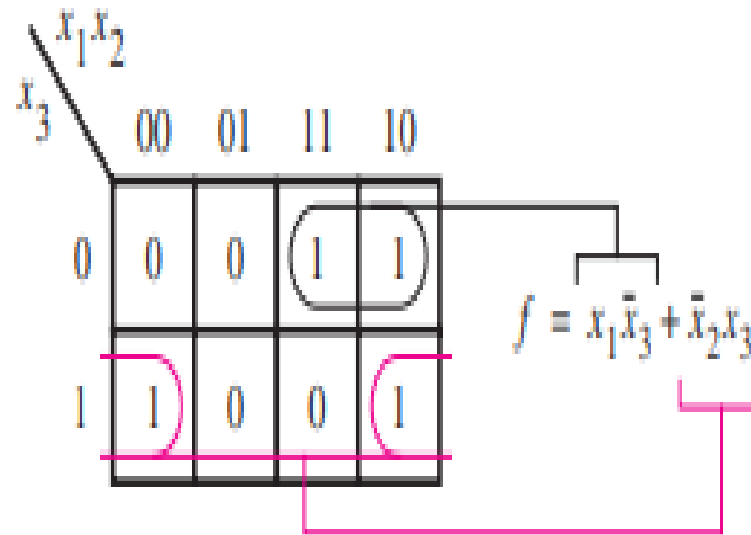
Location of three-variable minterms.

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Example 1. Simplify the Boolean function

$$f = \Sigma m(1,4,5,6)$$



Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Example2 : simplify the Boolean functions:

$$F(x, y, z) = \sum(0, 2, 4, 5, 6)$$

$$F(x, y, z) = \sum(0, 2, 4, 5, 6) = z' + xy'$$

		z'			
		xy 00	01	11	10
z	0	1	1	1	1
	1	0	0	0	1

Diagram illustrating the Karnaugh Map for the function $F(x, y, z) = \sum(0, 2, 4, 5, 6)$. The map shows the function values for combinations of x, y, z . The variables x and y are grouped together as xy , and z is the vertical axis. The map is a 2x4 grid. The top row (z=0) contains 1s for all combinations of x and y. The bottom row (z=1) contains 0s for all combinations of x and y except for the last one (xy=10), which contains a 1. An orange loop highlights the four 1s in the top row, representing the term z' . A gray loop highlights the single 1 in the bottom-right corner, representing the term xy' .

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Example 3 : Let the Boolean function

$$F = A'C + A'B + AB'C + BC$$

- a) Express it in sum of minterms.
- b) Find the minimal sum of products expression.

Solution-

$$\begin{aligned} \text{a) } F &= A'C + A'B + AB'C + BC \\ &= A' (B + B') C + A'B (C + C') + AB'C + (A + A')BC \\ &= A'BC + A'B'C + A'BC + A'BC' + AB'C + ABC + A'BC \\ &= m_3 + m_1 + m_3 + m_2 + m_5 + m_7 + m_3 \\ F &= \Sigma m(1, 2, 3, 5, 7) \end{aligned}$$

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Solution of Example 3-

b)

		AB		A'B		
		00	01	11	10	
C	0	0	1	0	0	
	1	1	1	1	1	

C

$$F(A, B, C) = \sum(1, 2, 3, 5, 7) = C + A'B$$

Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

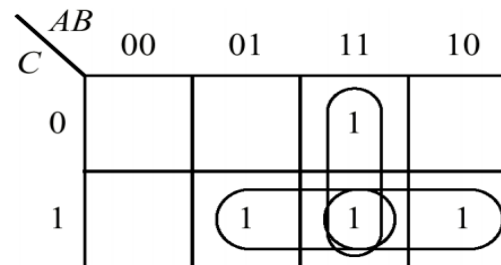
Example 4. Design a 3-bit Majority function.

Solution- **Majority function** is a threshold **function** that produces a 1 if and only if the **majority** of the inputs are 1.

Truth Table

Minterm Index	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

$$F = \sum m(3, 5, 6, 7)$$



$$F = BC + AB + AC$$

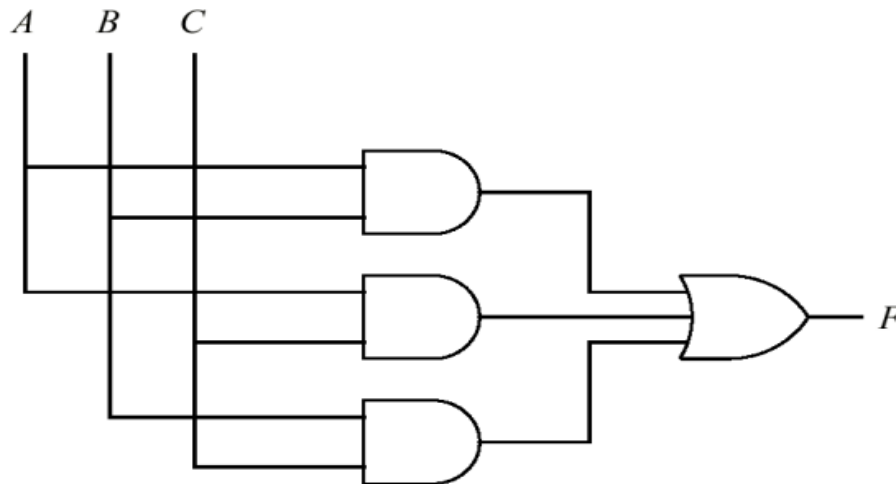
Optimized Implementation of Logic Functions – Karnaugh Map

Three-Variable K-Map

Solution of Example 4-

Logic Circuit for 3-bit Majority function-

$$F = AB + AC + BC$$



LECTURE 8

Optimized Implementation of Logic Functions – Karnaugh Map

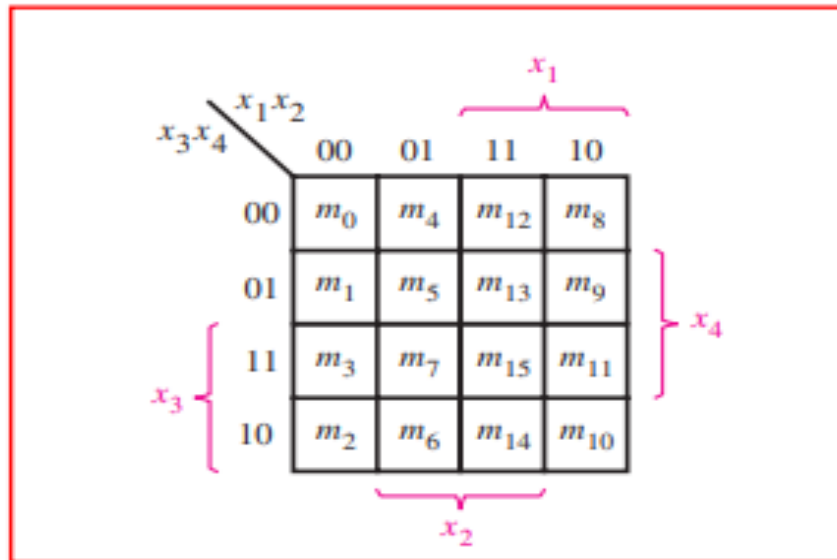
Four-Variable K-Map

- A four-variable map is constructed by placing 2 three-variable maps together to create four rows in the same fashion as we used 2 two-variable maps to form the four columns in a three-variable map.
- In a four-variable map it is possible to combine cells to produce product terms that correspond to a single cell, two adjacent cells (pair), a group of four adjacent cells (quad) and a group of eight adjacent cells (octa).
- Corner cells also can be combined to produce product term that correspond to a pair, quad or octa.

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

- 16 minterms
- Combinations of 16 adjacent squares



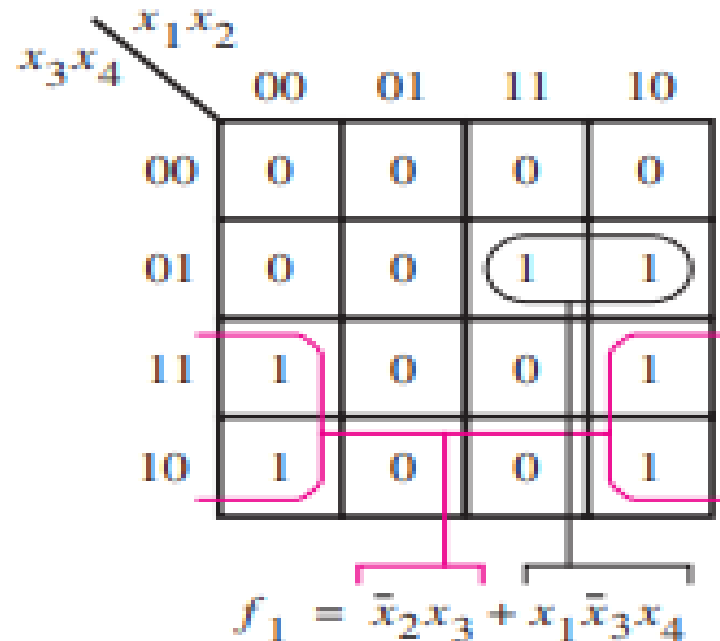
A four-variable Karnaugh map.

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 1: Simplify the Boolean function-

$$f_1(x_1, x_2, x_3, x_4) = \Sigma m(2, 3, 9, 10, 11, 13)$$

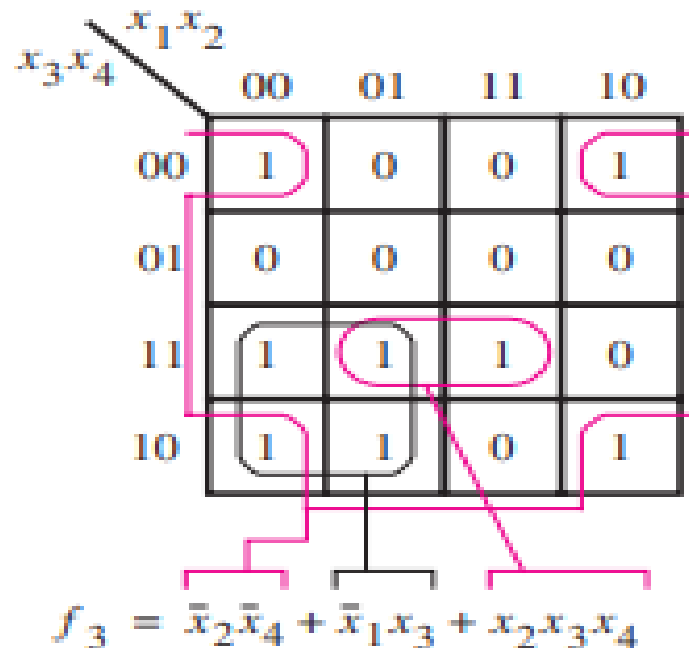


Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 2: Simplify the Boolean function-

$$f_1(x_1, x_2, x_3, x_4) = \Sigma m(0, 2, 3, 6, 7, 8, 10, 15)$$

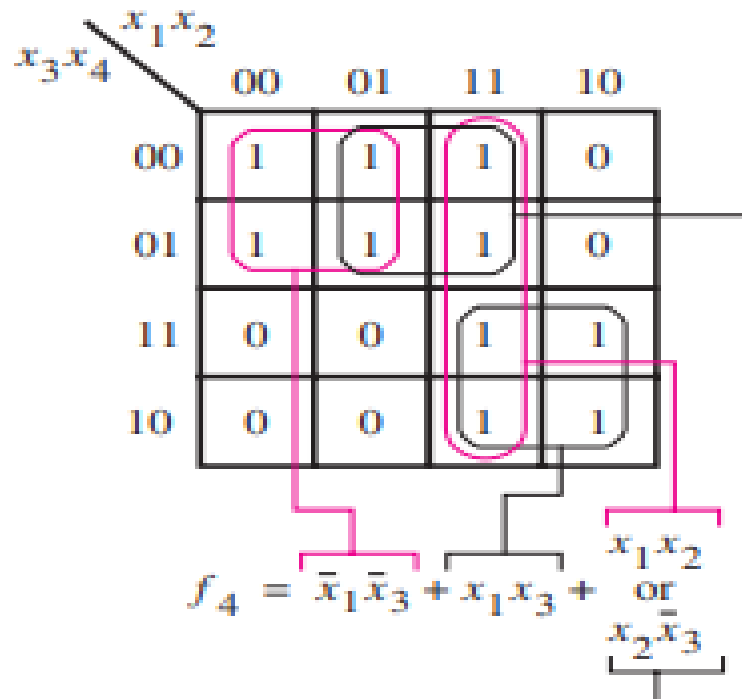


Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 3: Simplify the Boolean function-

$$f_1(x_1, x_2, x_3, x_4) = \sum m(0, 1, 4, 5, 10, 11, 12, 13, 14, 15)$$



Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Example 4. Simplify the given Boolean function using Karnaugh Map.

$$F(w, x, y, z) = w'x'y'z + x'yz + wy'z' + wxy'z + wx'y'z$$

Solution – First we need to express the function in sum of minterms form:

$$F = w'x'y'z + (w + w')x'yz + w(x + x')y'z' + wxy'z + wx'y'z$$

$$= w'x'y'z + wx'yz + w'x'yz + wxy'z' + wx'y'z' + wxy'z + wx'y'z$$

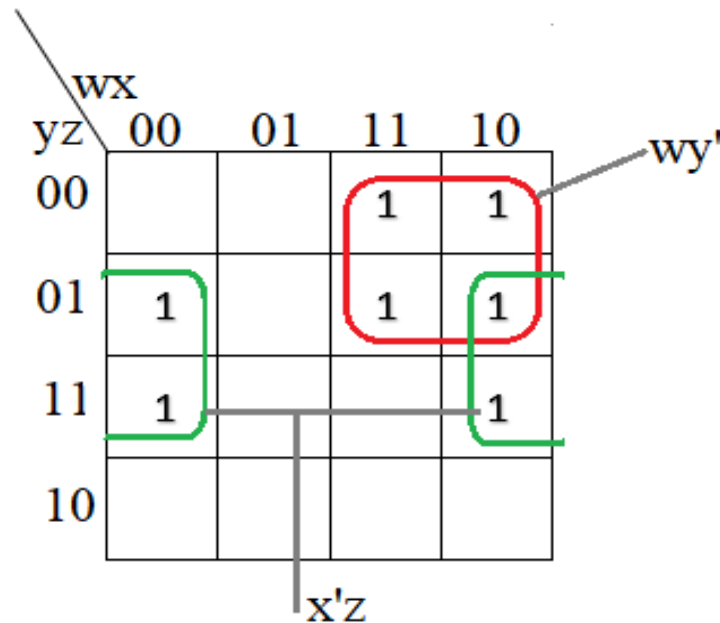
$$= m_1 + m_{11} + m_3 + m_{12} + m_8 + m_{13} + m_9$$

$$F = \Sigma m(1, 3, 8, 9, 11, 12, 13)$$

Optimized Implementation of Logic Functions – Karnaugh Map

Four-Variable K-Map

Solution for Example 4-



$$F = wy' + x'z$$

Optimized Implementation of Logic Functions – Karnaugh Map

Five-Variable K-Map

- We can use 2 four-variable maps to construct a five-variable map.
- It is easy to imagine a structure where one map is directly behind the other, and they are distinguished by $A = 0$ for one map and $A = 1$ for the other map.

		BC			
		00	01	11	10
DE	00	m0	m4	m12	m8
	01	m1	m5	m13	m9
	11	m3	m7	m15	m11
	10	m2	m6	m14	m10

A = 0

		BC			
		00	01	11	10
DE	00	m16	m20	m28	m24
	01	m17	m21	m29	m25
	11	m19	m23	m31	m27
	10	m18	m22	m30	m26

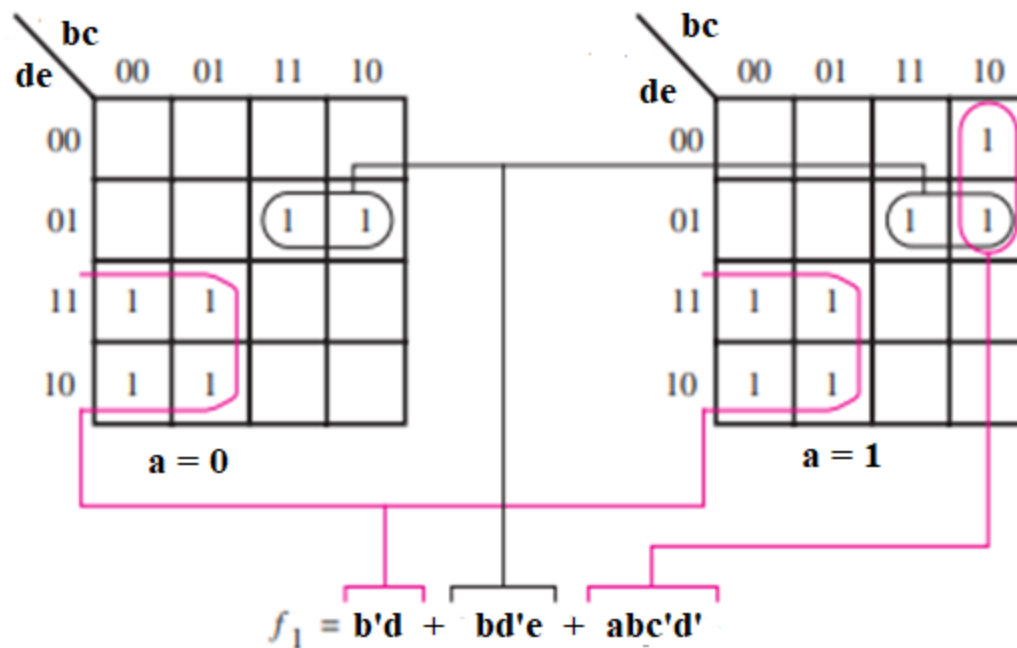
A = 1

Optimized Implementation of Logic Functions – Karnaugh Map

Five-Variable K-Map

Example 1. Simplify the Boolean function

$$f_1(a, b, c, d, e) = \Sigma m(2,3,6,7,9,13,18,19,22,23,24,25,29)$$

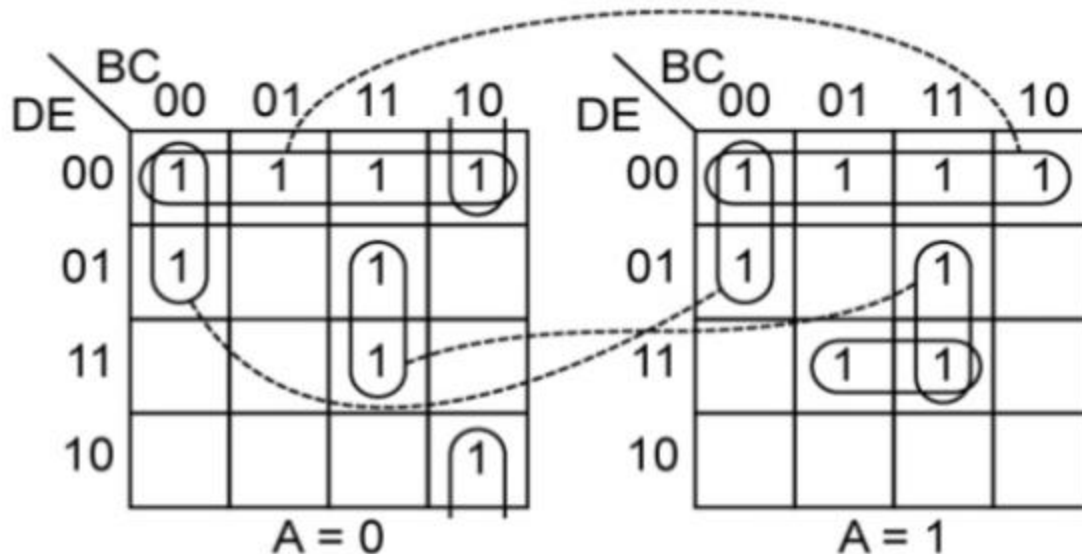


Optimized Implementation of Logic Functions – Karnaugh Map

Five-Variable K-Map

Example 2. Simplify the Boolean function

$$F(A,B,C,D,E) = \sum m(0, 1, 4, 8, 10, 12, 13, 15, 16, 17, 20, 23, 24, 28, 29, 31)$$



$$F = D'E' + B'C'D' + BCE + A'BC'E' + ACDE$$

Optimized Implementation of Logic Functions – Karnaugh Map

of Adjacent Squares and # of Literals

- The relationship between the number of adjacent squares and the number of literals in term-

K	Number of Adjacent Squares 2^k	Number of Literals in a Term in an n-variable Map			
		$n = 2$	$n = 3$	$n = 4$	$n = 5$
0	1	2	3	4	5
1	2	1	2	3	4
2	4	0	1	2	3
3	8		0	1	2
4	16			0	1
5	32				0

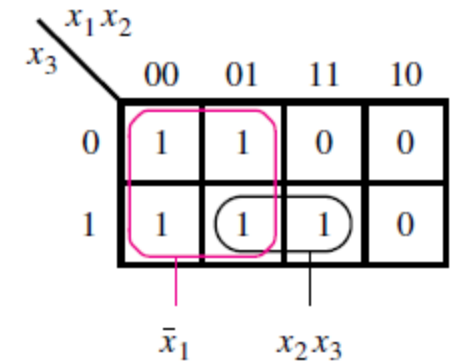
19ECE204/19CSE204
Digital Electronics and Systems
III Semester
B. Tech. (ECE & CSE)
Amrita School of Engineering, Bengaluru
Lecture 9 to 12

4.2 Strategy of Minimization

- Requirement:
 - Find few number of groups as possible
 - Find groups as large as possible groups of 1s (or 0s) that cover all cases where function has a value of 1 (or 0) -> results in fewer number of input variables in the corresponding product term (or sum term)
- Why Strategy of Minimization?
 - When many input variables are there in a function -> minimum cost implementation need to be derived

Terminology

- **Literal:** Each appearance of a variable, either uncomplemented or complemented form is called a Literal
 - Example: $x_1'x_2'x_3x_4$ has 4 literals
- **Implicant:** A product term that indicates the input the input valuation for which a given function is equal to 1 is called an implicant of the function. The most basic implicants are either minterms for Product terms (and maxterms in case of sum terms). For an n-variable function, a minterm is an implicant that consist of n literals.
 - Example: five minterms: $x_1'x_2'x_3'$, $x_1'x_2'x_3$, $x_1'x_2x_3'$, $x_1'x_2x_3$, and $x_1x_2x_3$. Finally, there are two implicants: First implicant that covers a group of four minterms results into x_1' and second implicant that covers group of two minterms results into x_2x_3



x_1x_2		00	01	11	10
x_3	0	1	1	0	0
	1	1	1	1	0
		$\bar{x}_1$		x_2x_3	

Three-variable function $f(x_1, x_2, x_3) = \sum m(0, 1, 2, 3, 7)$.

Terminology

- **Prime Implicant:** An implicant is called a *prime implicant* if it cannot be combined into another implicant that has fewer literals. Another way of stating this definition is to say that it is impossible to delete any literal in a prime implicant and still have a valid implicant.
- **Example:** there are two prime implicants: x_1' and x_2x_3 . It is not possible to delete a literal in either of them.
- **Cover:** A collection of implicants that account for all valuations for which a given function is equal to 1 (or 0 in case of POS) is called a *cover* of that function. A number of different covers exist for most functions. A set of all prime implicants is a cover. A cover defines a particular implementation of the function. $f = x_1' + x_2x_3$ is a cover here in this example.

		x_1x_2				
		x_3	00	01	11	10
0			1	1	0	0
1			1	1	1	0
			$\bar{x}_1$		x_2x_3	

Three-variable function $f(x_1, x_2, x_3) = \sum m(0, 1, 2, 3, 7)$.

Terminology

- **Cost:** Cost of a logic circuit is the number of gates plus the total number of inputs to all gates in the circuit. we will assume that primary inputs, namely, the input variables, are available in both true and complemented forms at zero cost.
- Example: The expression $f = x_1x_2' + x_3x_4'$ has a cost of 9 because it can be implemented using two AND gates and one OR gate.
- $g = (x_1x_2' + x_3)'(x_4' + x_5)$ *Cost = 9 input wires + 5 gates = 14*
- In general, the larger the circuit, the more important the cost issue becomes.
- The main objective is to obtain a minimum-cost circuit.

Strategy for Minimization

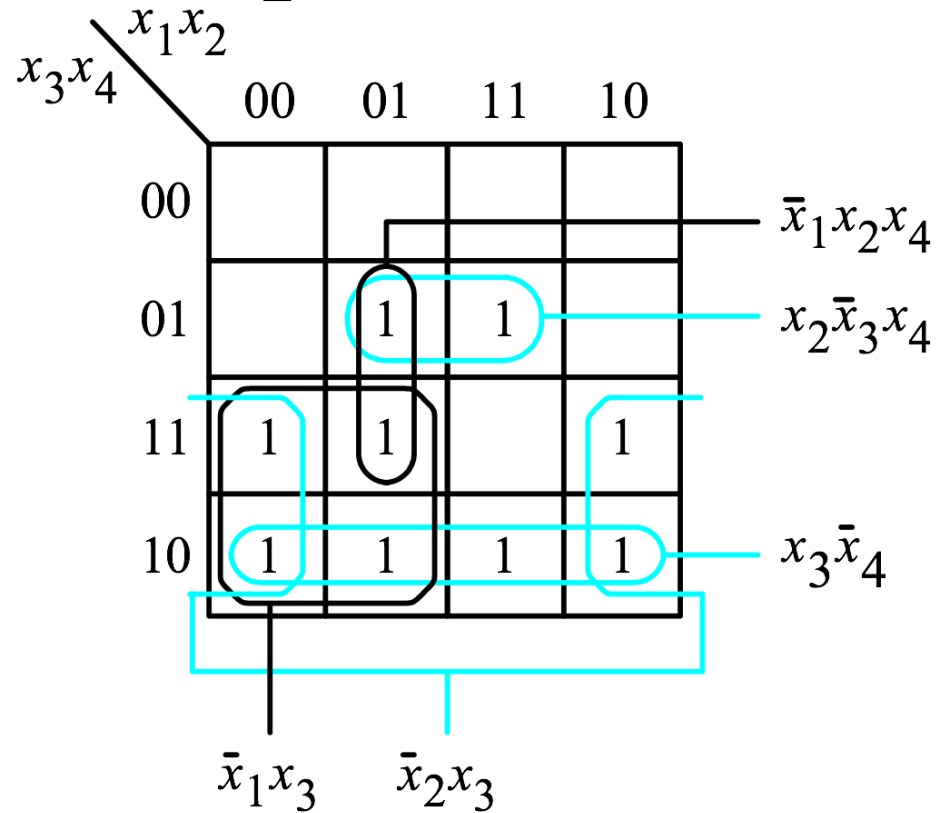
- If a prime implicant includes a minterm for which $f = 1$ that is not included in any other prime implicant, then it must be included in the cover and is called an *essential prime implicant*.
- Example: The prime implicants are essential.
- The term x_2x_3 is the only prime implicant that covers the minterm m_7 , and x_1' is the only one that covers the minterms m_0 , m_1 , and m_2 . Notice that the minterm m_3 is covered by both of these prime implicants. The minimum-cost realization of the function is
- $f = x_1' + x_2x_3$

x_1x_2 x_3					
		00	01	11	10
0		1	1	0	0
1		1	1	1	0

$\bar{x}_1$ x_2x_3

Three-variable function $f(x_1, x_2, x_3) = \sum m(0, 1, 2, 3, 7)$.

Example 1 for Minimization



$$PI = \{x_1'x_3, x_2'x_3, x_3x_4', x_2x_3'x_4, x_1'x_2x_4\}$$

$$EPI = \{x_3x_4', x_2'x_3, x_2x_3'x_4\}$$

$$Cover = \{x_3x_4', x_2'x_3, x_1'x_3, x_2x_3'x_4\}$$

$$f = \bar{x}_2x_3 + x_3\bar{x}_4 + x_2\bar{x}_3x_4 + \bar{x}_1x_3$$

$$Cost = 13 + 5 = 18$$

Figure 4.10. Four-variable function $f(x_1, \dots, x_4) = \Sigma m(2, 3, 5, 6, 7, 10, 11, 13, 14)$.

- The process of finding a minimum-cost circuit involves the following steps:
 1. Generate all prime implicants for the given function f .
 2. Find the set of essential prime implicants.
 3. If the set of essential prime implicants covers all valuations for which $f = 1$, then this set is the desired cover of f . Otherwise, determine the nonessential prime implicants that should be added to form a complete minimum-cost cover. The choice of nonessential prime implicants to be included in the cover is governed by the cost considerations.

Example 2

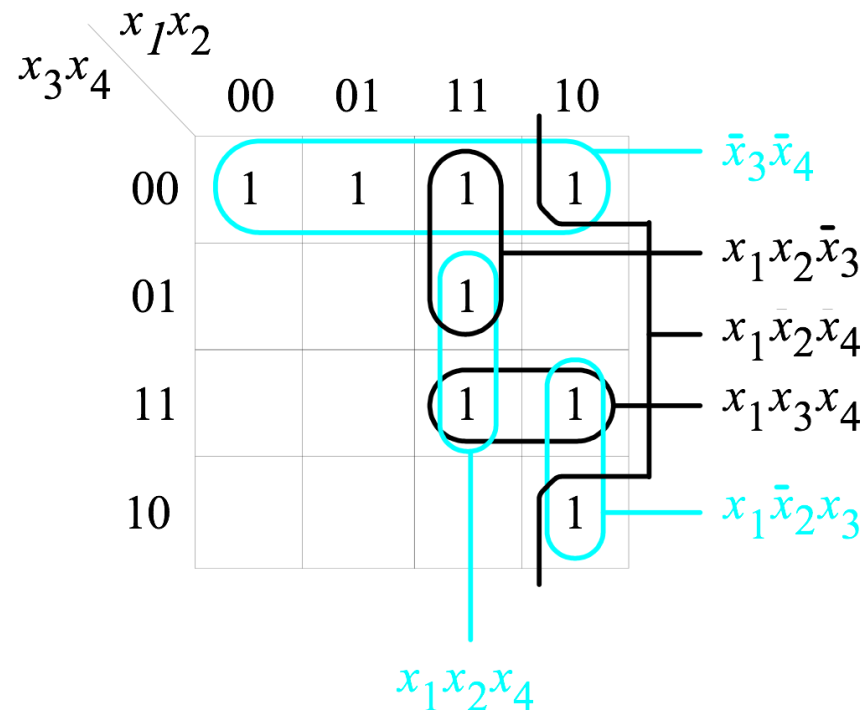


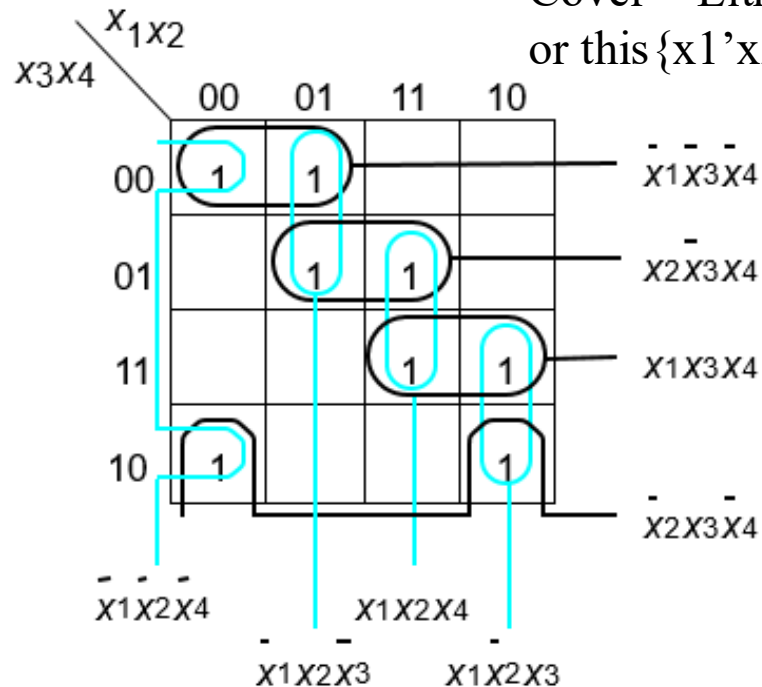
Figure 4.11. The function $f(x_1, \dots, x_4) = \Sigma m(0, 4, 8, 10, 11, 12, 13, 15)$.

Example 3

PI = { $x_1'x_3'x_4'$, $x_2x_3'x_4$, $x_1x_3x_4$, $x_2'x_3x_4'$, $x_1'x_2'x_4'$, $x_1'x_2x_3'$, $x_1x_2x_4$, $x_1x_2'x_3$ }

EPI = {No EPI}

Cover = Either this { $x_1'x_3'x_4'$, $x_2x_3'x_4$, $x_1x_3x_4$, $x_2'x_3x_4'$ }
or this { $x_1'x_2'x_4'$, $x_1'x_2x_3'$, $x_1x_2x_4$, $x_1x_2'x_3$ }



Sometimes there may not be any essential prime implicants at all. An example is given in Figure 4.12. Choosing any of the prime implicants and first including it, then excluding it from the cover leads to two alternatives of equal cost. One includes the prime implicants indicated in black, which yields

$$f = \bar{x}_1\bar{x}_3\bar{x}_4 + x_2\bar{x}_3x_4 + x_1x_3x_4 + \bar{x}_2x_3\bar{x}_4$$

The other includes the prime implicants indicated in blue, which yields

$$f = \bar{x}_1\bar{x}_2\bar{x}_4 + \bar{x}_1x_2\bar{x}_3 + x_1x_2x_4 + x_1\bar{x}_2x_3$$

Figure 4.12. The function $f(x_1, \dots, x_4) = \sum m(0, 2, 4, 5, 10, 11, 13, 15)$.

Minimization of Product-of-Sums Forms

- To find the minimum-cost sum-of-products (SOP) implementations of functions, we can use the same techniques and the principle of duality to obtain minimum cost product-of-sums (POS) implementations. In this case it is the maxterms for which $f = 0$ that have to be combined into sum terms that are as large as possible. Again, a sum term is considered larger if it covers more maxterms, and the larger the term, the less costly it is to implement.

Example 4

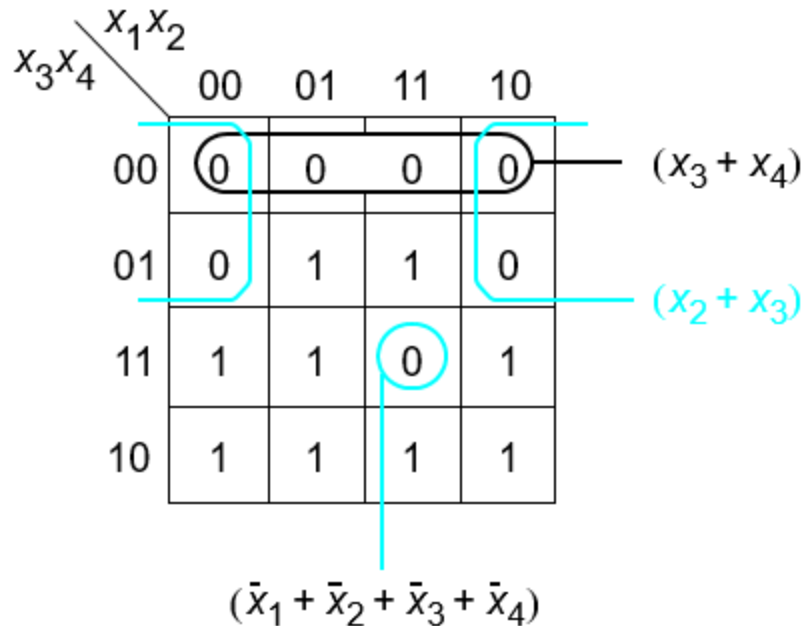
There are three maxterms that must be covered: M_4 , M_5 , and M_6 .
They can be covered by two sum terms shown in the figure, leading to the following implementation:

$x_1 x_2$					
		00	01	11	10
x_3	0	1	1	0	0
	1	1	1	1	0

$(\bar{x}_1 + x_3)$ (points to the 0s in row $x_3=0$)
 $(\bar{x}_1 + x_2)$ (points to the 0s in column $x_2=10$)

$$f = (\bar{x}_1 + x_2)(\bar{x}_1 + x_3)$$

Example 5



$$f = (x_2 + x_3)(x_3 + x_4)(\bar{x}_1 + \bar{x}_2 + \bar{x}_3 + \bar{x}_4)$$

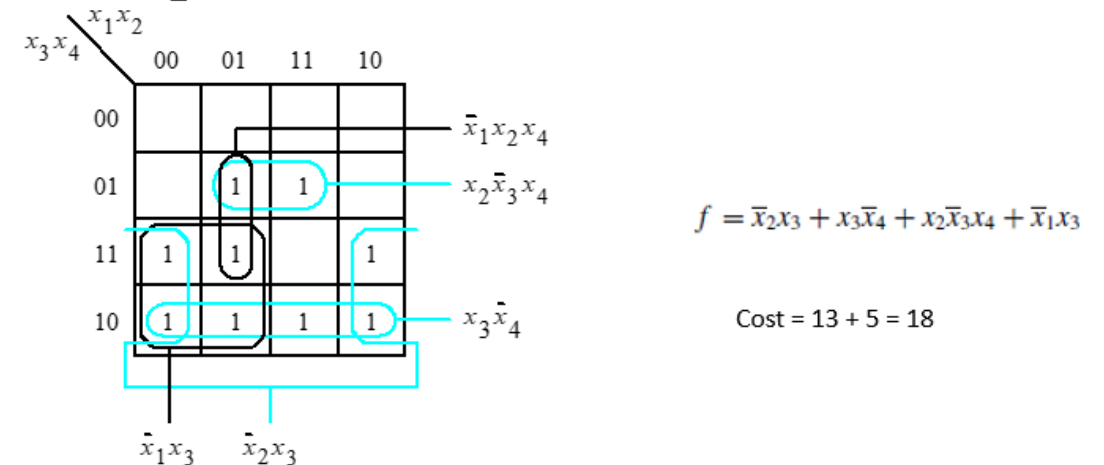
Figure 4.14. POS minimization of $f(x_1, \dots, x_4) = \prod M(0, 1, 4, 8, 9, 12, 15)$.

Assuming that both the complemented and uncomplemented versions of the input variables x_1 to x_4 are available at no extra cost, the cost of this circuit in POS is $11 + 4 = 15$.

But for the same function in the SOP implementation requires cost = $13 + 5 = 18$.

Hence, for the given function, the minimum cost implementation solution is POS

Example 1 for Minimization



$$f = \bar{x}_2x_3 + x_3\bar{x}_4 + x_2\bar{x}_3x_4 + \bar{x}_1x_3$$

$$\text{Cost} = 13 + 5 = 18$$

Figure 4.10. Four-variable function $f(x_1, \dots, x_4) = \sum m(2, 3, 5, 6, 7, 10, 11, 13, 14)$.

Incompletely Specified Functions

- In digital systems it often happens that certain input conditions can never occur and is a *don't-care condition*. A function that has don't-care condition(s) is said to be *incompletely specified*. Don't-care conditions, or *don't cares* for short, can be used to advantage in the design of logic circuits. Since these input valuations will never occur, the designer may assume that the function value for these valuations is either 1 or 0, whichever is more useful in trying to find a minimum-cost implementation.
- If there are k don't cares, then there are 2^k (*2 to the power k*) possible ways of assigning 0 or 1 values to them.

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00	0	1	d	0
01	0	1	d	0
11	0	0	d	0
10	1	1	d	1

(a) SOP implementation

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00	0	1	d	0
01	0	1	d	0
11	0	0	d	0
10	1	1	d	1

(b) POS implementation

$$f(x_1, \dots, x_4) = \sum m(2, 4, 5, 6, 10) + D(12, 13, 14, 15)$$

where D is the set of don't cares.

Part (a) of the figure indicates the best sum-of-products implementation. To form the largest possible groups of 1s, thus generating the lowest-cost prime implicants, it is necessary to assume that the don't cares $D12$, $D13$, and $D14$ (corresponding to minterms $m12$, $m13$, and $m14$) have the value of 1 while $D15$ has the value of 0. Then there are only two prime implicants, which provide a complete cover of f . The resulting implementation is

$$f = x_2\bar{x}_3 + x_3\bar{x}_4$$

Part (b) shows how the best product-of-sums implementation can be obtained. The same values are assumed for the don't cares. The result is

$$f = (x_2 + x_3)(\bar{x}_3 + \bar{x}_4)$$

Figure 4.15 Two implementations of the function $f(x_1, \dots, x_4) = \sum m(2, 4, 5, 6, 10) + D(12, 13, 14, 15)$.

Multiple-Output Circuits

- In practical digital systems it is necessary to implement a number of functions as part of some large logic circuit. Circuits that implement these functions can often be combined into a less-expensive single circuit with multiple outputs by sharing some of the gates needed in the implementation of individual functions.

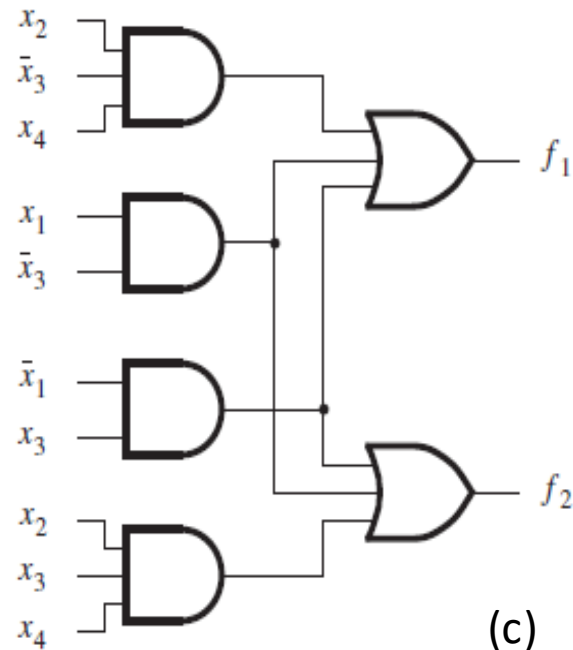
Example

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00			1	1
01		1	1	1
11	1	1		
10	1	1		

(a) Function f_1

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00			1	1
01			1	1
11	1	1		
10	1	1		

(b) Function f_2



(c)

- The cost of $f_1 = 4$ gates + 10 inputs wires = 14.
- The cost of f_2 is the same.
- Thus, the total cost is 28 if both functions are implemented by separate circuits.
- Low cost realization is possible if the two circuits are combined into a single circuit with two outputs.
- Because the first two product terms are identical in both expressions, the AND gates that implement them need not be duplicated. The combined circuit is shown in (c). Its cost is six gates and 16 inputs, for a total of 22.

$$f_1 = x_1\bar{x}_3 + \bar{x}_1x_3 + x_2\bar{x}_3x_4$$

$$f_2 = x_1\bar{x}_3 + \bar{x}_1x_3 + x_2x_3x_4$$

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00				
01	1	1	1	
11	1	1	1	
10		1		

(a) Optimal realization of f_3

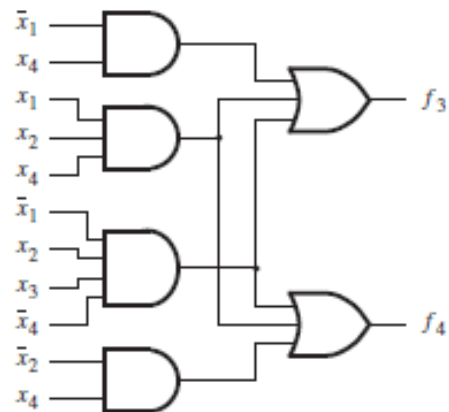
$x_3x_4 \backslash x_1x_2$	00	01	11	10
00				
01	1		1	1
11	1		1	1
10		1		

(b) Optimal realization of f_4

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00				
01	1	1	1	
11	1	1	1	
10		1		

(c) Optimal realization of f_3 and f_4 together

$x_3x_4 \backslash x_1x_2$	00	01	11	10
00				
01	1		1	1
11	1		1	1
10		1		



$$f_3 = \bar{x}_1x_4 + x_2x_4 + \bar{x}_1x_2x_3$$

$$f_4 = x_1x_4 + \bar{x}_2x_4 + \bar{x}_1x_2x_3\bar{x}_4$$

$$f_3 = x_1x_2x_4 + \bar{x}_1x_2x_3\bar{x}_4 + \bar{x}_1x_4$$

$$f_4 = x_1x_2x_4 + \bar{x}_1x_2x_3\bar{x}_4 + \bar{x}_2x_4$$

- Realizations of the individual functions f_3 and f_4 are obtained from parts (a) and (b). None of the AND gates can be shared, which means that the cost of the combined circuit = 6 AND gates + 2 OR gates + 21 inputs = 29.
- The combined realization of the functions in (c), shows that the first two implicants are identical in both expressions. The resulting circuit is given in (d). It has the cost = 6 gates + 17 inputs = 23.

Problems for Practice - Tutorial

- Repeat the above two examples for POS implementation
- 1. f_1 and f_2 in POS
- 2. f_3 and f_4 in POS

19ECE204

Digital Electronics and Systems

III Semester

B. Tech. (ECE & CSE)

Amrita School of Engineering, Bengaluru

Number Representation and Arithmetic Circuits

Overview

- Number Representation
- Unsigned Arithmetic
- Signed Arithmetic
- Arithmetic Overflow
- Fast Adders

Number Representation

- Types of Number Representation (Positional Representation)
 - Decimal - Base 10
 - Binary - Base 2
 - Octal - Base 8
 - Hexadecimal - Base 16
- Not limited to this

Number Representation- Positional Number System

- Lets consider decimal number system
 - Possible different numbers 0-9 - $(0 - \{N-1\}, N \text{ is base})$
 - Each digit represents a multiple of power of 10 (Base)
 - Example: $(8547)_{10} = 8 \times 10^3 + 5 \times 10^2 + 4 \times 10^1 + 7 \times 10^0$
 - This is what we call as one's, ten's, hundred's place etc..
- Similarly all the positional number system can be represented

Number Representation - Conversions

- Conversion from one representation to another is possible

- Binary to Decimal Conversion

$$V = b_{n-1} \times 2^{n-1} + \dots + b_2 \times 2^2 + b_1 \times 2^1 + b_0$$

- V = Final decimal value
- b - binary digits (b_{n-1} is MSB and b_0 is LSB)
- Decimal to Binary Conversion
 - Successive division of decimal number by 2
 - Take the remainder from bottom up as the result

Number Representation - Conversions



- Octal to Binary and Vice versa
 - Octal to Binary is just as straightforward; each octal digit is simply replaced by three bits that denote the same value. (Hint: Use 4-2-1 code)
 - Binary to Octal is by grouping binary numbers as group of three, then write the corresponding group value in octal. (Hint: Use 4-2-1 code)

Number Representation - Conversions



- Hexadecimal to Binary and Vice Versa
 - Each hexadecimal digit is simply replaced by four bits that denote the same value. (Hint: Use 8-4-2-1 code)
 - Binary to Hexadecimal is by grouping binary numbers as group of four, then write the corresponding group value in hexadecimal. (Hint: Use 8-4-2-1 code)
- In computers binary is dominant, but octal and hexadecimal is used as short hand notation for binary

Number Representation - Examples



Convert $(857)_{10}$

			Remainder	
$857 \div 2$	$=$	428	1	LSB
$428 \div 2$	$=$	214	0	
$214 \div 2$	$=$	107	0	
$107 \div 2$	$=$	53	1	
$53 \div 2$	$=$	26	1	
$26 \div 2$	$=$	13	0	
$13 \div 2$	$=$	6	1	
$6 \div 2$	$=$	3	0	
$3 \div 2$	$=$	1	1	
$1 \div 2$	$=$	0	1	MSB

Result is $(1101011001)_2$

Number Representation - Examples



$$\begin{array}{cccc} \underbrace{101} & \underbrace{011} & \underbrace{010} & \underbrace{111} \\ 5 & 3 & 2 & 7 \end{array}$$

$$(101011010111)_2 = (5327)_8$$

$$\begin{array}{cccc} \underbrace{1010} & \underbrace{1111} & \underbrace{0010} & \underbrace{0101} \\ A & F & 2 & 5 \end{array}$$

$$(1010111100100101)_2 = (AF25)_{16}$$

Number Representation

Decimal	Binary	Octal	Hexadecimal
00	00000	00	00
01	00001	01	01
02	00010	02	02
03	00011	03	03
04	00100	04	04
05	00101	05	05
06	00110	06	06
07	00111	07	07
08	01000	10	08
09	01001	11	09
10	01010	12	0A
11	01011	13	0B
12	01100	14	0C
13	01101	15	0D
14	01110	16	0E
15	01111	17	0F
16	10000	20	10
17	10001	21	11
18	10010	22	12

Unsigned Arithmetic

- Numbers that are positive only are called unsigned, and numbers that can also be negative are called signed.
- We will deal with binary arithmetic in this course
- Unsigned Binary Addition and Subtraction
 - Binary addition is performed in the same way as decimal addition except that the values of individual digits can be only 0 or 1

Unsigned Arithmetic - Addition

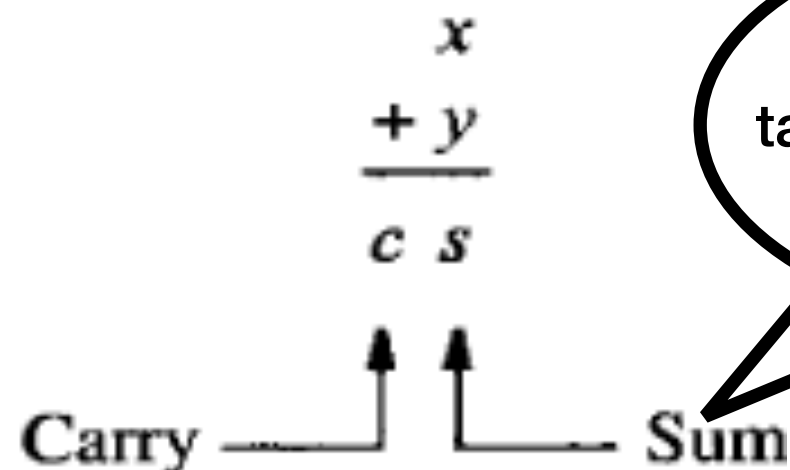
- Single Digit Addition, all possible combinations

$$\begin{array}{r} 0 \\ + 0 \\ \hline 00 \end{array}$$

$$\begin{array}{r} 0 \\ + 1 \\ \hline 01 \end{array}$$

$$\begin{array}{r} 1 \\ + 0 \\ \hline 01 \end{array}$$

$$\begin{array}{r} 1 \\ + 1 \\ \hline 10 \end{array}$$



Can we get a truth table for one bit unsigned addition ??

Unsigned Arithmetic - Half Adder

This circuit, which implements the addition of only two bits, is called a half-adder.

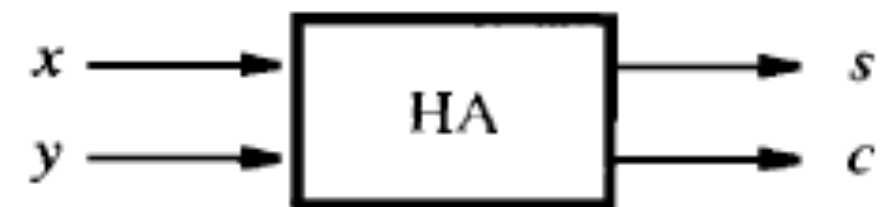
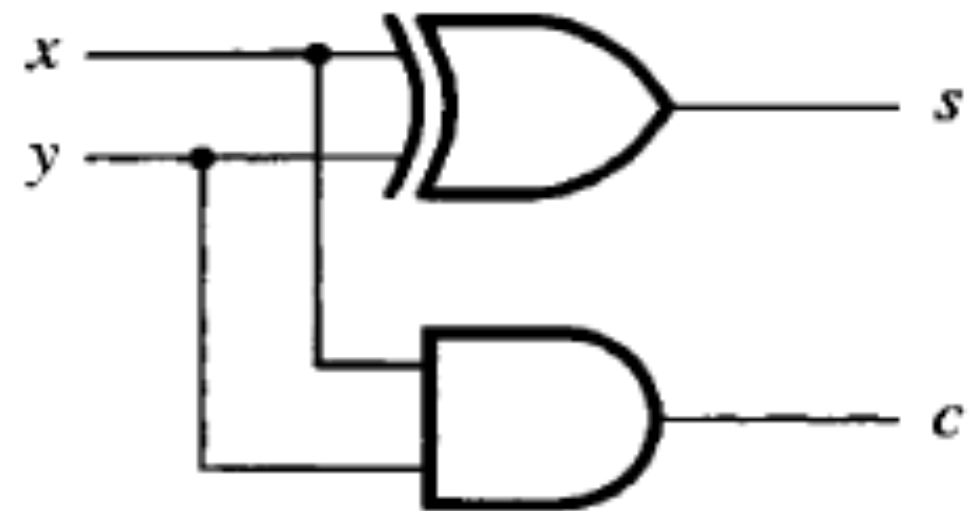
x	0	0	1	1
$+ y$	$+ 0$	$+ 1$	$+ 0$	$+ 1$
$c \ s$	0 0	0 1	0 1	1 0

Carry $\uparrow$ Sum $\uparrow$

x	y	Carry c	Sum s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$s = xy' + x'y$$

$$c = xy$$



Unsigned Arithmetic - Multi bit



By Hand Method

Generated carries	1 1 1 0	
$X = x_4x_3x_2x_1x_0$	0 1 1 1 1	$(15)_{10}$
$+ Y = y_4y_3y_2y_1y_0$	0 1 0 1 0	$(10)_{10}$
$S = s_4s_3s_2s_1s_0$	1 1 0 0 1	$(25)_{10}$

How to design a digital circuit for this ???

Unsigned Arithmetic - Multi bit



Generated carries	1 1 1 0	
$X = x_4x_3x_2x_1x_0$	0 1 1 1 1	$(15)_{10}$
$+ Y = y_4y_3y_2y_1y_0$	0 1 0 1 0	$(10)_{10}$
$S = s_4s_3s_2s_1s_0$	1 1 0 0 1	$(25)_{10}$

- Regular truth table approach would be impractical
 - The required truth table would have 10 input variables, S for each number X and Y. It would have $2^{10} = 1024$ rows!
 - A better approach is to consider the addition of each pair of bits, X_i and Y_i , separately.

Unsigned Arithmetic - Multi bit



	Generated carries	1 1 1 0	
$X = x_4x_3x_2x_1x_0$		0 1 1 1 1	$(15)_{10}$
$+ Y = y_4y_3y_2y_1y_0$		0 1 0 1 0	$(10)_{10}$
$S = s_4s_3s_2s_1s_0$		1 1 0 0 1	$(25)_{10}$

- For bit position 0, there is no carry-in, and hence the addition is the same as Half Adder
- For each other bit position i . the addition involves bits X_i and Y_i , and a carry-in C_i
- This is a three variable function, for which truth table can be derived.

Unsigned Arithmetic - Full

This circuit, which implements the addition of only three bits, is called a Full-adder.

Adder

c_i	x_i	y_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

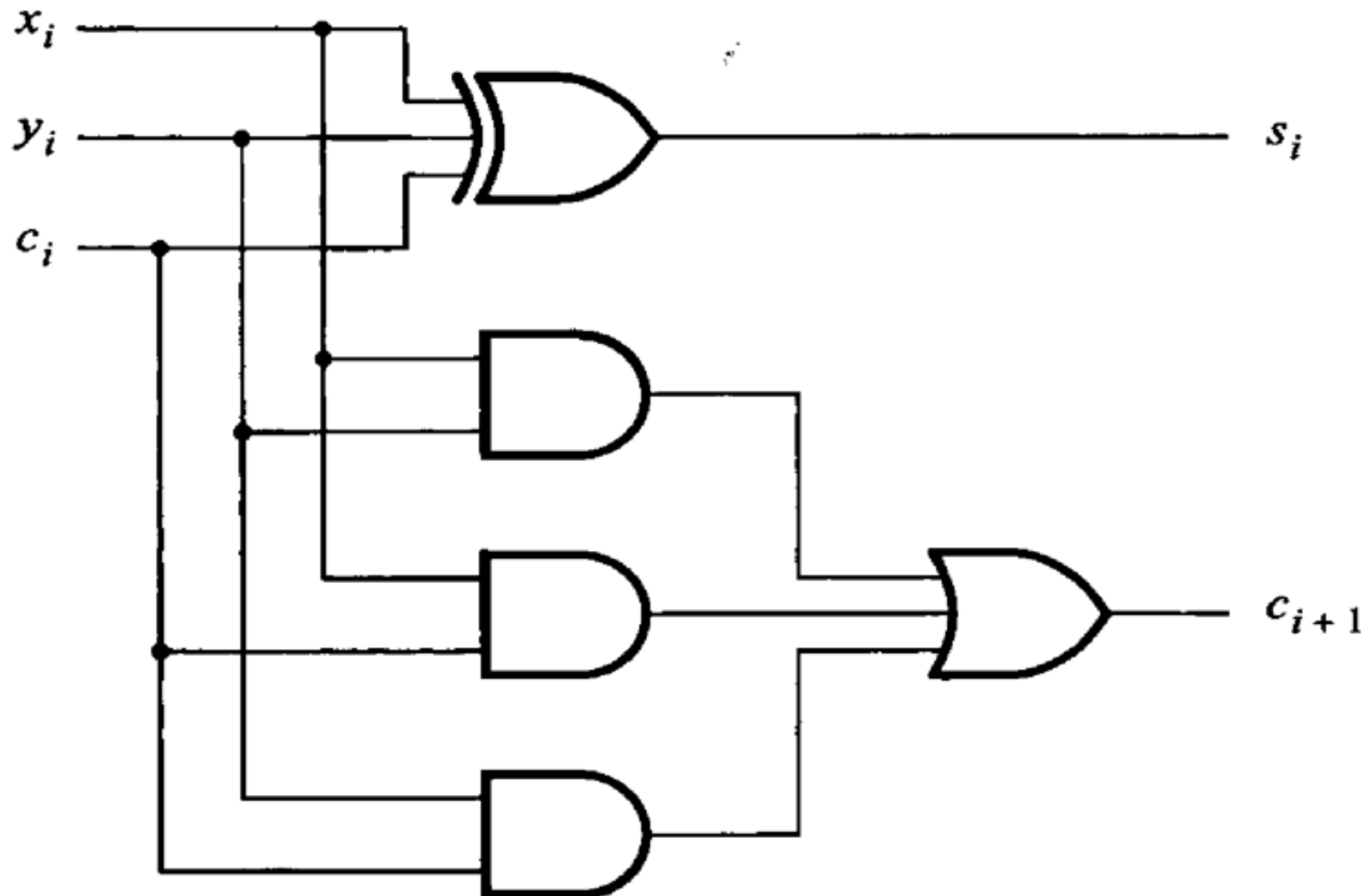
$c_i \backslash x_i y_i$	00	01	11	10
0		1		1
1	1		1	

$$s_i = x_i \oplus y_i \oplus c_i$$

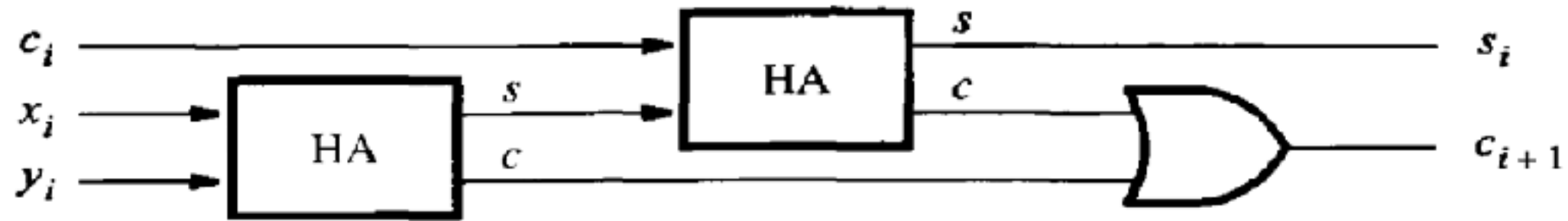
$c_i \backslash x_i y_i$	00	01	11	10
0			1	
1		1	1	1

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

Unsigned Arithmetic - Full Adder

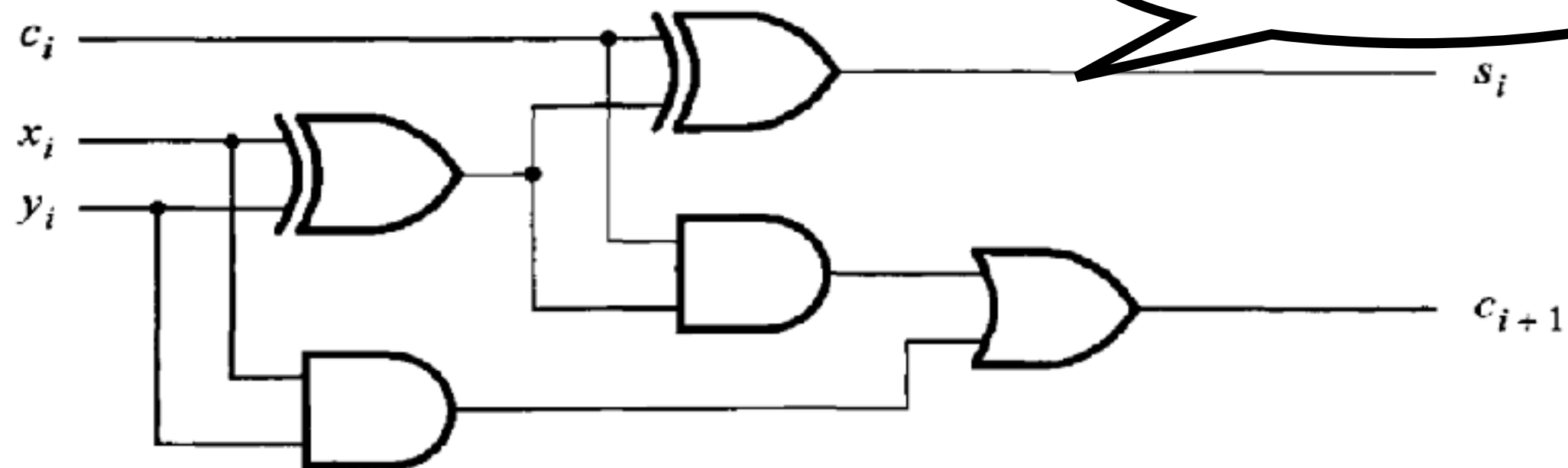


Unsigned Arithmetic - Decomposed Full Adder



(a) Block diagram

2 Half Adder = 1 Full Adder

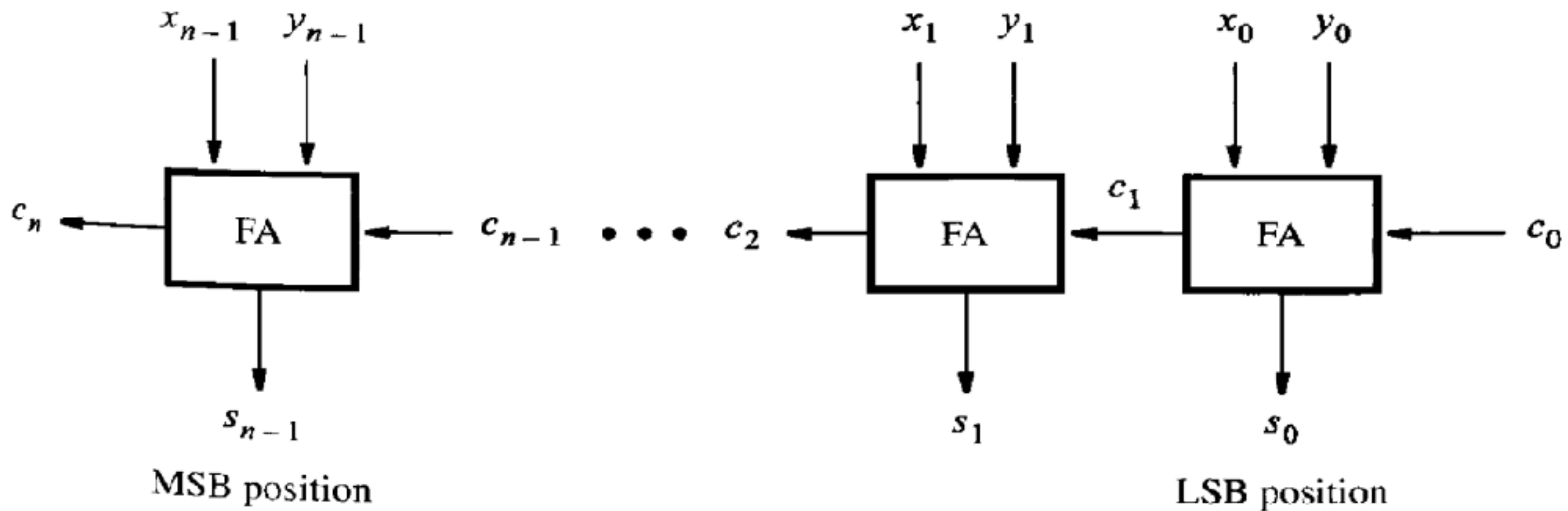


(b) Detailed diagram

Unsigned Arithmetic - Ripple Carry Adder

- To perform addition by hand, we start from the least-significant digit and add pairs of digits, progressing to the most-significant digit
- If a carry is produced in position i , then this carry is added to the operands in position $i + 1$
- The same arrangement can be used in a logic circuit that performs addition.

Unsigned Arithmetic - Ripple Carry Adder



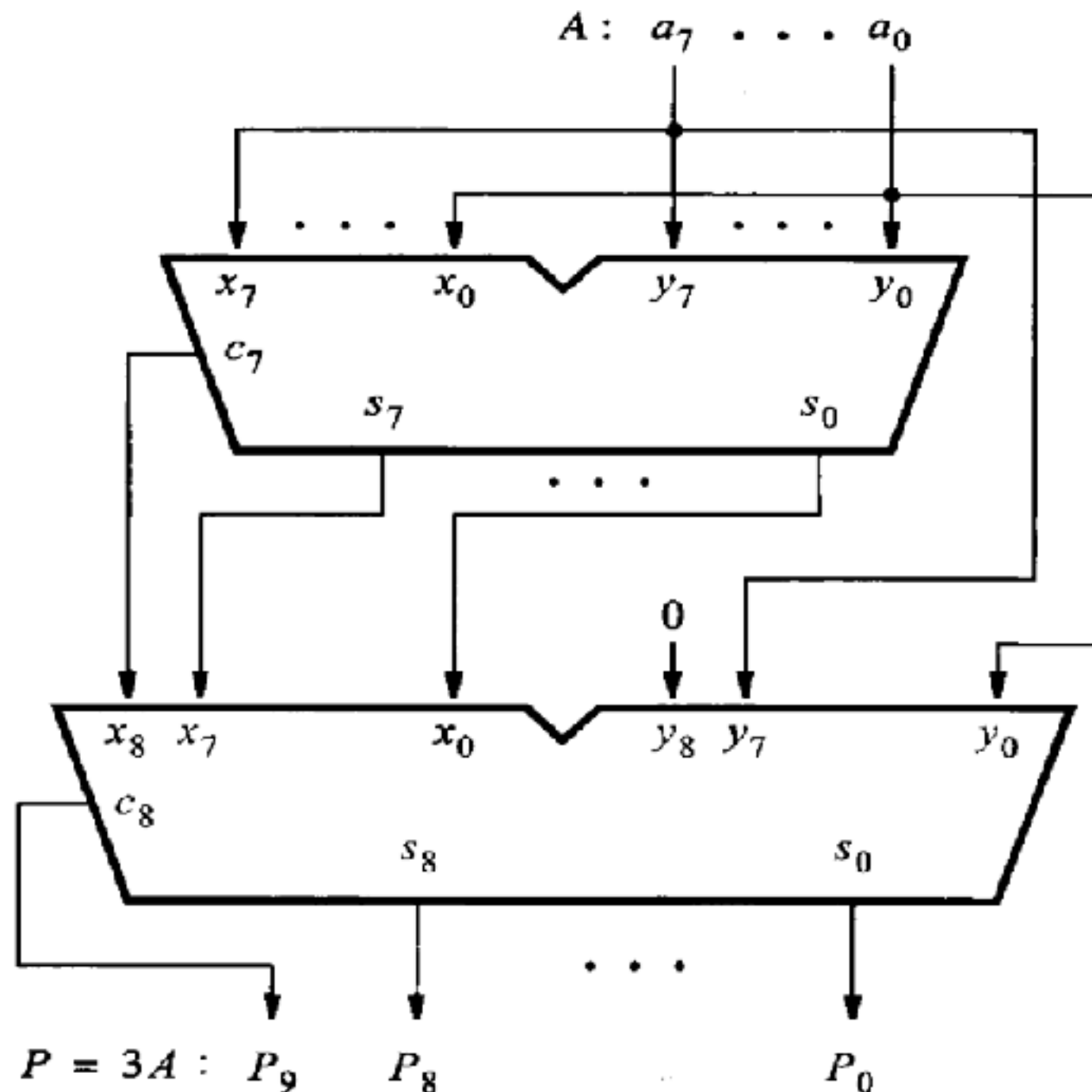
Unsigned Arithmetic - Ripple Carry Adder

- When the operands X and Y are applied as inputs to the adder, it takes some time before the output sum, S , is valid.
- Each full-adder introduces a certain delay before its S_i and C_{i+1} outputs are valid. Let this delay be denoted as Δt .
- Thus the carry-out from the first stage, C_1 , arrives at the second stage Δt after the application of the x_0 and y_0 inputs. The carry-out from the second stage, C_2 , arrives at the third stage with a $2\Delta t$ delay, and so on.
- The signal C_{n-1} is valid after a delay of $(n - 1)\Delta t$, which means that the complete sum is available after a delay of $n\Delta t$.

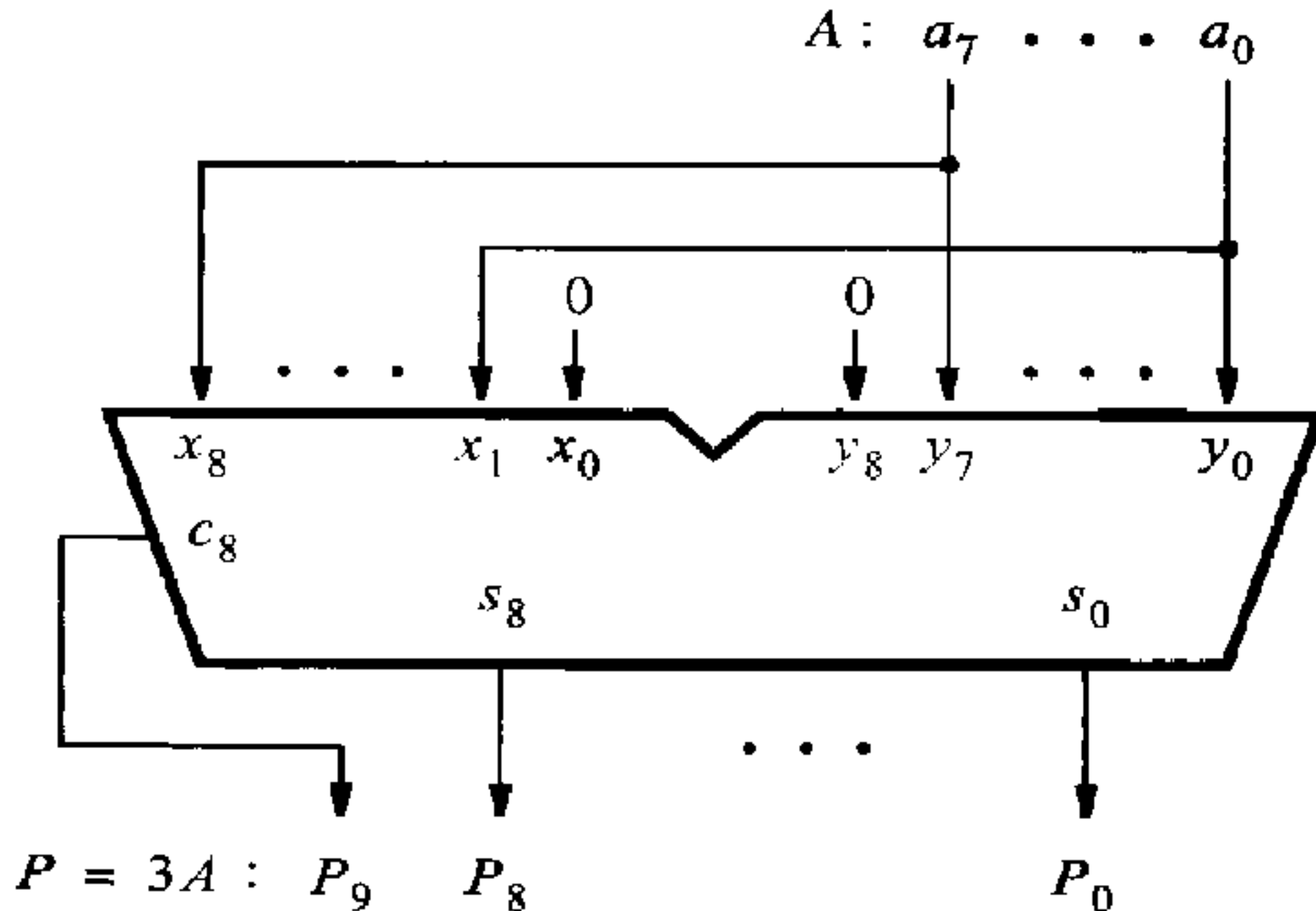
Unsigned Arithmetic - Design Example

- Design a multiplier that multiplies an eight bit unsigned number by 3.
- $P = 3A$
 - $A = a_7, a_6, \dots, a_0$
 - $P = p_7, p_6, \dots, p_0$

Unsigned Arithmetic - Design Example



Unsigned Arithmetic - Design Example



Unsigned Arithmetic - Half and Full Subtractor

- Design a Half Subtractor circuit with two inputs X, Y and two outputs D (Difference), B(Borrow)

- Hint:

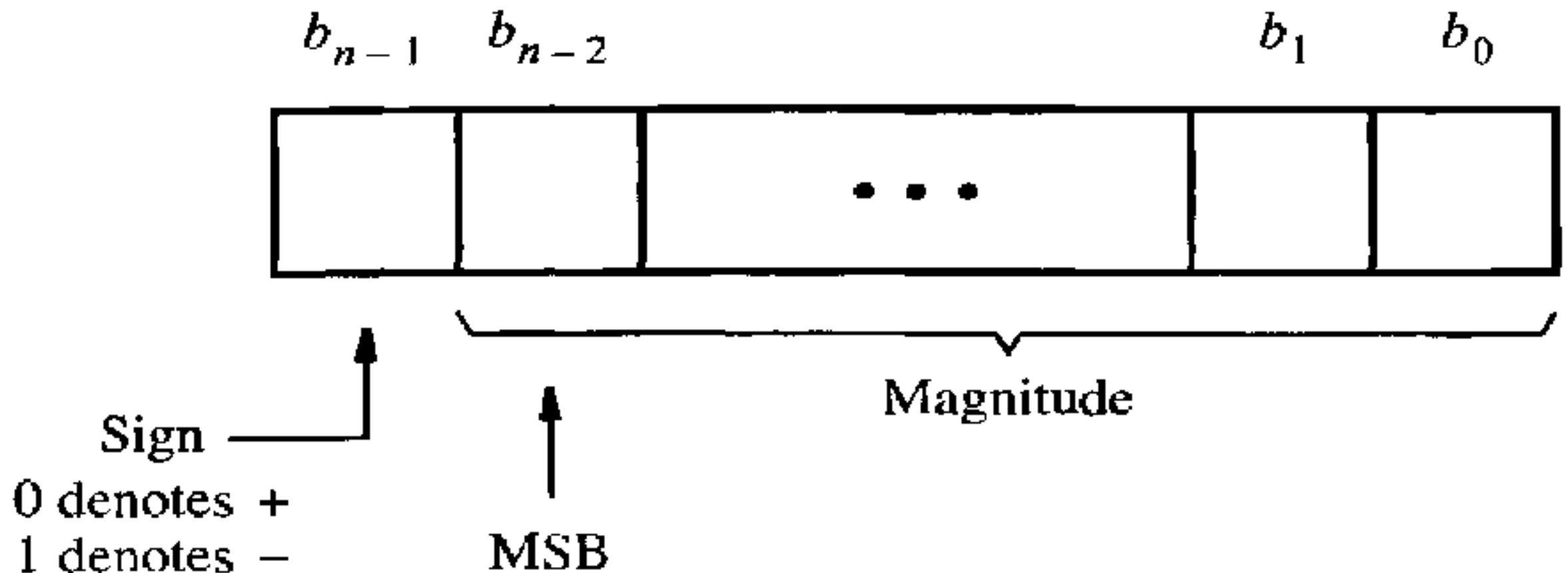
HS - Truth Table			
X	Y	B	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

- Design a Full Subtractor circuit with three inputs X, Y, Z and two outputs D (Difference), B(Borrow)

Signed Number Representation

- Positive numbers are represented using the positional number representation as explained in the previous section.
- Negative numbers can be represented in three different ways:
 - Sign-and-magnitude
 - 1's complement
 - 2's complement.

Signed Number Representation- Sign Magnitude Representation



Example:
 $+5 = 0101$
 $-5 = 1101$

Drawbacks

- Range
- Circuit complexity

Signed Number Representation

- 1's Complement

- In a complementary number system, the negative numbers are defined according to a subtraction operation involving positive numbers.
- In the 1's complement scheme, an n-bit negative number, K, is obtained by subtracting its equivalent positive number, P, from $2^n - 1$
 - $K = (2^n - 1) - P$

Example:

$$-5 = (2^4 - 1) - 0101 = 1111 - 0101 = 1010$$

$$-3 = (2^4 - 1) - 0011 = 1111 - 0011 = 1100$$

Simply it is complement of its positive number

Signed Number Representation

- 2's Complement

- In the 2's complement scheme, a negative number, K, is obtained by subtracting its equivalent positive number, P, from 2^n
 - $K = 2^n - P$

Example:

$$-5 = 2^4 - 0101 = 10000 - 0101 = 1011$$

$$-3 = 2^4 - 0011 = 10000 - 0011 = 1101$$

Simply it is 1's complement plus one

Signed Number Representation

$b_3b_2b_1b_0$	Sign and magnitude	1's complement	2's complement
0111	+7	+7	+7
0110	+6	+6	+6
0101	+5	+5	+5
0100	+4	+4	+4
0011	+3	+3	+3
0010	+2	+2	+2
0001	+1	+1	+1
0000	+0	+0	+0
1000	-0	-7	-8
1001	-1	-6	-7
1010	-2	-5	-6
1011	-3	-4	-5
1100	-4	-3	-4
1101	-5	-2	-3
1110	-6	-1	-2
1111	-7	-0	-1

Signed Arithmetic - Sign Magnitude

- If both operands have the same sign, then the addition of sign-and-magnitude numbers is simple. The magnitudes are added, and the resulting sum is given the sign of the operands.
- However, if the operands have opposite signs, the task becomes more complicated. Then it is necessary to subtract the smaller number from the larger one.
- This means that logic circuits that compare and subtract numbers are also needed.
- For this reason, the sign-and-magnitude representation is not used in computers

Signed Arithmetic - 1's Complement Addition

- An obvious advantage of the 1's complement representation is that a negative number is generated simply by complementing all bits of the corresponding positive number

$$\begin{array}{r} (+5) \\ + (+2) \\ \hline (+7) \end{array} \quad \begin{array}{r} 0101 \\ + 0010 \\ \hline 0111 \end{array}$$

$$\begin{array}{r} (-5) \\ + (+2) \\ \hline (-3) \end{array} \quad \begin{array}{r} 1010 \\ + 0010 \\ \hline 1100 \end{array}$$

$$\begin{array}{r} (+5) \\ + (-2) \\ \hline (+3) \end{array} \quad \begin{array}{r} 0101 \\ + 1101 \\ \hline 10010 \\ \text{Carry } 1 \rightarrow \\ \hline 0011 \end{array}$$

$$\begin{array}{r} (-5) \\ + (-2) \\ \hline (-7) \end{array} \quad \begin{array}{r} 1010 \\ + 1101 \\ \hline 10111 \\ \text{Carry } 1 \rightarrow \\ \hline 1000 \end{array}$$

Adding the extra carry is an overhead

Signed Arithmetic - 2's Complement Addition

$$\begin{array}{r} (+5) \\ + (+2) \\ \hline (+7) \end{array}$$

$$\begin{array}{r} 0101 \\ + 0010 \\ \hline 0111 \end{array}$$

$$\begin{array}{r} (-5) \\ + (+2) \\ \hline (-3) \end{array}$$

$$\begin{array}{r} 1011 \\ + 0010 \\ \hline 1101 \end{array}$$

$$\begin{array}{r} (+5) \\ + (-2) \\ \hline (+3) \end{array}$$

$$\begin{array}{r} 0101 \\ + 1110 \\ \hline 10011 \end{array}$$

↑
ignore

$$\begin{array}{r} (-5) \\ + (-2) \\ \hline (-7) \end{array}$$

$$\begin{array}{r} 1011 \\ + 1110 \\ \hline 11001 \end{array}$$

↑
ignore

No extra addition of carry is required, hence this is more suitable for addition operation

Signed Arithmetic - 2's Complement Subtraction

$$\begin{array}{r}
 (+5) \\
 - (+2) \\
 \hline
 (+3)
 \end{array}
 \quad
 \begin{array}{r}
 0101 \\
 - 0010 \\
 \hline
 \end{array}
 \Rightarrow
 \begin{array}{r}
 0101 \\
 + 1110 \\
 \hline
 10011
 \end{array}$$

↑
ignore

$$\begin{array}{r}
 (-5) \\
 - (+2) \\
 \hline
 (-7)
 \end{array}
 \quad
 \begin{array}{r}
 1011 \\
 - 0010 \\
 \hline
 \end{array}
 \Rightarrow
 \begin{array}{r}
 1011 \\
 + 1110 \\
 \hline
 11001
 \end{array}$$

↑
ignore

$$\begin{array}{r}
 (+5) \\
 - (-2) \\
 \hline
 (+7)
 \end{array}
 \quad
 \begin{array}{r}
 0101 \\
 - 1110 \\
 \hline
 \end{array}
 \Rightarrow
 \begin{array}{r}
 0101 \\
 + 0010 \\
 \hline
 0111
 \end{array}$$

$$\begin{array}{r}
 (-5) \\
 - (-2) \\
 \hline
 (-3)
 \end{array}
 \quad
 \begin{array}{r}
 1011 \\
 - 1110 \\
 \hline
 \end{array}
 \Rightarrow
 \begin{array}{r}
 1011 \\
 + 0010 \\
 \hline
 1101
 \end{array}$$

Hence, Subtraction can be done by adder itself

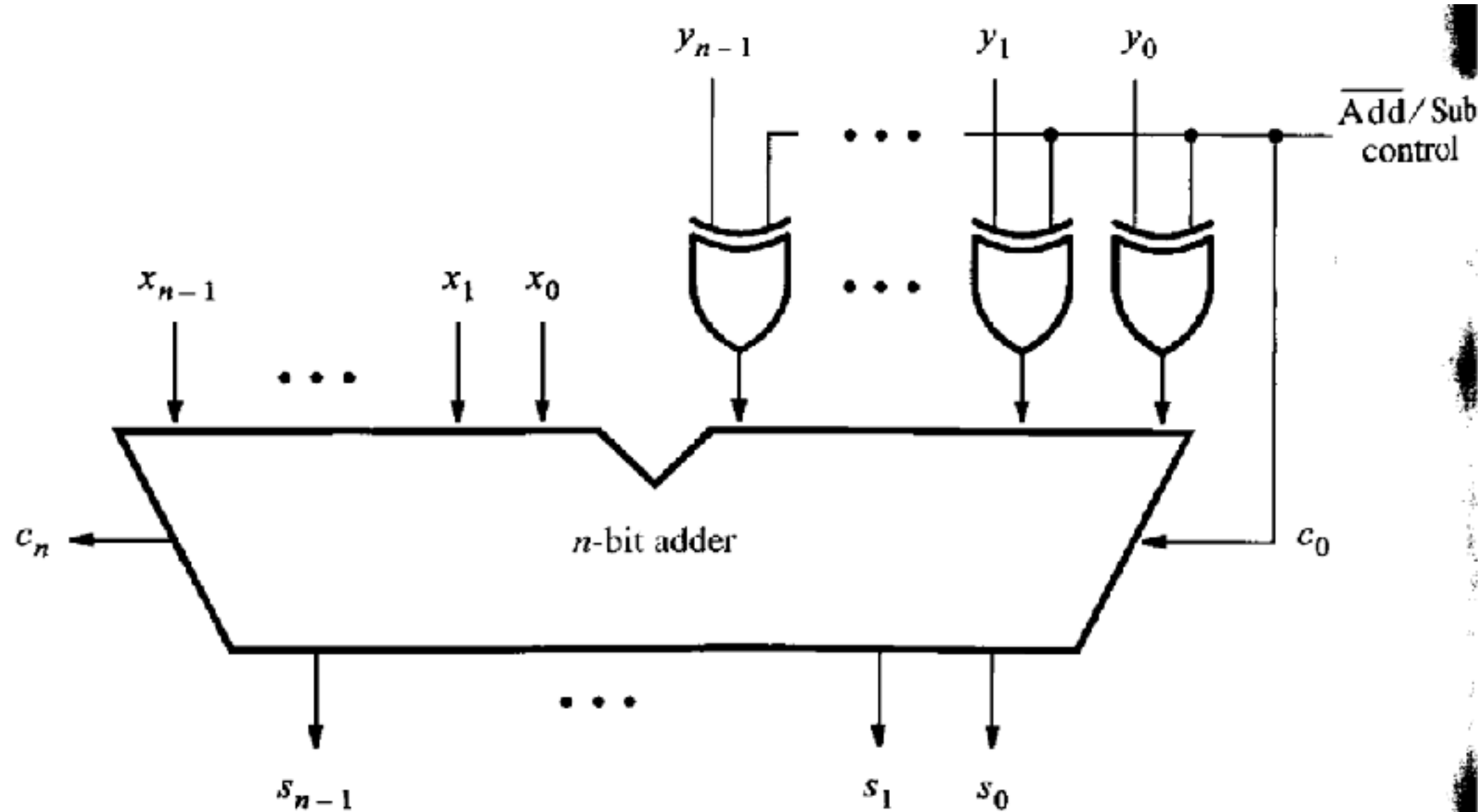
Signed Arithmetic - 2's Complement

- Perform the following
 - $(+10)_{10} + (-12)_{10}$
 - $(+10)_{10} - (+15)_{10}$

+10	2's comp form	01010
-12	2's comp form	10100
		— — — —
-2	2's comp form	11110
		— — — —

+10	2's comp form	01010
-(+15)	2's comp form	10001
		— — — —
-5	2's comp form	11011
		— — — —

Signed Arithmetic - Adder and Subtractor Unit



Arithmetic Overflow

- The result of addition or subtraction is supposed to fit within the significant bits used to represent the numbers.
- If n bits are used to represent signed numbers, then the result must be in the range -2^{n-1} to $2^{n-1} - 1$.
- If the result does not fit in this range, then we say that arithmetic overflow has occurred.
- To ensure the correct operation of an arithmetic circuit, it is important to be able to detect the occurrence of overflow.

Arithmetic Overflow

$$\begin{array}{r}
 (+7) \\
 + (+2) \\
 \hline
 (+9)
 \end{array}
 \quad
 \begin{array}{r}
 0111 \\
 + 0010 \\
 \hline
 1001
 \end{array}$$

$c_4 = 0$
 $c_3 = 1$

$$\begin{array}{r}
 (-7) \\
 + (+2) \\
 \hline
 (-5)
 \end{array}
 \quad
 \begin{array}{r}
 1001 \\
 + 0010 \\
 \hline
 1011
 \end{array}$$

$c_4 = 0$
 $c_3 = 0$

$$\begin{array}{r}
 (+7) \\
 + (-2) \\
 \hline
 (+5)
 \end{array}
 \quad
 \begin{array}{r}
 0111 \\
 + 1110 \\
 \hline
 10101
 \end{array}$$

$c_4 = 1$
 $c_3 = 1$

$$\begin{array}{r}
 (-7) \\
 + (-2) \\
 \hline
 (-9)
 \end{array}
 \quad
 \begin{array}{r}
 1001 \\
 + 1110 \\
 \hline
 10111
 \end{array}$$

$c_4 = 1$
 $c_3 = 0$

$$\begin{aligned}
 \text{Overflow} &= c_3 \bar{c}_4 + \bar{c}_3 c_4 \\
 &= c_3 \oplus c_4
 \end{aligned}$$

$$\text{Overflow} = c_{n-1} \oplus c_n$$

Fast Adders

- The performance of a large digital system is dependent on the speed of circuits that form its various functional units.
- Obviously, better performance can be achieved using faster circuits.
- This can be accomplished by using superior (usually newer) technology in which the delays in basic gates are reduced.
- But it can also be accomplished by changing the overall structure of a functional unit, which may lead to even more impressive improvement.
- To reduce the delay caused by the effect of carry propagation through the ripple-carry adder, we can attempt to evaluate quickly for each stage whether the carry-in from the previous stage will have a value 0 or 1.
- If a correct evaluation can be made in a relatively short time, then the performance of the complete adder will be improved.

Fast Adders - Carry Look Ahead Adder

- Based on Full adder circuit the carry-out function for stage i can be realized as

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

If we factor this expression as

$$c_{i+1} = x_i y_i + (x_i + y_i) c_i$$

then it can be written as

$$c_{i+1} = g_i + p_i c_i$$

where

$$g_i = x_i y_i$$

$$p_i = x_i + y_i$$

—

Fast Adders - Carry Look Ahead Adder

- Expanding in term of $i-1$ gives

$$\begin{aligned}c_{i+1} &= g_i + p_i(g_{i-1} + p_{i-1}c_{i-1}) \\ &= g_i + p_i g_{i-1} + p_i p_{i-1} c_{i-1}\end{aligned}$$

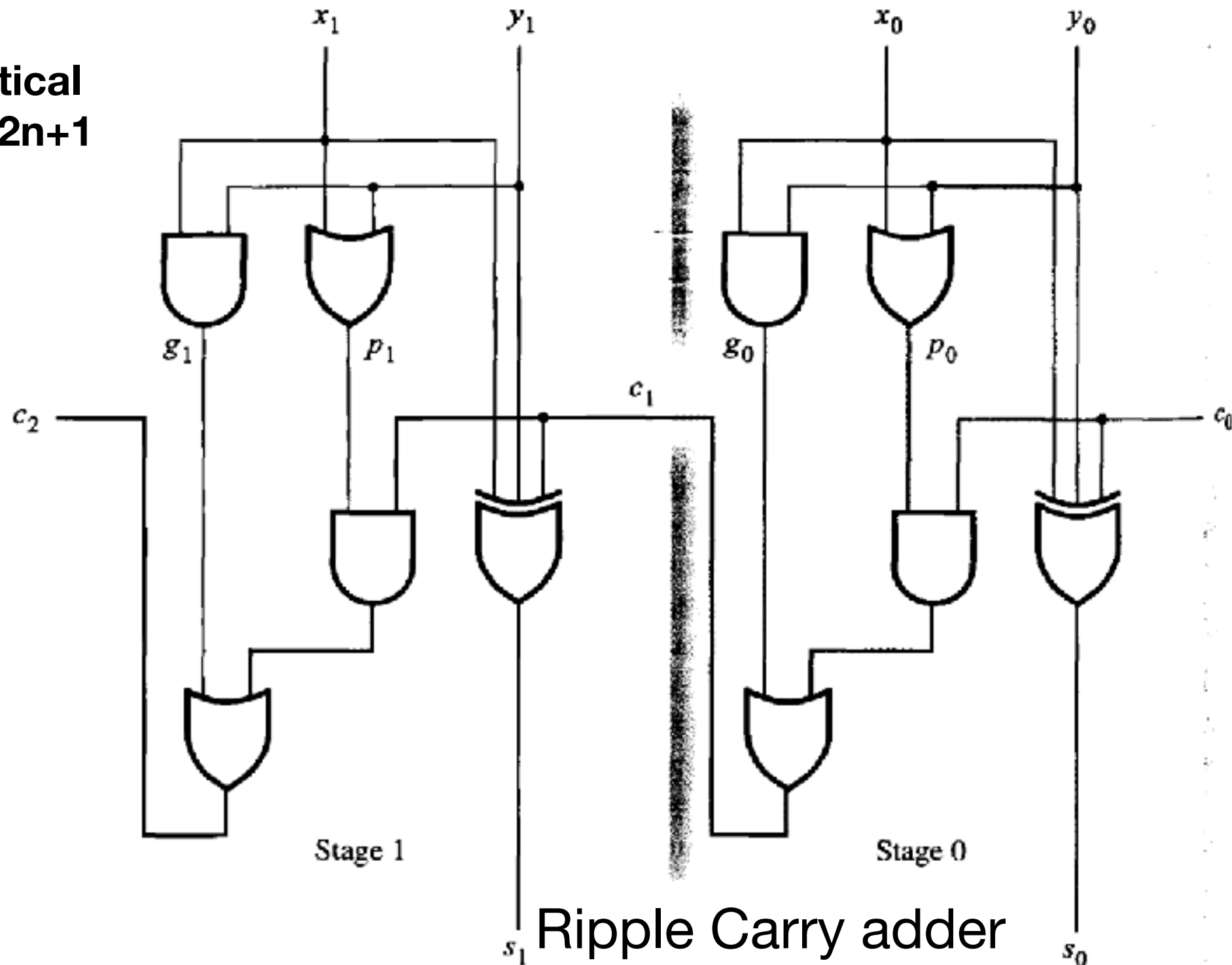
The same expansion for other stages, ending with stage 0, gives

$$c_{i+1} = g_i + p_i g_{i-1} + p_i p_{i-1} g_{i-2} + \cdots + p_i p_{i-1} \cdots p_2 p_1 g_0 + p_i p_{i-1} \cdots p_1 p_0 c_0$$

- This expression represents a two-level AND-OR circuit in which C_{i+1} is evaluated very quickly. An adder based on this expression is called a carry-lookahead adder.

Fast Adders - Carry Look Ahead Adder

Critical Path : From $x_0 - y_0$ to c_2
For n bit critical path will be $2n+1$



Fast Adders - Carry Look Ahead Adder

Critical Path : From $x_0 - y_0$ to c_2

Gate Delay for c_2 - 3

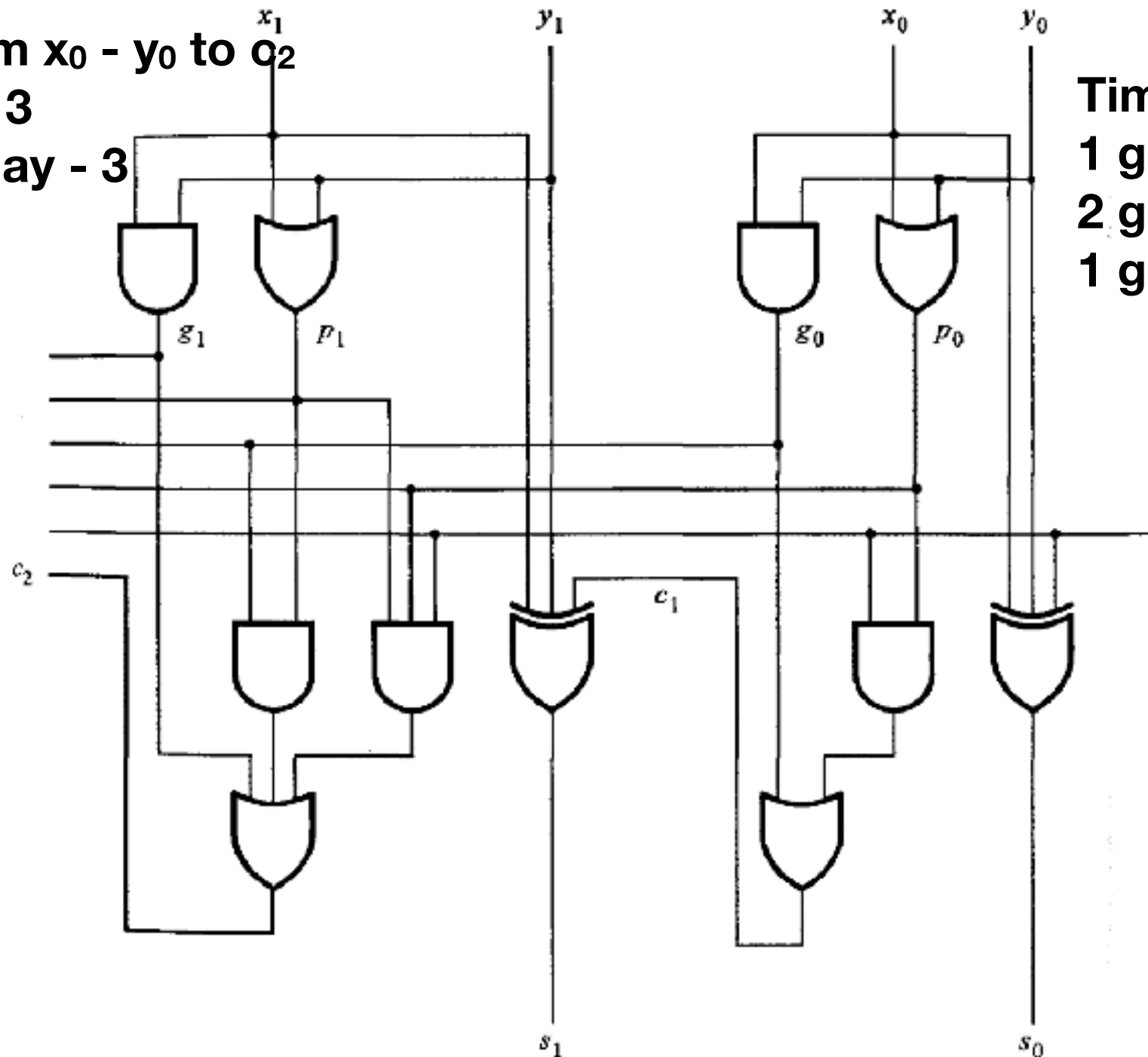
For n bits gate delay - 3

Time Delay

1 gate delay - P_i & G_i

2 gate delay - C

1 gate delay - S



Carry look Ahead Adder

Thank You !!!